Topics on Artificial Intelligence & Machine Learning

Daniel T. McGuiness, Ph.D

B.Sc
Data Science II
Lecture Book

WS.2026





WS.2026

Data Science II

Lecture Book

Daniel T. McGuiness, Ph.D

MCI

(2025, D. T. McGuines, Ph.D)

Current version is WS.2026.

This document includes the contents of Data Science II, official name being *Machine Learning and Data Science 2*, taught at MCI in the Mechatronik Design Innovation. This document is the part of the module MECH-B-5-MLDS-MLDS2-ILV taught in the B.Sc degree.

All relevant code of the document is done using *SageMath* where stated and Python v3.13.7.

This document was compiled with LuaTEX v1.22.0, and all editing were done using GNU Emacs v30.1 using AUCTEX and org-mode package.

This document is based on the following books and resources shown in no particular order:

Neural Networks: Methodology and Applications by Gérard Dreyfus , Springer *Python for Data Analysis: Data Wrangling with Pandas, Numpy, and iPython* by Wes McKinney , Springer *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* by Aurélien Géron , O'Reilly *TensorFlow for Deep Learning: From Linear Regression To Reinforcement Learning* by B. Ramsundar, and R. B. Zadeh , O'Reilly *AI and Machine Learning for Coders* by Moroney L. , O' Reilly *Neural Networks and Deep Learning* by Aggarwal S. , Springer *Python Machine Learning* by Raschka., et. al. , Packt *Machine Learning with Python Cookbook* by Albon C. , O' Reilly *CS229 Lecture Notes* by Ng A., et.al , - *Lecture Notes on Machine Learning* by Migel A., et. al , -

The document is designed with no intention of publication and has only been designed for education purposes.

The current maintainer of this work along with the primary lecturer

is D. T. McGuines, Ph.D. (dtm@mci4me.at).

Table of Contents

Part I Machine Learning Algorithms

Part II Neural Networks

Chapter 1 **Introduction to Artificial Neural Networks** **5**

1.1 From Biology to Silicon: Artificial Neurons 6
Current Developments and Overview · Biological Neurons · Logical Computations with Neurons · The Perceptron · Multilayer Perceptron and Backpropagation · Regression MLPs · Classification MLPs

1.2 Implementing MLPs with Keras 23
Building an Image Classifier Using Sequential API · Creating the model using the sequential API · Building a Regression MLP Using the Sequential API

Chapter 2 **Understanding PyTorch** **35**

2.1 PyTorch Fundamentals 36
Making some Tensors · Hardware Acceleration

2.2 Implementing Linear Regression 47
Linear Regression Using Tensors and Autograd · Linear Regression Using PyTorch's High-Level API

2.3 Implementing a Regression MLP 52

2.4 Mini-Batch Gradient Descent Using DataLoaders 54

2.5 Model Evaluation 57

2.6 Building Nonsequential Models Using Custom Modules 59
Building Models with Multiple Inputs · Building Models with Multiple Outputs

2.7 Building an Image Classifier with PyTorch 67
Using TorchVision to Load the Dataset · Building the Classifier

2.8 Fine-Tuning Hyperparameters with Optuna 73

2.9 Saving and Loading PyTorch Models 77

2.10 Compiling and Optimizing a PyTorch Model 80

Chapter 3 Training Deep Neural Networks **83**

3.1 The Vanishing/Exploding Gradients Problems 84
Glorot Initialization and He Initialization · Better Activation Functions · Batch Normalization · Layer Normalization · Gradient Clipping · Reusing Pretrained Layers · Transfer Learning with PyTorch · Unsupervised Pretraining · Pretraining on an Auxiliary Task · Faster Optimizers · AdaGrad · RMSProp · Adam

3.2 Learning Rate Scheduling 116
Exponential Scheduling · Cosine Annealing · Performance Scheduling · Warming Up the Learning Rate · Cosine Annealing with Warm Restarts · 1cycle Scheduling

3.3 Avoiding Overfitting Through Regularization 125
ℓ_1 and ℓ_2 Regularization · Dropout · Max-Norm Regularization · Practical Guidelines

Acronyms **135**

Supplemental List of Items

List of Figures

1.1 Nature is always a great source of inspiration for good and efficient design. For example, the beak of a bird is aerodynamically efficient and was used in designing the Bullet train [1].The field is of emulating models, systems, and elements of nature for the purpose of solving complex human problems is called bio mimetics [2]. . . . 5

1.2 The prolific advancements of computers and neural networks have allowed us to tackle problems once deemed impossible. A game of GO requires uncountable amount of moves, yet using ML it was possible to create a software capable of beating the world champion. 6

1.3 The original paper which defined how a network of neurons could be defined. [8] . 7

1.4 A neuron or nerve cell is an excitable cell that fires electric signals called action potentials across a neural network in the nervous system. Neurons communicate with other cells via synapses, which are specialized connections that commonly use minute amounts of chemical neurotransmitters to pass the electric signal from the presynaptic neuron to the target cell through the synaptic gap [11]. 9

1.5 A cortical column is a group of neurons forming a cylindrical structure through the cerebral cortex of the brain perpendicular to the cortical surface. The structure was first identified by Vernon Benjamin Mountcastle in 1957. He later identified minicolumns as the basic units of the neocortex which were arranged into columns. Each contains the same types of neurons, connectivity, and firing properties. Columns are also called hypercolumn, macrocolumn, functional column or sometimes cortical module. 10

1.6 Different kinds of logical operations which can be done with ANNs. 10

1.7 Threshold logic unit: an artificial neuron which computes a weighted sum of its inputs then applies a certain activation function. 11

1.8 Architecture of a perceptron with two inputs and three output neurons. 12

1.9 Architecture of a Multilayer Perceptron with five inputs, three hidden layer of four neurons, and three output neurons. 16

1.10 The activation function of a node in an Artificial Neural Networks (ANN) is a function which calculates the output of the node based on its individual inputs and their weights. Nontrivial problems can be solved using only a few nodes if the activation function is nonlinear [25]. Modern activation functions include the smooth version of the ReLU, the GELU, which was used in the 2018 BERT model [26], the logistic (sigmoid) function used in the 2012 speech recognition model developed by Hinton et al [27], the ReLU used in the 2012 AlexNet computer vision model [28] and in the 2015 ResNet model. 19
1.11 A random collection of dataset, making the Fashion Modified National Institute of Standards and Technology (MNIST). 24
1.12 An example of a data within the Fashion MNIST. 24
1.13 The plot of the neural network, showcasing its layers. 26
1.14 Learning curves: the mean training loss and accuracy measured over each epoch, and the mean validation loss and accuracy measured at the end of each epoch. . 30
1.15 Correctly classified Fashion MNIST images. 32

2.1 Wide & Deep neural network 59
2.2 Handling multiple inputs 61
2.3 Handling multiple outputs, in this example to add an auxiliary output for regularization 64

3.1 Sigmoid activation function saturation. 85
3.2 Leaky ReLU: like ReLU, but with a small slope for negative values. 90
3.3 ELU and SELU activation functions. 91
3.4 GELU, Swish, parametrized Swish, Mish, and ReLU^2 activation functions 93
3.5 Reusing pretrained layers 103
3.6 Greedy layer-wise pretraining used in the early days of deep learning; nowadays the unsupervised part is typically done in one shot on all the data rather than one layer at a time. Here (L) stands for locked layers. 107
3.7 Regular versus Nesterov momentum optimization: the former applies the gradients computed before the momentum step, while the latter applies the gradients computed after. 111

List of Tables

3.1 Initialization parameters for each type of activation function. 86
3.2 Starting points for training DNNs. 131

List of Examples

List of Exercises

List of Theorems



Part I

Machine Learning Algorithms

Part Contents

It is a capital mistake to theorize before one has data. Insensibly one begins to twist facts to suit theories, instead of theories to suit facts.

(Sir Arthur Conan Doyle, Sherlock Holmes)



Part II

Neural Networks

Part Contents

Chapter 1 Introduction to Artificial Neural Networks 5

Chapter 2 Understanding PyTorch 35

Chapter 3 Training Deep Neural Networks 83

Acronyms 135

There must be a trick to the train of thought, a recursive formula. A group of neurons starts working automatically, sometimes without external impulse. It is a kind of iterative process with a growing pattern. It wanders about in the brain, and the way it happens must depend on the memory of similar patterns.

(Stanislaw M. Ulam, Adventures of a Mathematician)

Chapter 1

Introduction to Artificial Neural Networks

Table of Contents

1.1 From Biology to Silicon: Artificial Neurons 6
1.2 Implementing MLPs with Keras 23

It is quite apparent that life imitates life and engineers are inspired by nature.

It seems only logical, then, to look at the brain's architecture for inspiration on how to build an intelligent machine as it is the only intelligence that we know of.

This is the logic that sparked ANNs, which are Machine Learning (ML) models inspired by the networks of **biological neurons** found in our brains. However, whereas planes were inspired by birds, they don't have to flap their wings to fly. Similarly, ANNs have gradually become quite different from their biological cousins. Some researchers even argue that we should drop the biological analogy altogether such as calling them **units** rather than **neurons** [3], as some consider this naming to decrease the amount of creativity we can give to the topic.

ANNs are at the very core of **deep learning**. They are versatile, powerful, and scalable, making them ideal to tackle large and highly complex ML tasks such as classifying billions of images,[1] powering speech recognition services, recommending the best videos to watch to hundreds of millions of users

[1] Such as google images.



Figure 1.1 Nature is always a great source of inspiration for good and efficient design. For example, the beak of a bird is aerodynamically efficient and was used in designing the Bullet train [1].The field is of emulating models, systems, and elements of nature for the purpose of solving complex human problems is called bio mimetics [2].



Figure 1.2 The prolific advancements of computers and neural networks have allowed us to tackle problems once deemed impossible. A game of GO requires uncountable amount of moves, yet using ML it was possible to create a software capable of beating the world champion.

every day, or learning to beat the world champion at the game of Go.[2]

[2] Such as DeepMind's AlphaGo [4].

We will treat this chapter as a formal introduction to ANN, starting with a tour of the very first ANN architectures and leading up to **multilayer perceptrons**, which are heavily used today.

In the second part, we will look at how to implement ANNs using TensorFlow's Keras Application-Programming Interface (API). This is a beautifully designed and simple high-level API for building, training, evaluating, and running ANN. While it may look simple at first glance, it is expressive and flexible enough to let you build a wide variety of neural network architectures.

For most of your use cases, using `keras` will be enough.

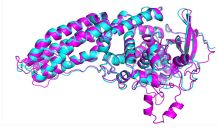
Alpha Fold — 1.1 Information

AlphaFold is an artificial intelligence (AI) program developed by DeepMind, which performs predictions of protein[3] structure. It is designed using deep learning techniques [5].

AlphaFold 1 (2018) placed 1st in the overall rankings of the 13th CASP[4] in December 2018. It was particularly successful at predicting the most accurate structures for targets rated as most difficult by the competition organizers, where no existing template structures were available from proteins with partially similar sequences.

AlphaFold 2 (2020) repeated this placement in the CASP14 competition in November 2020. It achieved a level of accuracy much higher than any other entry [6]. It scored above 90 on CASP's global distance test (GDT) for approximately two-thirds of the proteins, a test measuring the similarity between a computationally predicted structure and the experimentally determined structure, where 100 represents a complete match [5]. The inclusion of **metagenomic** data has improved the quality of the prediction of multiple sequence alignments. One of the biggest sources of the training data was the custom-built **Big Fantastic Database** of 65,983,866 protein families, represented as multiple sequence alignments and **Hidden Markov** models, covering 2,204,359,010 protein sequences from reference databases, metagenomes, and metatranscriptomes [7].

[4] Critical Assessment of Structure Prediction.

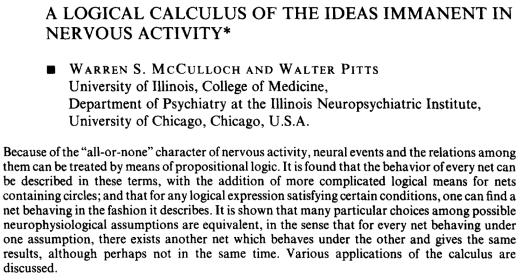


[3] Protein folding is the physical process by which a protein, after synthesis by a ribosome as a linear chain of amino acids, changes from an unstable random coil into a more ordered three-dimensional structure. This structure permits the protein to become biologically functional or active.

Understanding and simulating the protein folding process has been an important challenge for computational biology since the late 1960s.

1.1 From Biology to Silicon: Artificial Neurons

While it may seem ANNs are the cutting edge in ML, They have been around for quite a while: they were first introduced back in 1943 by the *neurophysiologist* Warren McCulloch[5] and the mathematician Walter Pitts. In their landmark paper *A Logical Calculus of Ideas Immanent in Nervous Activity*, They presented a simplified computational model of how biological neurons might work together in animal brains to perform complex computations using propositional logic.



A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY*

WARREN S. MCCULLOCH AND WALTER PITTS
University of Illinois, College of Medicine,
Department of Psychiatry at the Illinois Neuropsychiatric Institute,
University of Chicago, Chicago, U.S.A.

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the calculus are discussed.

Figure 1.3: The original paper which defined how a network of neurons could be defined. [8]

[5] **Warren Sturgis McCulloch** *(1898 - 1969)* was an American neurophysiologist and cybernetician known for his work on the foundation for certain brain theories and his contribution to the **cybernetics** movement. Along with *Walter Pitts*, McCulloch created computational models based on mathematical algorithms called threshold logic which split the inquiry into two distinct approaches, one approach focused on biological processes in the brain and the other focused on the application of neural networks to artificial intelligence [8].

> This was the first ANN architecture [8].

Since then, many other architectures have been invented. The early successes of ANNs led to the widespread belief that we would soon be conversing with truly intelligent machines. When it became clear in the 1960s that this promise would go unfulfilled,[6] funding was diverted elsewhere, and ANNs entered a long winter. This is also known as the 1st AI winter [9]. In the early 1980s, new architectures were invented and better training techniques were developed, sparking a revival of interest in connectionism, the study of ANNs. But progress was slow, and by the 1990s other powerful ML techniques had been invented, such as Support Vector Machines (SVM). These techniques seemed to offer better results and stronger theoretical foundations than ANNs, so once again the study of ANNs was put on hold and this is known as the 2nd AI winter.

[6] at least for quite a while.

> *We are now witnessing yet another strong wave of interest in ANNs. Will this wave die out like the previous ones did?*

Well, here are a few good reasons to believe that this time is different and that the renewed interest in ANNs will have a much more profound impact on our lives:

There is now a **huge quantity of data available** to train ANNs, and ANNs frequently outperform other ML techniques on very large and complex problems. One of the major turning points of ANN was the fundamental question of:

> *Is our understanding of the model at fault or is it merely the lack of data to train?*

The tremendous increase in computing power since the 1990s now makes it possible to train large ANNs in a reasonable amount of time. This is in part due to **Moore's law**,[7] but also thanks to the gaming industry, which has stimulated the production of powerful GPU cards by the millions which have become the norm to train ML instead of CPUs.

[7] The observation that the number of transistors in an Integrated Circuit (IC) doubles about every two years. Moore's law is an observation and projection of a historical trend. Rather than a law of physics, it is an empirical relationship.

In addition to previous statements, cloud platforms have made this power accessible to everyone. The training algorithms have been improved. From a realistic standpoint, they are only slightly different from the ones used in the 1990s, but these relatively small tweaks have had a huge positive

impact.

Some theoretical limitations of ANNs have turned out to be benign in practice. For example, many people thought that ANN training algorithms were doomed because they were likely to get stuck in local optima [10], but it turns out that this is **NOT** a big problem in practice, especially for larger ANNs:

> The local optima often perform almost as well as the global optimum.

1.1.1 Current Developments and Overview

ANNs seem to have entered a virtuous circle of funding and progress. Amazing products based on ANNs regularly make the headline news, which pulls more and more attention and funding toward them, resulting in more and more progress and even more amazing products.

- There is now a *huge quantity of data available to train neural networks*, and ANNs frequently outperform other ML techniques on very large and complex problems.
- The tremendous increase in computing power since the 1990s now makes it possible to train large neural networks in a reasonable amount of time. This is in part due to Moores law,[8] but also thanks to the gaming industry, which has stimulated the production of powerful graphical processing units (GPUs) by the millions:

  > GPU cards were initially designed to accelerate graphics, but it turns out that neural networks perform similar computations (such as large matrix multiplications), so they can also be accelerated using GPUs.

  Moreover, cloud platforms have made this power accessible to everyone.
- The training algorithms have been improved. To be fair they are only slightly different from the ones used in the 1990s, but these relatively small tweaks have had a huge positive impact.
- Some theoretical limitations of ANNs have turned out to be benign in practice. For example, many people thought that ANN training algorithms were doomed because they were likely to get stuck in local optima, but it turns out that this is not a big problem in practice, especially for larger neural networks: the local optima often perform almost as well as the global optimum.
- The invention of the Transformer architecture in 2017 (see Chapter 15) has been a game changer: it can process and generate all sorts of data (e.g., text, images, audio) unlike earlier, more specialized, architectures, and it performs great across a wide variety of tasks from robotics to protein folding. Moreover, it scales rather well, which has made it possible to train very large foundation models that can be reused across many different tasks, possibly with a bit of fine-tuning (thats transfer learning), or just by prompting the model in the right way (thats in-context learning, or ICL). For instance, you can give it a few examples of the task at hand

[8] the number of components in integrated circuits has doubled about every 2 years over the last 50 years.

(thats few-shot learning, or FSL), or ask it to reason step-by-step (thats chain-of-thought prompting, or CoT). Its a new world!

- ANNs seem to have entered a virtuous circle of funding and progress. Amazing products based on ANNs regularly make the headline news, which pulls more and more attention and funding toward them, resulting in more and more progress and even more amazing products. AI is no longer just powering products in the shadows: since chatbots such as ChatGPT were released, the general public is now directly interacting daily with AI assistants, and the big tech companies are competing fiercely to grab this gigantic market: the pace of innovation is wild.

1.1.2 Biological Neurons

Before we discuss artificial neurons, let's take a quick look at its biological counterpart. It is an unusual-looking cell mostly found in animal brains which we can see in **Fig.** 1.4.

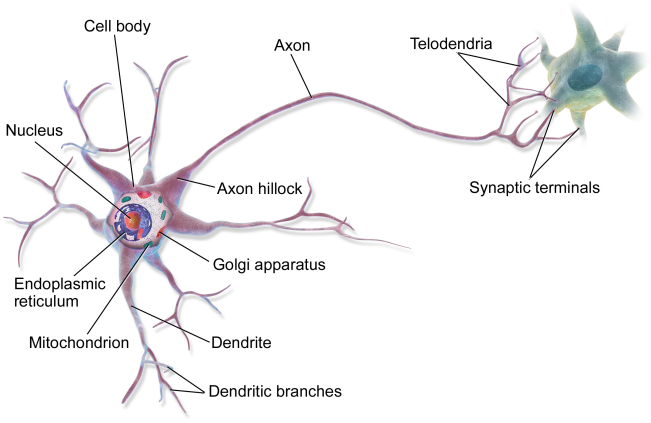


Figure 1.4
A neuron or nerve cell is an excitable cell that fires electric signals called action potentials across a neural network in the nervous system. Neurons communicate with other cells via synapses, which are specialized connections that commonly use minute amounts of chemical neurotransmitters to pass the electric signal from the presynaptic neuron to the target cell through the synaptic gap [11].

It's composed of a cell body containing the nucleus and most of the cell's complex components, many branching extensions called dendrites, plus one very long extension called the **axon**. The axon's length may be just a few times longer than the cell body, or up to tens of thousands of times longer.

Near its extremity the axon splits off into many branches called *telodendria*, and at the tip of these branches are minuscule structures called synaptic terminals,[9] which are connected to the dendrites or cell bodies of other neurons. Biological neurons produce short electrical impulses called **action potentials**, which travel along the axons and make the synapses release chemical signals called neurotransmitters. When a neuron receives a sufficient amount of these neurotransmitters within a few ms, it fires its own electrical impulses.[10]

Therefore, individual biological neurons seem to behave in a simple way, but they're organised in a vast network of billions, with each neuron typically connected to thousands of other neurons. Highly complex computations can be performed by a network of fairly simple neurons, much like a complex anthill can emerge from the combined efforts of simple ants. The architecture of biological neural networks (BNNs) is the subject of active research, but some parts of the brain have been mapped [12]. These efforts show that neurons are often organised in consecutive layers, especially in the cerebral cortex.[11]

[9] or simply synapses.

[10] Actually, it depends on the neurotransmitters, as some of them inhibit the neuron from firing.

[11] This is the outer layer of the brain.

1.1.3 Logical Computations with Neurons

McCulloch and *Pitts* proposed a very simple model of the biological neuron, which later became known as an **artificial neuron** where it has one or more binary (on/off) inputs and one binary output. The artificial neuron activates its output when more than a certain number of its inputs are active. In their paper, *McCulloch* and *Pitts* showed that even with such a simplified model it is possible to build an ANN which can compute any logical proposition you want. To see how such a network works, let's build a few ANNs which perform various logical computations, assuming a neuron is activated when at least two (2) of its input connections are active which can be seen in **Fig.** 1.6.

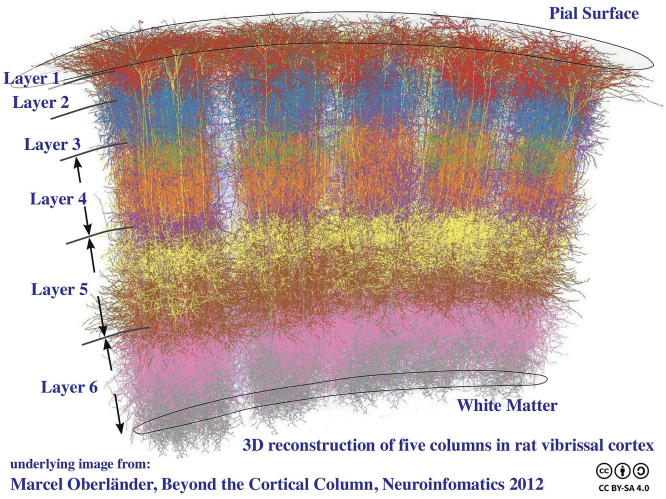


Figure 1.5
A cortical column is a group of neurons forming a cylindrical structure through the cerebral cortex of the brain perpendicular to the cortical surface. The structure was first identified by Vernon Benjamin Mountcastle in 1957. He later identified minicolumns as the basic units of the neocortex which were arranged into columns. Each contains the same types of neurons, connectivity, and firing properties. Columns are also called hypercolumn, macrocolumn, functional column or sometimes cortical module.

Let's see what these networks do:

- The first network on the left is the **identity function**: if neuron **A** is activated, then neuron **C** gets activated as well;[12] but if neuron **A** is off, then neuron **C** is off as well.
- The second network performs a logical **AND** where the neuron **C** is activated only when both neurons **A** and **B** are activated (a single input signal is not enough to activate neuron **C**).
- The third network performs a logical **OR** where neuron **C** gets activated if either neuron **A** or

[12] since it receives two input signals from neuron A

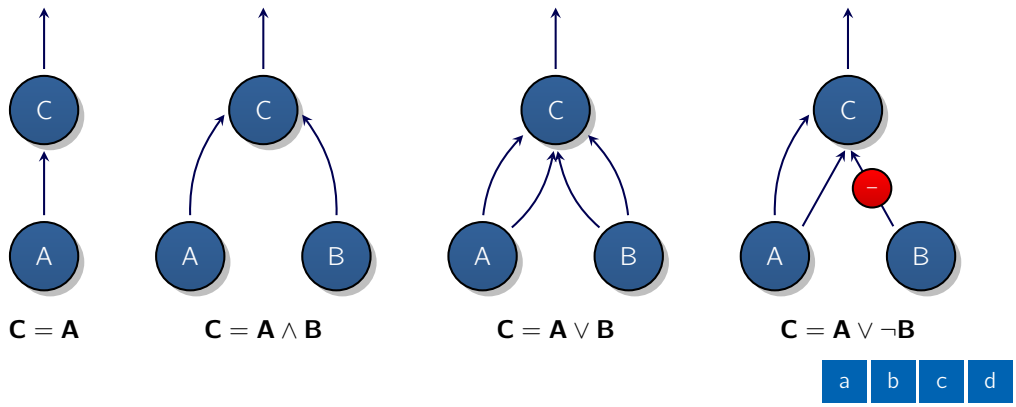


Figure 1.6
Different kinds of logical operations which can be done with ANNs.

neuron **B** is activated (or both).

- Finally, if we suppose that an input connection can inhibit the neuron's activity,[13] then the fourth network computes a slightly more complex logical proposition. Here neuron **C** is activated only if neuron **A** is active and neuron B is off. If neuron **A** is active all the time, then we get a logical **NOT**In this case, neuron **C** is active when neuron **B** is off, and vice versa.

[13] which is the case with biological neurons.

We can imagine how these networks can be combined to compute complex logical expressions.

1.1.4 The Perceptron

The perceptron is one of the simplest ANN architectures, invented in 1957 by Frank Rosenblatt[14] [13]. It is based on a slightly different artificial neuron called a Threshold Logic Unit (TLU), or sometimes a Linear Threshold Unit (LTU) which can be seen in **Fig.** 1.7. The inputs and output are numbers,[15] and each input connection is associated with a **weight**.

The first thing TLU does is to compute a linear function of its inputs:

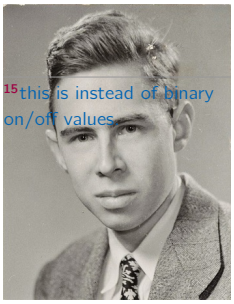
$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w}^T x + b$$

Then it applies a **step function** to the result:

$$h_w(x) = \text{step}(z) \qquad \text{where} \qquad z = \mathbf{w}^T x + b$$

For those of you who are observant, this should look familiar as it is similar to **logistic regression**,[16] except it uses a step function instead of the logistic function. Similar to logistic regression, the model parameters are the input weights w and the bias term b.

The most common step function used in perceptrons is the **Heaviside step** and sometimes the **sign**



[14] **Frank Rosenblatt** *(1928 - 1971)* was an American psychologist notable in the field of Artificial Intelligence (AI). He is sometimes called the father of deep learning for his pioneering work on ANNs.

[15] this is instead of binary on/off values

[16] A topic which was covered in **B.Sc Data Science I**.

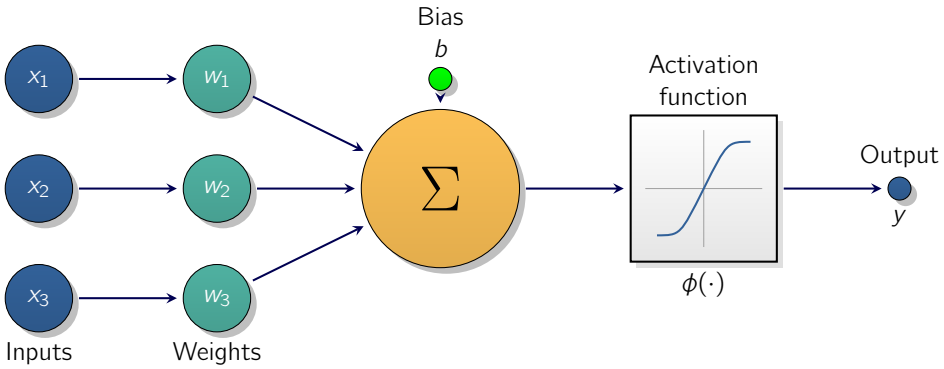


Figure 1.7 **Threshold logic unit: an artificial neuron which computes a weighted sum of its inputs then applies a certain activation function.**

function is used instead [14].

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if} \quad z < 0, \\ 1 & \text{if} \quad z \geqslant 0. \end{cases} \qquad \text{sgn}(z) = \begin{cases} -1 & \text{if} \quad z < 0, \\ 0 & \text{if} \quad z = 0, \\ +1 & \text{if} \quad z > 0, \end{cases}$$

A single TLU can be used for simple **linear binary classification**:

> It computes a linear function of its inputs, and if the result exceeds a threshold, it outputs the positive class. Otherwise, it outputs the negative class. They exhibit a similar behaviour to logistic regression or linear SVM classification.

It is possible, for example, use a single TLU to classify *iris flowers*[17] [15] based on **petal length** and **width**. Training such a TLU would require finding the right values for w_1, w_2, b.

[17] a famous dataset used by statisticians and ML researchers

A perceptron is composed of one or more TLUs organized in a single layer, where every TLU is connected to every input. Such a layer is called a **fully connected layer**, or a dense layer. The inputs constitute the input layer and since the layer of TLUs produces the final outputs, it is called the output layer.

This perceptron can classify instances simultaneously into three (**3**) different binary classes, which makes it a **multilabel classifier**. It may also be used for multiclass classification.

Using linear algebra, the following equation can be used to efficiently calculate the outputs of a layer of artificial neurons for several instances at once.

$$\hat{\mathbf{Y}} = \phi(\mathbf{XW} + \mathbf{b})$$

where:

- $\hat{\mathbf{Y}}$ is the output matrix and has one row per instance and one column per neuron.

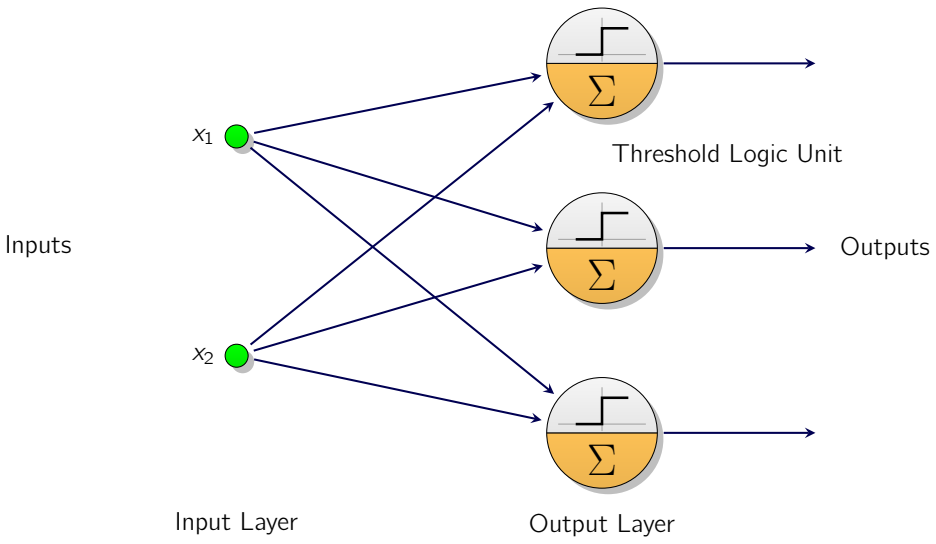


Figure 1.8 Architecture of a perceptron with two inputs and three output neurons.

- **X** is the matrix of input features and has one row per instance and one column per feature.
- **W** is the weight matrix containing all the connection weights and has one row per input and one column per neuron.
- **b** is the bias term containing all the bias terms which is one per neuron.
- ϕ is called the activation function. When the artificial neurons are TLUs, it is a **step function**.

Bending the Rules 1.2 Information

In mathematics, the sum of a matrix and a vector is **undefined**.

However, in the context of data science, we allow **broadcasting**. Here, adding a vector to a matrix means adding it to every row in the matrix. So, $\mathbf{XW} + \mathbf{b}$ first multiplies **X** by **W**, which results in a matrix with one row per instance and one column per output, which then adds the vector **b** to every row of that matrix, which **adds each bias term to the corresponding output**, for every instance.

Moreover, ϕ is then applied itemwise to each item in the resulting matrix.

Now the question is:

How is this perceptron trained?

The original perceptron training algorithm proposed by Rosenblatt was largely inspired by Hebb's rule [16]. In his 1949 book *The Organization of Behaviour*, Donald Hebb suggested that when a biological neuron triggers another neuron often, the connection between these two neurons grows stronger [17].

Siegrid Löwel later summarized Hebb's idea in the catchy phrase,

Cells that fire together, wire together.

This means the connection weight between two (2) neurons **tends to increase** when they fire simultaneously.

This rule later became known as Hebb's rule[18] (or Hebbian learning [18])

Perceptrons are trained using a variant of this rule that takes into account the error made by the network when it makes a prediction. The perceptron learning rule **reinforces connections that help reduce the error**.

More specifically, the perceptron is fed one training instance at a time, and for each instance it makes its predictions. For every output neuron that produced a wrong prediction, it reinforces the

[18] The following is an excerpt from primary author of the idea.

Let us assume that the persistence or repetition of a reverberatory activity (or "trace") tends to induce lasting cellular changes that add to its stability. . . . When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased. [17]

connection weights from the inputs that would have contributed to the correct prediction.

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta \left(y_j - \hat{y}_j \right) x_i x$$

where:

- $w_{i,j}$ is the connection weight between the ith input and the jth neuron.
- x_i is the ith input value of the current training instance.
- $\hat{y}^j$ is the output of the jth output neuron for the current training instance.
- y^j is the target output of the jth output neuron for the current training instance.
- η is the learning rate.

The decision boundary of each output neuron is **linear**, therefore perceptrons are incapable of learning complex patterns. However, if the training instances are **linearly separable**, Rosenblatt demonstrated that this algorithm would converge to a solution.

> This is called the perceptron convergence theorem.

python

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 0) # Iris setosa

per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)
X_new = [[2, 0.5], [3, 1]]
# predicts True and False for these 2 flowers
y_pred = per_clf.predict(X_new)

print(y_pred)
```

For those of you who have taken a **Data Science I** course by yours truly, you may have noticed that the perceptron learning algorithm strongly resembles *stochastic gradient descent*. In fact, `sklearn`'s `Perceptron` class is equivalent to using an `SGDClassifier` with the following hyperparameters:

- `loss="perceptron"`,
- `learning_rate="constant"`,
- `eta0=1`, the learning rate
- `penalty=None`, no regularization.

In their 1969 monograph Perceptrons, Marvin Minsky and Seymour Papert highlighted a number of

serious weaknesses of perceptrons: in particular, they are incapable of solving some trivial problems (e.g., the exclusive OR (XOR) classification problem).

XOR Problem 1.3 Information

A simple logic gate problem which is proven to be unsolvable using a single-layer perceptron.

This is true of any other linear classification model, but researchers had expected much more from perceptrons, and some were so disappointed, they dropped neural networks altogether in favour of higher-level problems such as logic, problem solving, and search. The lack of practical applications also didn't help.

It turns out that some of the limitations of perceptrons can be eliminated by stacking multiple perceptrons. The resulting ANN is called a Multi-layer Perceptrons (MLP) and a MLP can solve the XOR problem [19].

Limitations of Perceptrons 1.4 Information

Perceptrons do **NOT** output a class probability. This is one reason to prefer logistic regression over perceptrons. Moreover, perceptrons do **NOT** use any regularization by default, and training stops as soon as there are no more prediction errors on the training set, so the model typically does not generalize as well as logistic regression or a linear SVM classifier. However, perceptrons may train a bit faster.

1.1.5 Multilayer Perceptron and Backpropagation

An MLP is composed of one input layer, one or more layers of TLUs called **hidden layers**, and one final layer of TLUs called the output layer. The layers close to the input layer are usually called the lower layers, and the ones close to the outputs are usually called the upper layers.

The signal flows only in one direction,[19] so this architecture is an example of a Feedforward Neural Networks (FNN) [20].

[19] From input to output.

When an ANN contains a deep stack of hidden layers, it is called a Deep Neural Networks (DNN). The field of deep learning studies DNNs, and more generally it is interested in models containing deep stacks of computations [21].

For many years researchers struggled to find a way to train MLPs, without success. In the early 1960s several researchers discussed of using **gradient descent** to train ANNs. This requires computing the gradients of the model's error with regard to the model parameters and at that time, it wasn't clear at the time how to do this efficiently with such a complex model containing so many parameters.

Then, in 1970, a researcher named *Seppo Linnainmaa* introduced in his master's thesis a technique to compute all the gradients automatically and efficiently. This algorithm is now called **reverse-mode**

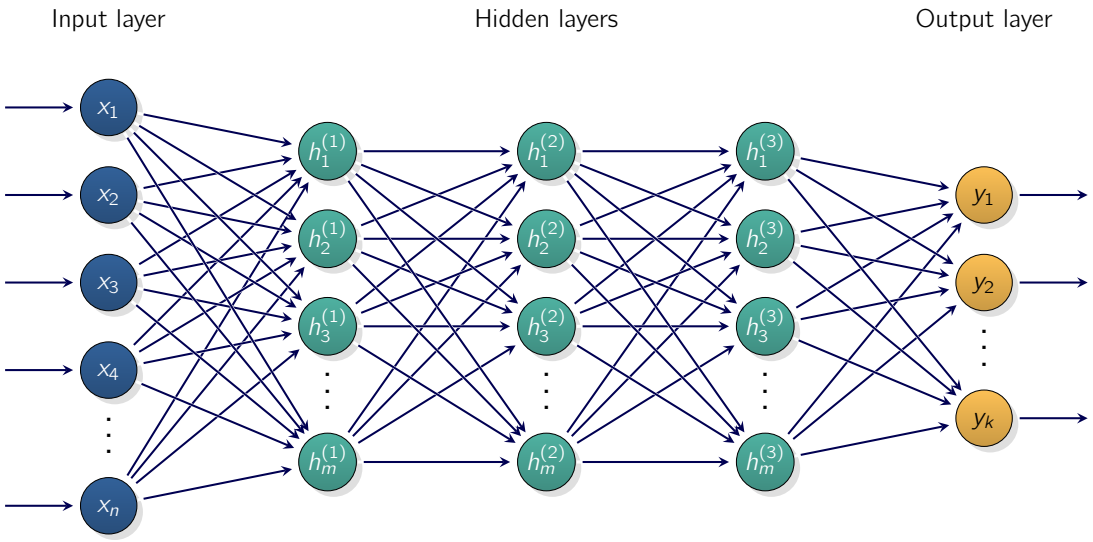


Figure 1.9
Architecture of a Multilayer Perceptron with five inputs, three hidden layer of four neurons, and three output neurons.

automatic differentiation [22]. In just two (**2**) passes through the network,[20] it is able to compute the gradients of the neural network's error with regard to every single model parameter.

[20] one forward, one backward.

To put it simply:

> It can find out how each connection weight and each bias should be tweaked to reduce the ANNs error.

These gradients can then be used to perform a gradient descent step. Repeating the process of computing the gradients automatically and taking a gradient descent step, the neural network's error will gradually drop until it eventually reaches a minimum.

This combination of reverse-mode automatic differentiation and gradient descent is now called **backpropagation** [23].

> There are various automatic differentiation techniques (i.e., forward and reverse), with each having its own advantages and disadvantages. Reverse-mode autodiff is well suited when the function to differentiate has many variables (e.g., connection weights and biases) and few outputs (e.g., one loss).

Backpropagation can actually be applied to all sorts of computational graphs, not just neural networks: Linnainmaa's M.Sc thesis was not about neural nets, it was more general. It was several more years before backprop started to be used to train neural networks, but it still wasn't mainstream.

Then, in 1986, David Rumelhart, Geoffrey Hinton, and Ronald Williams published a groundbreaking paper analyzing how backpropagation allowed neural networks to learn useful internal representations [24]. Their results were so impressive that backpropagation was quickly popularized in the field.

Today, it is by far the most popular training technique for Neural Networks (NN)s.

Let's run through how backpropagation works again in a bit more detail:

1. It handles one mini-batch at a time, and goes through the full training set multiple times. Each pass is called an **epoch**.
2. Each mini-batch enters the network through the input layer. The algorithm then computes the output of all the neurons in the first hidden layer, for every instance in the mini-batch. The result is passed on to the next layer, its output is computed and passed to the next layer, and so on until we get the output of the last layer, the output layer.
3. Next, the algorithm measures the network's output error. This means, it uses a loss function that compares the desired output and the actual output of the network, and returns some measure of the error.
4. It then computes how much each output bias and each connection to the output layer contributed to the error. This is done analytically by applying the chain rule, which makes this step fast and precise.
5. The algorithm then measures how much of these error contributions came from each connection in the layer below, again using the chain rule, working backward until it reaches the input layer. As explained earlier, this reverse pass efficiently measures the error gradient across all the connection weights and biases in the network by propagating the error gradient backward through the network.
6. Finally, the algorithm performs a gradient descent step to tweak all the connection weights in the network, using the error gradients it just computed.

Initialize all the hidden layers' connection weights **randomly**, or training will fail.

For example, if we were to initialize all weights and biases to zero, then all neurons in a given layer will be perfectly **identical**, and therefore backpropagation will affect them in exactly the same way, so they will remain identical.

In other words, despite having hundreds of neurons per layer, our model will act as if it had only one neuron per layer and therefore it won't act too smart. If we instead randomly initialize the weights, we **break the symmetry** and allow back-propagation to train a diverse team of neurons.

In short, backpropagation makes predictions for a mini-batch (forward pass), measures the error, then goes through each layer in reverse to measure the error contribution from each parameter (reverse pass), and finally tweaks the connection weights and biases to reduce the error, which is the gradient descent step.

For back-propagation to work properly, Rumelhart and his colleagues made a key change to the

MLP's architecture by replacing the step function with the logistic function:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

Which is also called the **sigmoid function**. This was an important improvement as step function contains only flat segments, so there is no gradient to work with, while the *sigmoid* function has a well-defined nonzero derivative everywhere. In fact, the backpropagation algorithm works well with many other activation functions, not just the sigmoid function.

Here are two (2) other popular choices:

Other Activation Functions — 1.5 Information

Hyperbolic tangent function

$$\tanh(z) = 2\sigma(2z) - 1$$

Similar sigmoid function, is also S-shaped, **continuous**, and **differentiable**, but its output value ranges from -1 to 1, instead of 0 to 1 like the sigmoid function.

This bigger range tends to make each layer's output more or less centered around 0 at the beginning of training, which often helps speed up convergence.

The rectified linear unit function

$$\text{ReLu}(z) = \max(0, z)$$

It is continuous bu **NOT** differentiable at $z = 0$ as the slope changes abruptly, which can make gradient descent bounce around, and its derivative is 0 for $z < 0$. In practice, however, it works very well and has the advantage of being fast to compute, so it has become the default.

Importantly, the fact that it does **NOT** have a maximum output value helps reduce some issues during gradient descent.

With all these information you might wonder:

> What is the point of an activation function, let alone whether it is linear or not?

Chaining several linear transformations, gives you only linear transformation.

> For example:

$$f(x) = 2x + 3 \quad \text{and} \quad g(x) = 5x - 1 \quad \rightarrow \quad f\big(g(x)\big) = 2(5x - 1) + 3 = 10x + 1$$

As this example shows, we don't have some nonlinearity between layers, then even a deep stack of layers is equivalent to a single layer, and we can't solve very complex problems with that.

A large DNN with non-linear activations can theoretically approximate any continuous function.

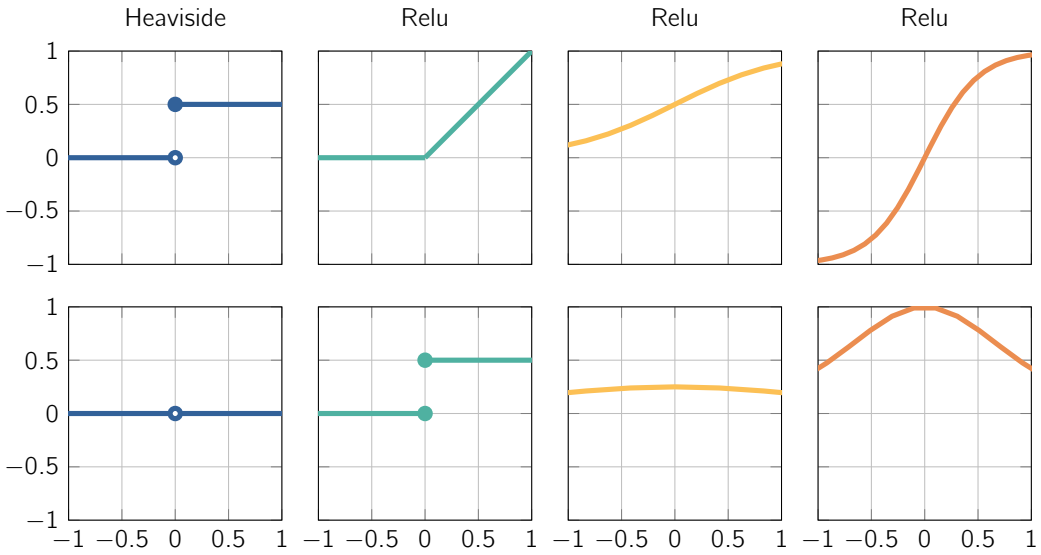


Figure 1.10 The activation function of a node in an ANN is a function which calculates the output of the node based on its individual inputs and their weights. Nontrivial problems can be solved using only a few nodes if the activation function is nonlinear [25]. Modern activation functions include the smooth version of the ReLU, the GELU, which was used in the 2018 BERT model [26], the logistic (sigmoid) function used in the 2012 speech recognition model developed by Hinton et al [27], the ReLU used in the 2012 AlexNet computer vision model [28] and in the 2015 ResNet model.

1.1.6 Regression MLPs

First, MLPs can be used for **regression tasks**. If we want to predict a single value,[21] we just need a single output neuron, where its output is the predicted value.

[21] e.g., the price of a house, given many of its features.

For multivariate regression,[22] we need one output neuron per output dimension.

[22] i.e., to predict multiple values at once.

> As an example, to locate the center of an object in an image, we need to predict 2D coordinates, so we need two (**2**) output neurons. If you also want to place a bounding box around the object, then we need two more numbers which are the `width` and the `height` of the object. So, in the end you end up with four (**4**) output neurons.

`sklearn` includes an `MLPRegressor` class, so let's use it to build an MLP with three (**3**) **hidden layers** composed of 50 neurons each, and train it on the California housing dataset.

For simplicity, we will use `sklearn`'s `fetch_california_housing()` function to load the data instead of downloading from a sketchy website.

python

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, random_state=42)

```

Continues on next page →

← Continued from previous page

```python
mlp_reg = MLPRegressor(hidden_layer_sizes=[50, 50, 50], random_state=42)
pipeline = make_pipeline(StandardScaler(), mlp_reg)
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_valid)
rmse = root_mean_squared_error(y_valid, y_pred)

print(rmse)
```

The following code starts by fetching and splitting the dataset, then it creates a pipeline to standardize the input features before sending them to the `MLPRegressor`.

> This is very important for ANN as they are trained using **gradient descent**, and gradient descent does **NOT** converge very well when the features have very different scales.

Finally, the code trains the model and evaluates its validation error. The model uses the ReLU activation function in the hidden layers, and it uses a variant of gradient descent called Adam[23] to minimize the mean squared error, with a little bit of ℓ_2 regularisation:[24]

We get a validation RMSE of about `0.505`, which is comparable to what we would get with a random forest classifier.

> This MLP does **NOT** use any activation function for the output layer, so it's free to output any value it wants.

This is generally fine, but if we want to guarantee the output will always be positive, then we should use the ReLU activation function in the output layer, or the **softplus activation** function, which is a smooth variant of ReLU.

Softplus Activation Function — 1.6 Information

The softplus function is

$$f(x) = \ln(1 + \exp(z)).$$

It is a **smooth approximation** (in fact, an analytic function) to the ramp function, which is known as the rectifier or ReLU (rectified linear unit) in ML. For large negative x it is

$$\ln(1 + \exp(x)) = \ln(1 + \epsilon) \gtrapprox \ln 1 = 0,$$

whereas for large positive x it is

$$\ln(1 + \exp(x)) \gtrapprox \ln(\exp(x)) = x$$

which is just above x.

Finally, if you want to guarantee that the predictions will always fall within a given range of values, then you should use the sigmoid function or the hyperbolic tangent, and scale the targets to the appropriate range: 0 to 1 for sigmoid and -1 to 1 for tanh.

Sadly, the `MLPRegressor` class does not support activation functions in the output layer.

[23] An optimization algorithm which improves upon gradient descent by using **adaptive learning rates** for each parameter, combining the benefits of momentum and **RMSprop** [29]. It calculates both the first moment, mean of gradients, and the second moment, uncentered variance of gradients, to adapt the learning rate during training. This makes it computationally efficient, suitable for large datasets, and effective for non-stationary or sparse problems.

[24] We can control its strength via the `alpha` hyperparameter, which defaults to `0.0001`.

Building and training a standard MLP with `sklearn` is very convenient, but features are limited. This is why we will switch to Keras in the second part of this chapter.

The `MLPRegressor` class uses the Mean Square Error (MSE), which is usually what we want for regression, but if we have a lot of outliers in the training set, we may prefer to use the Mean Absolute Error (MAE) instead. Alternatively, we may want to use the Huber loss, which is a combination of both. It is quadratic when the error is smaller than a threshold δ (typically 1) but linear when the error is larger than δ.

The linear part makes it less sensitive to outliers than the mean squared error, and the quadratic part allows it to converge faster and be more precise than the mean absolute error. However, `MLPRegressor` only supports the MSE.

1.1.7 Classification MLPs

MLPs can also be used for **classification** tasks. For a **binary classification** problem, we just need a single output neuron using the *sigmoid* activation function:

> The output will be a number between 0 and 1, which we can interpret as the estimated probability of the positive class.

The estimated probability of the negative class is equal to **one minus that number**.

MLPs can also easily handle multi-label binary classification tasks.

> For example, we could have an email classification system which predicts whether each incoming email is **ham** or **spam**, and simultaneously predicts whether it is an **urgent** or **nonurgent** email.

Here, we would need two (**2**) output neurons, both using the sigmoid activation function where:

- the first would output the probability that the email is spam, and
- the second would output the probability that it is urgent.

More generally, we would dedicate one output neuron for each positive class.

Note the output probabilities do **NOT** necessarily add up to 1.

This lets the model output any combination of labels where we can have nonurgent ham, urgent ham, nonurgent spam, and perhaps even urgent spam.[25]

If each instance can belong only to a single class, out of three or more possible classes,[26] then we need to have one output neuron per class, and we should use the *softmax* activation function for

[25] However, this would probably be an error.

[26] e.g., classes 0 through 9 for digit image classification.

the whole output layer. The softmax function will ensure all the estimated probabilities are between 0 and 1 and that they add up to 1, given the classes are exclusive.[27]

Regarding the loss function, as we are predicting probability distributions, the cross-entropy loss[28] is generally a good choice.

`sklearn` has an `MLPClassifier` class in the `sklearn.neural_network` package. It is almost identical to the `MLPRegressor` class, except that it minimises the cross entropy rather than the MSE.

[27] This is called **multiclass classification** [30].

[28] in literature this is also known as or **x-entropy** or **log loss** for short.

1.2 Implementing MLPs with Keras

Keras is TensorFlow's high-level deep learning API. It allows you to build, train, evaluate, and execute all sorts of neural networks. The original Keras library was developed by *Francois Chollet* as part of a research project and was released as a standalone open source project in March 2015. It quickly gained popularity, owing to its ease of use, flexibility, and beautiful design.

Application 1.7 Information

Keras used to support multiple backends, including TensorFlow and PlaidML. but since version 2.4, Keras is **TensorFlow-only**. Similarly, TensorFlow used to include multiple high-level APIs, but Keras was officially chosen as its preferred high-level API when TensorFlow 2 came out.

Installing TensorFlow will automatically install Keras as well, and Keras will **NOT** work without TensorFlow installed.

Other popular deep learning libraries include PyTorch by Facebook and JAX by Google.13

1.2.1 Building an Image Classifier Using Sequential API

Before we begin doing anything, we need to **load a dataset**.

We will use Fashion MNIST. There are 70,000 grayscale images of 28×28 pixels each, with 10 classes where images represent fashion items rather than handwritten digits, so each class is more diverse, and the problem turns out to be significantly challenging.

Using Keras to load the dataset

The python library, `keras` provides utility functions to fetch and load common datasets, including MNIST, Fashion MNIST, and a few more.

Let's load Fashion MNIST. It's already shuffled and split into a training set (60,000 images) and a test set (10,000 images), but we'll hold out the last 5,000 images from the training set for validation:

python

```
import tensorflow as tf

fashion_mnist = tf.keras.datasets.fashion_mnist.load_data()
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist
X_train, y_train = X_train_full[:-5000], y_train_full[:-5000]
X_valid, y_valid = X_train_full[-5000:], y_train_full[-5000:]
```

TensorFlow is usually imported as `tf`, and the Keras API is available via `tf.keras`.



Figure 1.11
A random collection of dataset, making the Fashion MNIST.

Data Representation in MNIST — 1.8 Information

When loading MNIST or Fashion MNIST using `tf.keras` rather than `sklearn`, an important difference is that every image is represented as a 28×28 array rather than a 1D array of size 784 with intensities are represented as integers (0 - 255) rather than floats (0.0 - 255.0).

Let's take a look at the shape and data type of the training set:

```python
print("The size of the training dataset: ", X_train.shape)
print("The type of the training dataset: ", X_train.dtype)
```

```text
The size of the training dataset:  (11610, 8)
The type of the training dataset:  float64
```

To make it simple, we'll scale the pixel intensities down to the 0 - 1 range by dividing them by 255.0. This operation also converts the integer values to floats.



Figure 1.12
An example of a data within the Fashion MNIST.

```python
X_train, X_valid, X_test = X_train / 255., X_valid / 255., X_test / 255.
```

Using Fashion MNIST, we need the list of class names to know what we are dealing with:

```python
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

For example, the first image in the training set represents an ankle boot, shown in **Fig.** 1.6, and we can see some samples of the Fashion MNIST datasets in **Fig.** 1.11.

1.2.2 Creating the model using the sequential API

It is time to build our first ANN. Here is a classification MLP with two (2) hidden layers:

```python
tf.random.set_seed(42)
model = tf.keras.Sequential()
model.add(tf.keras.layers.InputLayer(shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(300, activation="relu"))
model.add(tf.keras.layers.Dense(100, activation="relu"))
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

Let's try to understand the code:

`tf.random.set_seed(42)`
: Here we set `tf` random seed to make the results reproducible. This is to make sure the random weights of the hidden layers and the output layer will be the same every time we run our code.

  We could also choose to use the `tf.keras.utils.set_random_seed()` function, which conveniently sets the random seeds for TensorFlow.

`model = tf.keras.Sequential()`
: This line creates a *Sequential model*. This is the simplest kind of Keras model for ANN, composed of a single stack of layers connected sequentially.[29]

[29] This is called the sequential API.

`model.add(tf.keras.layers.InputLayer(shape=[28, 28]))`
: Now we build the first layer (an Input layer) and add it to our model. We specify the input shape, which doesn't include the batch size, only the shape of the instances. Keras needs to know the shape of the inputs so it can determine the shape of the connection weight matrix of the first hidden layer.

`model.add(tf.keras.layers.Flatten())`
: Here we add a Flatten layer. Its role is to convert each input image into a 1D array

  For example, if the layer receives a batch of shape $32 \times 28 \times 28$, it will reshape it to 32×784. In other words, if it receives input data `X`, it computes `X.reshape(-1, 784)`.

  This layer doesn't have any parameters as it's just there to do some simple pre-processing.

`model.add(tf.keras.layers.Dense(300, activation="relu"))`
: We add a Dense hidden layer with 300 neurons. It will use the ReLU activation function. Each Dense layer manages its own weight matrix, containing all the connection weights between the neurons and their inputs. It also manages a vector of bias terms.[30]

[30] This is one per neuron.

`model.add(tf.keras.layers.Dense(100, activation="relu"))`
: We add a second Dense hidden layer with 100 neurons, also using the ReLU activation function.

`model.add(tf.keras.layers.Dense(10, activation="softmax"))`
: We add a Dense output layer with 10 neurons,[31] using the softmax activation function as the classes are exclusive.

[31] one per class.

Writing `activation="relu"` is same as `activation=tf.keras.activations.relu`.[32]

[32] Other activation functions are available in `tf.keras.activations` package.

Instead of adding the layers one by one as we just did, it's often more convenient to pass a list of layers when creating the Sequential model. You can also drop the Input layer and instead specify the `input_shape` in the first layer:

```python
tf.keras.backend.clear_session()
tf.random.set_seed(42)

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input([28, 28])),
    tf.keras.layers.Dense(300, activation="relu"),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

The model's `summary()` method displays all the model's layers, including each layer's name, which is automatically generated, its output shape, and its number of parameters. The summary ends with the total number of parameters, including **trainable** and **non-trainable** parameters. Here we only have trainable parameters:

```python
tf.keras.utils.plot_model(model, imagePath+"mnist-_model.pdf", show_shapes=True)
```

Dense layers often have a lot of parameters.

> For example, the first hidden layer has 784×300 connection weights, with 300 bias terms, which adds up to 235,500 parameters.

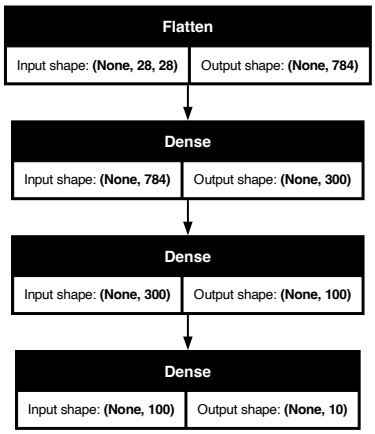


Figure 1.13 The plot of the neural network, showcasing its layers.

This gives the model quite a lot of flexibility to fit the training data, but it also means the model runs the risk of **over-fitting**, especially when we do **NOT** have a lot of training data.

Each layer in a model must have a unique name.[33] We can set the layer names explicitly using the constructor's name argument, but generally it's simpler to let Keras name the layers automatically, as we just did. Keras takes the layer's class name and converts it to snake case.[34] Keras also ensures that the name is **globally unique**, even across models, by appending an index if needed, as in `dense_2`.

[33] For example, we can call a layer `dense_2`.

[34] i.e., a layer from the `MyCoolLayer` class is named `my_cool_layer` by default.

This naming scheme makes it possible to merge models easily without getting name conflicts.

All global state managed by Keras is stored in a Keras session, which you can clear using `tf.keras.backend.clear_session()`.

We can easily get a model's list of layers using the layers attribute, or use the `get_layer()` method to access a layer by name:

```python
print(model.layers)
```

```text
[<Flatten name=flatten, built=True>,
<Dense name=dense, built=True>,
<Dense name=dense_1, built=True>,
<Dense name=dense_2, built=True>]
```

All the parameters of a layer can be accessed using its `get_weights()` and `set_weights()` methods. For a Dense layer, this includes both the connection weights and the bias terms:

```python
hidden1 = model.layers[1]
weights, biases = hidden1.get_weights()
print(weights)
```

```text
[[-0.05415904  0.00010975 -0.00299759 ...  0.05136904  0.0740822
   0.06472497]
 [ 0.05510217 -0.01353022 -0.00363479 ...  0.07100512 -0.04926914
  -0.02905609]
 [-0.07024231  0.02524897 -0.04784295 ... -0.0521326   0.05084455
  -0.06636713]
 ...
 [ 0.0067075  -0.00256791 -0.064556   ...  0.05266081  0.03520959
  -0.02309504]
 [ 0.05826265 -0.0361187  -0.04228947 ...  0.05612285 -0.03179397
   0.06843598]
 [ 0.06636336 -0.00123435 -0.00247347 ...  0.01809192  0.03434542
   0.00700523]]
```

Please observe that the Dense layer initialized the connection weights randomly.

> This is needed to break symmetry.

The biases were initialized to zero (**0**), which is fine.

```python
print(biases)
```

```text
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Continues on next page →

← Continued from previous page

```text
      0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

If we want to use a different initialization method, we can set `kernel_initializer` or alternatively, we can use `bias_initializer` when creating the layer.

1.9 Information

Weight Matrix Shape

The shape of the weight matrix depends on the number of inputs, which is why we specified the `input_shape` when creating the model. If you do not specify the input shape, it's also fine as Keras will simply wait until it knows the input shape before it actually builds the model parameters.

This will happen either when you feed it some data[35], or when we call its `build()` method. Until the model parameters are built, we will **NOT** be able to do certain things, such as display the model summary or save the model.

So, if we know the input shape when creating the model, it is best to specify it.

[35] such as during training.

Model Compiling

After a model is created, we need to call its `compile()` method to specify the loss function and the optimizer to use, or you can specify a list of extra metrics to compute during training and evaluation:

```python
model.compile(loss="sparse_categorical_crossentropy",
              optimizer="sgd",
              metrics=["accuracy"])
```

Before continuing, we need to explain what is going on here.

We use the `sparse_categorical_crossentropy` loss because we have sparse labels,[36] and the classes are **exclusive**.

[36] i.e., for each instance, there is just a target class index, from 0 to 9 in this case.

- If we had one target probability per class for each instance (such as one-hot vectors, e.g., [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.] to represent class 3), then we would need to use the `categorical_crossentropy` loss instead.
- If we were doing binary classification or multi-label binary classification, then we would use the **sigmoid activation** function in the output layer instead of the softmax activation function, and we would use the `binary_crossentropy` loss.

Regarding the optimiser, `sgd` means that we will train the model using **stochastic gradient descent**. Keras will perform the backpropagation algorithm described earlier.[37]

[37] i.e., reverse-mode autodiff plus gradient descent.

Finally, as this is a classifier, it's useful to measure its accuracy during training and evaluation, which is why we set `metrics=["accuracy"]`.

Training and Evaluating Models

Now the model is ready to be trained. For this we simply need to call its `fit()` method:

```python
history = model.fit(X_train, y_train, epochs=30,
                    validation_data=(X_valid, y_valid))
```

We pass it the input features (`X_train`) and the target classes (`y_train`), as well as the number of epochs to train:[38]

[38] or else it would default to just 1, which would definitely not be enough to converge to a good solution.

We also pass a validation set which is optional. Keras will measure the loss and the extra metrics on this set at the end of each epoch, which is very useful to see how well the model really performs.

> If the performance on the training set is much better than on the validation set, the model is probably overfitting the training set, or there is a bug, such as a data mismatch between the training set and the validation set.

And that's it.

The ANN is trained. At each epoch during training, Keras displays the number of mini-batches processed so far on the left side of the progress bar.

The batch size is 32 by default, and since the training set has 55,000 images, the model goes through 1,719 batches per epoch: 1,718 of size 32, and 1 of size 24.

After the progress bar, we can see the mean training time per sample, and the loss and accuracy[39] on both the training set and the validation set and notice that the training loss went down, which is a good sign, and the validation accuracy reached 88.94% after 30 epochs.

[39] or any other extra metrics you asked for.

That's slightly below the training accuracy, so there is a little bit of overfitting going on, but **NOT** a huge amount.

If the training set was very skewed, with some classes being overrepresented and others underrepresented, it would be useful to set the `class_weight` argument when calling the `fit()` method, to give a larger weight to underrepresented classes and a lower weight to overrepresented classes.

These weights would be used by Keras when computing the loss. If we need per-instance weights, set the `sample_weight` argument. If both `class_weight` and `sample_weight` are provided, then Keras multiplies them. Per-instance weights could be useful, for example, if some instances were labeled by experts while others were labeled using a crowdsourcing platform: you might want to give more weight to the former.

We can also provide sample weights[40] for the validation set by adding them as a third item in the `validation_data` tuple. The `fit()` method returns a History object containing the training parameters (`history.params`), the list of epochs it went through (`history.epoch`), and most importantly a dictionary (`history.history`) containing the loss and extra metrics it measured at the end of each epoch on the training set and on the validation set, if any.

[40] but not class weights.

```python
print(history.params)
print(history.epoch)
```

```text
{'verbose': 'auto', 'epochs': 30, 'steps': 1719}
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24,
↪ 25, 26, 27, 28, 29]
```

If we use this dictionary to create a Pandas DataFrame and call its `plot()` method, you get the learning curves shown in **Fig.** 1.14.

You can see that both the training accuracy and the validation accuracy steadily increase during training, while the training loss and the validation loss decrease.

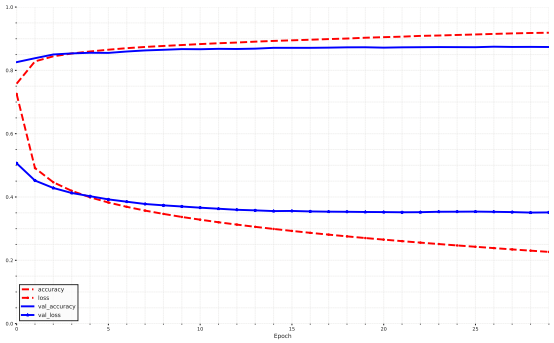


Figure 1.14
Learning curves: the mean training loss and accuracy measured over each epoch, and the mean validation loss and accuracy measured at the end of each epoch.

This is good.

The validation curves are relatively close to each other at first, but they get further apart over time, which shows that there's a little bit of overfitting. In this particular case, the model looks like it performed better on the validation set than on the training set at the beginning of training, but that's not actually the case.

The validation error is calculated at the end of each epoch, while the training error is computed using a **running mean** during each epoch, so the training curve should be shifted by half an epoch to the left.

If we do that, we will see that the training and validation curves overlap almost perfectly at the beginning of training. The training set performance ends up beating the validation performance, as is generally the case when you train for long enough.

We can tell that the model has **NOT** quite converged yet, as the validation loss is still going down, so it would be better to continue training. This is as simple as calling the `fit()` method again, as Keras just continues training where it left off: you should be able to reach about 89.8% validation accuracy, while the training accuracy will continue to rise up to 100%.

> This is not always the case.

If you are not satisfied with the performance of your model, it is a good idea to back and tune the hyperparameters.

1. First check the learning rate (η).

2. If that doesn't help, try another optimizer, and always retune the learning rate after changing

any hyperparameter,

3. If the performance is still not great, try tuning model hyperparameters such as the number of layers, the number of neurons per layer, and the types of activation functions to use for each hidden layer.

You can also try tuning other hyperparameters, such as the batch size (it can be set in the `fit()` method using the `batch_size` argument, which defaults to 32).

Once you are satisfied with your model's validation accuracy, you should evaluate it on the test set to estimate the generalization error before you deploy the model to production. You can easily do this using the `evaluate()` method.

> It also supports several other arguments, such as `batch_size` and `sample_weight`.

It is common to get slightly lower performance on the test set than on the validation set, as hyperparameters are **tuned on the validation set**, **NOT** the test set. However, in this example, we did not do any hyperparameter tuning, so the lower accuracy is just bad luck.

> Resist the temptation to tweak the hyperparameters on the test set, or else your estimate of the generalization error will be too optimistic.

Using Model to Make Predictions

It is time to use the model's `predict()` method to make predictions on new instances. As we don't have actual new instances, we'll just use the first three (**3**) instances of the test set:

```python
X_new = X_test[:3]
y_proba = model.predict(X_new)
print(y_proba.round(2))
```

```text
[[0.   0.   0.   0.   0.   0.12 0.   0.01 0.   0.87]
 [0.   0.   1.   0.   0.   0.   0.   0.   0.   0. ]
 [0.   1.   0.   0.   0.   0.   0.   0.   0.   0. ]]
```

For each instance the model estimates one probability per class, from class 0 to class 9. This is similar to the output of the `predict_proba()` method in `sklearn` classifiers.

> For example, for the first image it estimates that the probability of class 9 (ankle boot) is 87%, the probability of class 7 (sneaker) is 1%, the probability of class 5 (sandal) is 12%, and the probabilities of the other classes are negligible.

In other words, it is highly confident that the first image is footwear, most likely ankle boots but possibly sneakers or sandals. If you only care about the class with the highest estimated probability,[41] then we can use the `argmax()` method to get the highest probability class index for each instance:

[41] Even if that probability is quite low.

```python
y_pred = y_proba.argmax(axis=-1)
print(y_pred)
```

```text
[9 2 1]
```

Here, the classifier actually classified all three images correctly, where these images are shown in **Fig.** 1.15.

1.2.3 Building a Regression MLP Using the Sequential API

Instead of classifying categories, lets try to estimate a **value**. For this application, we need a different dataset. Let's switch back to the California housing problem and tackle it using the same MLP as earlier, with 3 hidden layers composed of 50 neurons each, but this time building it with `tf.keras`.

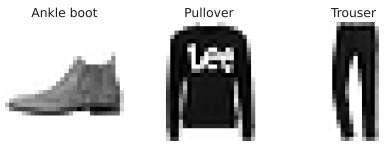


Figure 1.15 Correctly classified Fashion MNIST images.

Using the sequential API to build, train, evaluate, and use a regression MLP is quite similar to what we did for classification. The primary differences in the following code example are the fact that the output layer has a single neuron[42] and it uses no activation function, the loss function is the MSE, the metric is Root-Mean Square Error (RMSE), and we're using an Adam optimizer like `sklearn`s `MLPRegressor` did.

[42] since we only want to predict a single value.

In addition, in this example we don't need a Flatten layer, and instead we're using a Normalization layer as the first layer: it does the same thing as `sklearn`s `StandardScaler`, but it must be fitted to the training data using its `adapt()` method before you call the model's `fit()` method. Let's look at the code:

```python
housing = fetch_california_housing()
X_train_full, X_test, y_train_full, y_test = train_test_split(
    housing.data, housing.target, random_state=42)
X_train, X_valid, y_train, y_valid = train_test_split(
    X_train_full, y_train_full, random_state=42)
```

```python
tf.random.set_seed(42)
norm_layer = tf.keras.layers.Normalization(input_shape=X_train.shape[1:])
model = tf.keras.Sequential([
    norm_layer,
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(1)
])
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])
norm_layer.adapt(X_train)
```

Continues on next page →

← Continued from previous page

```python
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
mse_test, rmse_test = model.evaluate(X_test, y_test)
X_new = X_test[:3]
y_pred = model.predict(X_new)
```

As we can see, the sequential API is quite clean and straightforward. However, although Sequential models are extremely common, it is sometimes useful to build neural networks with more complex topologies, or with multiple inputs or outputs. For this purpose, Keras offers the functional API.

```python
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:])
normalized = normalization_layer(input_)
hidden1 = hidden_layer1(normalized)
hidden2 = hidden_layer2(hidden1)
concat = concat_layer([normalized, hidden2])
output = output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

```python
input_wide = tf.keras.layers.Input(shape=[5])  # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6])  # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()
norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])
output = tf.keras.layers.Dense(1)(concat)
model = tf.keras.Model(inputs=[input_wide, input_deep], outputs=[output])
```

```python
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=["RootMeanSquaredError"])

X_train_wide, X_train_deep = X_train[:, :5], X_train[:, 2:]
X_valid_wide, X_valid_deep = X_valid[:, :5], X_valid[:, 2:]
X_test_wide, X_test_deep = X_test[:, :5], X_test[:, 2:]
X_new_wide, X_new_deep = X_test_wide[:3], X_test_deep[:3]

norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit((X_train_wide, X_train_deep), y_train, epochs=20,
                    validation_data=((X_valid_wide, X_valid_deep), y_valid))
mse_test = model.evaluate((X_test_wide, X_test_deep), y_test)
y_pred = model.predict((X_new_wide, X_new_deep))
```

```python
input_wide = tf.keras.layers.Input(shape=[5])  # features 0 to 4
input_deep = tf.keras.layers.Input(shape=[6])  # features 2 to 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()
norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
concat = tf.keras.layers.concatenate([norm_wide, hidden2])
output = tf.keras.layers.Dense(1)(concat)
aux_output = tf.keras.layers.Dense(1)(hidden2)
model = tf.keras.Model(inputs=[input_wide, input_deep],
                       outputs=[output, aux_output])

```

```python
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss=("mse", "mse"), loss_weights=(0.9, 0.1), optimizer=optimizer,
              metrics=["RootMeanSquaredError", "RootMeanSquaredError"])
```

```python
norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit(
    (X_train_wide, X_train_deep), (y_train, y_train), epochs=20,
    validation_data=((X_valid_wide, X_valid_deep), (y_valid, y_valid))
)
```

```python
eval_results = model.evaluate((X_test_wide, X_test_deep), (y_test, y_test))
weighted_sum_of_losses, main_loss, aux_loss = eval_results
```

```python
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

```python
y_pred_tuple = model.predict((X_new_wide, X_new_deep))
y_pred = dict(zip(model.output_names, y_pred_tuple))
```

```python
import shutil

shutil.rmtree("my_keras_model", ignore_errors=True)
```

```python
model.export("my_keras_model")
```

Chapter 2

Understanding PyTorch

Table of Contents

2.1 PyTorch Fundamentals 36
2.2 Implementing Linear Regression 47
2.3 Implementing a Regression MLP 52
2.4 Mini-Batch Gradient Descent Using DataLoaders 54
2.5 Model Evaluation 57
2.6 Building Nonsequential Models Using Custom Modules 59
2.7 Building an Image Classifier with PyTorch 67
2.8 Fine-Tuning Hyperparameters with Optuna 73
2.9 Saving and Loading PyTorch Models 77
2.10 Compiling and Optimizing a PyTorch Model 80

PyTorch is a powerful open source Deep Learning (DL) library developed by Facebook's AI Research, now called Meta AI. It is the Python successor of the **Torch** library, originally written in the Lua programming language.[1] With PyTorch, we can build all sorts of NN models and train them at scale using Graphical Processing Unit (GPU)s, or other hardware accelerators, as we will see later. In many ways it is similar to `NumPy`, except it also supports hardware acceleration and `autodiff`, and includes optimizers and ready-to-use neural net components.

When PyTorch was originally released in 2016, Google's **TensorFlow** library was by far the most popular:

> it was fast, it scaled well, and it could be deployed across many platforms.

But its programming model was unfortunately **complex** and **static**, making it difficult to use and debug.



[1] A powerful, lightweight, and efficient **scripting language** primarily designed to be embedded into other applications. Known for its incredible speed, minimalism, and easy-to-learn syntax, it is widely used in game development, such as *Roblox* and *World of Warcraft*, and for configuring software like *Neovim*.

PyTorch was designed from the ground up to provide a more **flexible**, Pythonic approach to building NNs.

In particular, as we will see later on, it uses dynamic computation graphs,[2] making it intuitive and easy to debug. PyTorch is also beautifully coded and documented, and focuses on its fundamental task:

> making it easy to build and train high-performance NN.

Last but certainly not least, it leans strongly into the **open source culture** and benefits from an enthusiastic and dedicated community, and a rich ecosystem.

In September 2022, PyTorch's governance was even transferred to the **PyTorch Foundation**, a subsidiary of the **Linux Foundation**.

All these qualities resonated well with researchers:

> PyTorch quickly became the most used framework in academia, and once a majority of DL papers were based on PyTorch, a large part of the industry was gradually converted as well.[3]

Chapter Structure In this chapter, we will learn how to train, evaluate, fine-tune, optimize, and save NN with PyTorch. We will start by getting familiar with the core building blocks of PyTorch, namely **tensors** and **autograd**. Next, we will test the waters by building and training a simple **linear regression model**, and then we will upgrade this model to a multilayer NN, first for regression, then for classification. Along the way, we will see how to build custom NN with multiple inputs or outputs. Finally, we will discuss how to automatically fine-tune hyperparameters using the **Optuna library**, and how to optimize and export our models.

[2] also known as define-by-run, a programming approach used in DL where the computational graph is built dynamically, on-the-fly, as the code runs, during the forward pass.

[3] To be fair, most of TensorFlow's usability issues were fixed in version 2, and Google also launched JAX, which is well designed and extremely fast, so PyTorch still has some healthy competition. The good news is that the APIs of all these libraries have converged quite a bit, so switching from one to the other is much easier than it used to be.

2.1 PyTorch Fundamentals

The core of PyTorch is the tensor.[4] It's a **multidimensional array** with a **shape** and a **data** type, used for numerical computations.

Tensors v. Array — 2.1 Information

Some of you whom have previous experience with Python may see similarities with `np.array` and might wonder are they the same thing just under a different name.

Well, yes, but a tensor also has two (**2**) additional features:

Continues on next page →

[4] There are things called **tensors** in mathematics and physics, covered in **M.Sc Higher Mathematics I** within ML tensors mean something simpler where they're really just multidimensional arrays for numerical computations, plus a few extra features.

← Continued from previous page

it can live on a GPU,[5] and supports **auto-differentiation**.[6]

From now on, every NN we will build from now on will input and output *tensors*.[7] So let's start by looking at how to create and manipulate tensors.

[5] or other hardware accelerators, as we will see.

[6] Tensor auto-differentiation (often called **autograd**) is the engine which computes exact derivatives for tensor operations without manual calculus. PyTorch tracks every mathematical operation performed on tensors to build a dynamic computation graph, then apply the chain rule to calculate gradients automatically.

[7] This is similar to `scikit-learn` models input and output `NumPy` arrays, which was the focus of **B.Sc Data Science I**.

2.1.1 Making some Tensors

First, let's import the PyTorch library and then create a PyTorch tensor similar to how we would a NumPy array, such as a 2×3 array:

```python
import torch

X = torch.tensor([[1.0, 4.0, 7.0], [2.0, 3.0, 6.0]])
print("Our tensor is:")
print("--------------")
print(X)
```

```
Our tensor is:
--------------
tensor([[1., 4., 7.],
        [2., 3., 6.]])
```

Similar to a NumPy array, a tensor can contain `float`, `int`, `bool`, or `complex` numbers.

> However, we can have just **one data type per tensor**.

Initializing a tensor with values of different types, the most general one will be selected.

For example, `pytorch` will follow the hierarchy below:

`complex` > `float` > `int` > `bool`.

We can also select the data type **explicitly** when creating the tensor,

For example `dtype=torch.float16` for 16-bit floats.

> Tensors of **strings** or **objects** are **NOT** supported.

We do operations on tensors similar to how we would do them in `np.array`:

```python
print(f"We can do quite a few operations with tensors.\n\n" +
      "For example:\n" +
      "------------\n" +
      f'Determine its shape: {X.shape}\n' +
      f'Determine its type : {X.dtype}\n' +
      f'Index operations   : {X[0, 1]}\n' +
      f'Get columns        : {X[:, 1]}')
```

Continues on next page →

← Continued from previous page

```
                                                
```

← Continued from previous page

```python
indices=tensor([1, 0, 0]))Transpose and multiplication:
    tensor([[66., 56.],
        [56., 49.]])
```

PyTorch prefers the argument name `dim` in operations such as `max()`, but it also supports `axis`, which is similar to one used in NumPy or Pandas.

We can also convert a tensor to a NumPy array using the `numpy()` method, and vice versa, create a tensor from a NumPy array:

```python
import numpy as np

X.numpy()
torch.tensor(np.array([[1., 4., 7.], [2., 3., 6.]]))

```

Here, the default precision for floats is 32 bits in PyTorch, whereas it's 64 bits in NumPy.

Less is better 2.2 Information

It's generally better to use 32 bits in DL as this takes half the Random Access Memory (RAM) and speeds up computations, and NNs do not actually need the extra precision offered by 64-bit floats. So when calling the `torch.tensor()` function to convert a NumPy array to a tensor, it's best to specify `dtype=torch.float32`.

```python
torch.tensor(np.array([[1., 4., 7.], [2., 3., 6.]]), dtype=torch.float32)
```

Alternatively, we can use `torch.FloatTensor()` which automatically converts the array to 32 bits:

```python
torch.FloatTensor(np.array([[1., 4., 7.], [2., 3., 6]]))
```

Both `torch.tensor()` and `torch.FloatTensor()` shown beforehand make a copy of the given NumPy array. However, if we prefer, we can use `torch.from_numpy()` which creates a tensor on the Central Processing Unit (CPU) which just uses the NumPy array's data directly, sans copying.

```python
X2_np       = np.array([[1., 4., 7.], [2., 3., 6]])
# X2_np and X2 share the same data in memory
X2          = torch.from_numpy(X2_np)
X2_np[0, 1] = 88
print(X2)
```

A word of caution

> modifying the NumPy array will also modify the tensor, and vice versa.

We can also modify a tensor in place using indexing and slicing, as with a NumPy array:

```python
X[:, 1] = -99
print(X)
```

```
tensor([[  1., -99.,   7.],
        [  2., -99.,   6.]])
```

PyTorch's API provides numerous in-place operations, such as `abs_()`, `sqrt_()`, and `zero_()`, which modify the input tensor directly.[8]

[8] they can sometimes save some memory and speed up your models.

For example, the `relu_()` method applies the ReLU activation function in place by replacing all negative values with 0s:

```python
X.relu_()
print(X)
```

```
tensor([[1., 0., 7.],
        [2., 0., 6.]])
```

PyTorch's in-place operations are easy to spot at a glance because their name always ends with an underscore. With very few exceptions (e.g., `zero_()`), removing the underscore gives you the regular operation.[9]

[9] e.g., `abs_()` is in place, `abs()` is not.

We will cover many more operations as we go, but now let's look at how to use **hardware acceleration** to make computations much faster.

2.1.2 Hardware Acceleration

PyTorch tensors can be copied easily to the GPU, assuming that our machine has a compatible GPU, and have the required libraries installed. PyTorch has excellent support for Nvidia GPUs, as well as several other hardware accelerators:

- Apple's Metal Performance Shaders (MPS) to accelerate computations on Apple silicon such as the *M1*, *M2*, and later chips, as well as some Intel Macs with a compatible GPU.
- *AMD Instinct* accelerators and *AMD Radeon* GPU, through the ROCm software stack, or via *DirectML* on Windows.
- Intel GPUs and CPUs on Linux and Windows via Intel's oneAPI.
- Google TPUs via the `torch_xla` library.

Let's check whether PyTorch can access an Nvidia GPU or Apple's MPS, otherwise let's fall back to the CPU:

```python
if torch.cuda.is_available():
    device = "cuda"
```

Continues on next page →

← Continued from previous page

python
```
elif torch.backends.mps.is_available():
    device = "mps"
else:
    device = "cpu"

print(device)
```
```
mps
```

2.3 Information

We Need More Power

DL generally requires a lot of compute power, especially once we start diving into computer vision and Natural Language Processing (NLP), in the following chapters.

prepare to be very, very **patient**.

Now let's create a tensor on that GPU. To do that, one option is to create the tensor on the CPU, then copy it to the GPU using the `to()` method and we can always tell which device a tensor lives on by looking at its `device` attribute:

python
```
M = torch.tensor([[1., 2., 3.], [4., 5., 6.]])
M = M.to(device)
print(M.device)
```
```
mps:0
```

The `cpu()`, `mps()`, and `cuda()` methods are short for `to("cpu")`, `to("mps")`, and `to("cuda")`, respectively.

Alternatively, we can create tensor directly on the GPU using the `device` argument:

python
```
M = torch.tensor([[1., 2., 3.], [4., 5., 6.]], device=device)
print(M)
```
```
tensor([[1., 2., 3.],
        [4., 5., 6.]], device='mps:0')
```

2.4 Information

Having Multiple GPUs

In the rare event that we have **multiple** Nvidia GPUs, we can refer to the desired GPU by **appending** the GPU index:

`"cuda:0"` (or just `"cuda"`) for GPU # 0, `"cuda:1"` for GPU #1, and so on.

Once the tensor is on the GPU, we can run operations on it normally, and they will all take place on the GPU:

```python
R = M @ M.T  # run some operations on the GPU
print(R)
```

```
tensor([[14., 32.],
        [32., 77.]], device='mps:0')
```

Note that the result `R` also lives on the GPU, meaning we can perform multiple operations on the GPU without having to transfer **data back and forth between** the CPU and the GPU. This is crucial in DL as data transfer between devices can often become a **performance bottleneck**.

Effect of GPU Acceleration

Depends on the GPU. The more expensive ones are dozens of times faster than the cheap ones. But speed alone is not the only important factor as the data throughput is also crucial. If our model is compute heavy,[10] the GPU's speed and amount of RAM will typically matter most, but if it is a shallower model, then pumping the training data into the GPU might become the bottleneck.

[10] e.g., a very deep neural net.

Let's run a little test to compare the speed of a matrix multiplication running on the CPU versus the GPU:

```python
M = torch.rand((1000, 1000))  # on the CPU
M @ M.T  # warmup
%timeit M @ M.T

M = M.to(device)
M @ M.T  # warmup
%timeit M @ M.T
```

The GPU gave us a 29 times speed boost. We can just imagine the speedup we could get using a more powerful GPU.

Smaller is Slower — 2.5 Information

The speed gain using smaller matrices is less than that of using large matrices. That's because GPUs work by breaking large operations into smaller operations and running them in parallel across thousands of cores.

If the task is small, it cannot be broken up into that many pieces, and the performance gain is therefore smaller. In fact, when running many tiny tasks, it can sometimes be faster to just run the operations on the CPU.

All right, now that we've seen what tensors are and how to use them on the CPU or the GPU, let's look at PyTorch's auto-differentiation feature.

Autograd

PyTorch comes with an efficient implementation of reverse-mode auto-differentiation, called **autograd**, which stands for *automatic gradients*. It is quite easy to use.

> For example, consider $f(x) = x^2$. Calculus tells us that the derivative of this function is $f'(x) = 2x$. If we evaluate $f(5)$ and $f'(5)$, we get 25 and 10, respectively.

Let's see if PyTorch agrees:

```python
x = torch.tensor(5.0, requires_grad=True)
f = x ** 2
f
f.backward()
x.grad
```

Great, we got the correct results: f is 25, and `x.grad` is 10.

> The `backward()` function automatically computed the gradient $f'(x)$ at the same point $x = 5.0$.

Let's go through this code line by line and get a feeling of what it has done:

- First, we created a tensor x, equal to 5.0, and we told PyTorch that it's a **variable** by specifying `requires_grad=True`. Knowing this, PyTorch will automatically keep track of all operations involving x:

  > this is needed because PyTorch must capture the computation graph in order to run backprop on it and obtain the derivative of f with regard to x.[11]

- Then we compute `f = x ** 2`. The result is a tensor equal to `25.0`, the square of `5.0`. Here, `f` also carries a `grad_fn` attribute which represents the operation that created this tensor,[12] and which tells PyTorch how to backpropagate the gradients through this particular operation. This `grad_fn` attribute is how PyTorch keeps track of the computation graph.

- Next, we call `f.backward()`. This method backpropagates the gradients through the computation graph, starting with `f`, and all the way back to the leaf nodes. Here, it is x.

- Lastly, we can just read the `x` tensor's `grad` attribute, which was computed during backprop. This gives us the derivative of `f` with regard to `x`.

[11] In this computation graph, the tensor x is a *leaf node*.

[12] `**`, power, hence the name `PowBackward0`

PyTorch creates a new computation graph on the fly during *each forward pass*, as the operations are executed. This allows PyTorch to support very dynamic models containing loops and conditionals.

How Grad Works — 2.6 Information

The way PyTorch accumulates gradients in each variable's `grad` attribute can be surprising at first, especially coming from **TensorFlow** or **JAX**. In these frameworks, computing the gradients of `f` with regard to `x` just returns the gradients, without affecting `x`. In PyTorch, if we call `backward()` on a

Continues on next page →

← Continued from previous page

tensor, it will accumulate the gradients in every variable that was used to compute it.

So if we call `backward()` on two tensors `t1` and `t2` that both used the same variable `v`, then `v.grad` will be the sum of their gradients.

After computing the gradients, we would generally want to perform a gradient descent step by subtracting a fraction of the gradients from the model variables.[13] In our simple example, running gradient descent will gradually push `x` toward `0`, since that's the value that minimizes $f(x) = x^2$.

[13] at least when training a NN.

For us to do a gradient descent step, we must temporarily disable gradient tracking since we don't want to track the gradient descent step itself in the computation graph.[14] This can be done by placing the gradient descent step inside a `torch.no_grad()` context, like this:

[14] in fact, PyTorch would raise an exception if we tried to run an in-place operation on a tracked variable.

```python
learning_rate = 0.1
with torch.no_grad():
    x -= learning_rate * x.grad  # gradient descent step
```

The variable `x` gets decremented by `0.1 * 10.0 = 1.0`, down from `5.0` to `4.0`. Another way to avoid gradient computation is to use the variable's `detach()` method:

this creates a new tensor detached from the computation graph, with `requires_grad=False`, but still pointing to the same data in memory.

We can then update this detached tensor:

```python
x_detached = x.detach()
x_detached -= learning_rate * x.grad
```

Since `x_detached` and `x` share the same memory, modifying `x_detached` also modifies `x`.

The `detach()` method can be handy when we need to run some computation on a tensor without affecting the gradients,[15] or when we need fine-grained control over which operations should contribute to gradient computation. Using `no_grad()` is generally preferred when performing inference or doing a gradient descent step, as it provides a convenient context-wide method to disable gradient tracking.

[15] e.g., for evaluation or logging.

Lastly, before you repeat the whole process,[16] it's essential to zero out the gradients of every model parameter:[17]

[16] Which is forward pass + backward pass + gradient descent step.

[17] you don't need a `no_grad()` context for this since the gradient tensor has `requires_grad=False`.

```python
x.grad.zero_()
```

Don't Forget your Gradients 2.7 Information

If we forget to zero out the gradients at each training iteration, the `backward()` method will just accumulate them, causing incorrect gradient descent updates. Since there won't be any explicit error, just low performance,[18] this issue may be **hard to debug**.

[18] and perhaps infinite or `NaN` values.

Putting everything together, the whole training loop looks like this:

```python
learning_rate = 0.1
x = torch.tensor(5.0, requires_grad=True)
for iteration in range(100):
    f = x ** 2  # forward pass
    f.backward()  # backward pass
    with torch.no_grad():
        x -= learning_rate * x.grad  # gradient descent step
    x.grad.zero_()  # reset the gradients
```

If we want to use in-place operations to save memory and speed up our models a bit by avoiding unnecessary copy operations, we have to be careful:

in-place operations don't always play nicely with **autograd**.

a. As we saw earlier, you cannot apply an in-place operation to a leaf node,[19] as PyTorch wouldn't know where to store the computation graph. For example `x.cos_()` or `x += 1` would cause a **RuntimeError**.

[19] i.e., a tensor with `requires_grad=True`.

b. consider the following code, which computes $z(t) = \exp(t) + 1$ at $t = 2$ and then tries to compute the gradients:

Oh no! Although z is computed correctly, the last line causes a **RuntimeError**, complaining that:

> one of the variables needed for gradient computation has been modified by an in-place operation.

Indeed, the intermediate result `z = t.exp()` was lost when we ran the in-place operation `z += 1`, so when the backward pass reached the exponential operation, the gradients could not be computed.

A simple fix is to replace `z += 1` with `z = z + 1`. It looks similar, but it's no longer an in-place operation:

a new tensor is created and assigned to the same variable, but the original tensor is unchanged and recorded in the computation graph of the final tensor.

Surprisingly, if we were to replace `exp()` with `cos()` in the previous code example, the gradients will be computed correctly. This is because the outcome depends on the way each operation is implemented:

- Some operations, such as `exp()`, `relu()`, `rsqrt()`, `sigmoid()`, `sqrt()`, `tan()`, and `tanh()`, save their outputs in the **computation graph** during the forward pass, then use these outputs to compute the gradients during the backward pass.

  For example, since the derivative of `exp(x)` is equal to `exp(x)`, it makes a lot of sense to store the output of this operation in the computation graph during the forward pass, then use this output during the backward pass to get the gradients as we have no need to store additional data, and no need to recompute `exp(x)`.

  This means that you must not modify such an operation's output in place, or you will get an **error during the backward pass**.

- Other operations, such as `abs()`, `cos()`, `log()`, `sin()`, `square()`, and `var()`, save their inputs instead of their output:[20]

  Such an operation doesn't care if you modify its output in place, but you must not modify its inputs in place before the backward pass:[21]

- Some operations, such as `max()`, `min()`, `norm()`, `prod()`, `sgn()`, and `std()`, save both the inputs and the outputs, so we must not modify either of them in place before the backward pass.

- Lastly, a few operations, such as `ceil()`, `floor()`, `mean()`, `round()`, and `sum()`, save neither their inputs nor their outputs:[22] You can safely modify them in place.

Try Before Commit 2.8 Information

Implement models first without any in-place operations, then if we need to save some memory or speed up your model a bit, we can try converting some of the most costly operations to their in-place counterparts.

Just make sure that your model still outputs the same result for a given input, and also make sure you don't modify in place a tensor needed for backprop.[23]

Now let's apply what we've learned so far by building and training a simple linear regression model with PyTorch.

[20] For example, the derivative of `abs(x)` is `1` when `x < 0` and `+1` when `x > 0`. If this operation saved its output in the computation graph, the backward pass would be unable to know whether `x` was positive or negative, given `abs(x)` is always positive, so it wouldn't be able to compute the gradients. This is why this operation must save its input instead.

[21] E.g., to compute something else based on the same inputs.

[22] For example, the derivative of `floor(x)` is always zero, at least for non-integer inputs, so the `floor()` operation just saves the shape of the inputs during the forward pass, then during the backward pass it produces gradients of the same shape but full of zeros. For integer inputs, `autograd` also returns zeros instead of `NaN`.

[23] Which in this case we will get a `Runtime Error`.

2.2 Implementing Linear Regression

We will start by implementing linear regression using **tensors** and **autograd** directly, and then will simplify the code using PyTorch's high-level API, and also add GPU support.

2.2.1 Linear Regression Using Tensors and Autograd

Let's tackle the same *California housing dataset*.

It is assumed that the dataset is already downloaded using `sklearn.datasets.fetch_california_housing()`, and have been split into a training set (`X_train` and `y_train`), a validation set (`X_valid` and `y_valid`), and a test set (`X_test` and `y_test`), using `sklearn.model_selection.train_test_split()`.

Next, let's convert it to tensors and **normalize** it. We could use a `StandardScaler` for this, but let's just use **tensor** operations instead, to get a bit of practice:

python
```
X_train = torch.FloatTensor(X_train)
X_valid = torch.FloatTensor(X_valid)
X_test = torch.FloatTensor(X_test)
means = X_train.mean(dim=0, keepdims=True)
stds = X_train.std(dim=0, keepdims=True)
X_train = (X_train - means) / stds
X_valid = (X_valid - means) / stds
X_test = (X_test - means) / stds
```

Let's also convert the targets to tensors. Given our predictions will be column vectors,[24] we need to ensure that our targets are also column vectors.

[24] i.e., matrices with a single column.

The Usefulness of Column Vectors — 2.9 Information

Column vectors, shape `[m, 1]`, and row vectors, shape `[1, m]`, are often preferred over 1D vectors, shape `[m]`, in ML, as they avoid ambiguity in some operations, such as **matrix multiplication** or **broadcasting**, and they make the code more consistent whether there's just one feature or more.

Unfortunately, the `NumPy` arrays representing the targets are 1D, so we need to reshape the tensors to column vectors by adding a 2nd dimension of size 1:[25]

[25] Just like in NumPy, the `reshape()` method allows you to specify `1` for one of the dimensions. This dimension's size is automatically calculated to ensure the new tensor has the same number of cells as the original.

python
```
y_train = torch.FloatTensor(y_train).view(-1, 1)
y_valid = torch.FloatTensor(y_valid).view(-1, 1)
y_test = torch.FloatTensor(y_test).view(-1, 1)
```

Now that the data is ready, let's create the parameters of our linear regression model:

```python
torch.manual_seed(42)
n_features = X_train.shape[1]  # there are 8 input features
w = torch.randn((n_features, 1), requires_grad=True)
b = torch.tensor(0., requires_grad=True)
```

We now have a weights parameter `w`,[26] and a bias parameter `b`, which is a single **scalar**. The weights are initialized randomly, while the bias is initialized to zero. We could have initialized the weights to zero as well in this case, but when we get to NNs it will be important to initialize the weights randomly to break the symmetry between neurons, so we might as well get into the habit now.

[26] A column vector with one weight per input dimension, in this case 8.

Reproducibility 2.10 Information

We called `torch.manual_seed()` to ensure that the results are reproducible. However, PyTorch does **not guarantee perfectly reproducible results** across different releases, platforms, or devices, so if we do not run the code in this chapter with PyTorch 2.8.0, we may get different results.

Moreover, since a GPU splits each operation into multiple chunks and runs them in **parallel**, the order in which these chunks finish can vary across runs, and this may slightly affect the result due to floating-point precision errors.

These minor differences may compound during training, and lead to very different models. To avoid this, we can tell PyTorch to use only deterministic algorithms by calling `torch.use_deterministic_algorithms(True)` and setting `torch.backends.cudnn.benchmark = False`. However, deterministic algorithms are often **slower than stochastic ones**, and some operations don't have a deterministic version at all, so we will get an error if your code tries to use one.

Next, let's train our model, very much like we did in **B.Sc Data Science I**, except we will use **autodiff** to compute the gradients rather than using a closed-form equation. For now we will use Batch Gradient Descent (BGD), using the full training set at each training step:

```python
learning_rate = 0.4
n_epochs = 20
for epoch in range(n_epochs):
    y_pred = X_train @ w + b
    loss = ((y_pred - y_train) ** 2).mean()
    loss.backward()
    with torch.no_grad():
        b -= learning_rate * b.grad
        w -= learning_rate * w.grad
        b.grad.zero_()
        w.grad.zero_()
    print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {loss.item()}")
```

Let's walk through this code:

1. Define the `learning_rate` hyperparameter. Here, we can experiment with different values to

find a value that converges fast and gives a precise result.

2. Run **20 epochs**.[27]
3. Run the **forward pass**. We compute the predictions `y_pred`, and the mean squared error loss.
4. Run `loss.backward()` to compute the gradients of the loss with regard to every model parameter. This is autograd in action.
5. Use the gradients `b.grad` and `w.grad` to perform a gradient descent step. Notice that we're running this code inside a with `torch.no_grad()` context, as discussed earlier.
6. Once we've done the gradient descent step, we reset the gradients to zero.
7. Print the epoch number and the current loss at each epoch. The `item()` method extracts the value of a scalar tensor.

[27] We could implement early stopping to find the right moment to stop and avoid overfitting, but we will keep things simple for now.

And that's it. If we run this code, we should see the training loss going down like this:

We've just trained your first model using PyTorch. We can now use the model to make predictions for some new data `X_new`.[28] For example, let's make predictions for the first three instances in the test set:

[28] which must be represented as a PyTorch tensor.

It's best to use a with `torch.no_grad()` context during inference as PyTorch will consume less RAM and run faster since it won't have to keep track of the computation graph.

Summary Implementing linear regression using PyTorch's low-level API wasn't too hard, but using this approach for more complex models would get really messy and difficult. So PyTorch offers a higher-level API to simplify all this. Let's rewrite our model using this higher-level API.

2.2.2 Linear Regression Using PyTorch's High-Level API

PyTorch provides an implementation of linear regression in the `torch.nn.Linear` class, so let's use it:

```python
import torch.nn as nn

torch.manual_seed(42)  # to get reproducible results
model = nn.Linear(in_features=n_features, out_features=1)
```

The `nn.Linear` class, which is short for `torch.nn.Linear`, is one of numerous modules provided by PyTorch. Each module is a subclass of the `nn.Module` class. To build a simple linear regression model, a single `nn.Linear` module is all we need. However, for most NNs we will need to assemble many modules, and therefore it is better to think of modules as mathematical building blocks where many modules contain model parameters.

For example, the `nn.Linear` module contains a **bias vector**, with one bias term per neuron, and a **weight matrix**, with one row per neuron and one column per input dimension, which is the transpose of the weight matrix.

Since our model has a single neuron (because `out_features=1`), the bias vector contains a single bias term, and the weight matrix contains a single row. These parameters are accessible directly as attributes of the **nn.Linear** module:

Notice both parameters were automatically initialized randomly.[29] These parameters are instances of the `torch.nn.Parameter` class, which is a subclass of the `tensor.Tensor` class: this means that we can use them exactly like normal tensors. A module's `parameters()` method returns an iterator over all of the module's attributes of type Parameter, as well as all the parameters of all its submodules, recursively if it has any.

[29] which is why we used `manual_seed()` to get reproducible results.

It does not return regular tensors, even those with `requires_grad=True`, which is the main difference between a regular tensor and a Parameter:

There's also a `named_parameters()` method that returns an iterator over pairs of parameter names and values. A module can be called just like a regular function.

For example, let's make some predictions for the first two instances in the training set:[30]

[30] since the model is not trained yet, its parameters are random and the predictions are terrible.

When we use a module as a function, PyTorch internally calls the module's `forward()` method. In the case of the `nn.Linear module`, the `forward()` method computes `X @ self.weight.T + self.bias`, where `X` is the input.

Observe that the result contains the `grad_fn` attribute, showing that **autograd** did its job and tracked the computation graph while the model was making its predictions.

Passing Custom Functions 2.11 Information

If we were to pass a custom function to a module's `register_forward_hook()` method, it will be **called automatically** every time the module itself is called. This is particularly handy for **logging** or **debugging**.

To remove a hook, just call the `remove()` method on the object returned by `register_forward_hook()`. Please observe that hooks only work if we call the model like a function, not if you call its `forward()` method directly.[31] We can also register functions to run during the backward pass using `register_backward_hook()`.

[31] Which is which is why we should never do that.

Now that we have our model, we need to create an optimizer to update the model parameters, and we must also choose a loss function:

```python
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
mse = nn.MSELoss()
```

PyTorch provides a few different optimizers.[32] Here we're using the simple Stochastic Gradient Descent (SGD) optimizer, which can be used for SGD, mini-batch Gradient Descent (GD), or batch GD. To initialize it, we must give it the model parameters and the **learning rate** (η).

[32] We will discuss them in the next chapter.

For the loss function, we create an instance of the `nn.MSELoss class`. This is also a module, so we can use it like a function, giving it the predictions and the targets, and it will compute the MSE. The `nn` module contains many other loss functions and other neural net tools, as we will see.

Next, let's write a small function to train our model:

```python
def train_bgd(model, optimizer, criterion, X_train, y_train, n_epochs):
    for epoch in range(n_epochs):
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
        print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {loss.item()}")
```

Compare this training loop with our earlier training loop. It's very similar, but we're now using higher-level constructs rather than working directly with tensors and autograd.

Here are a few things to note:

- In PyTorch, the loss function object is commonly referred to as the **criterion**, to distinguish it from the loss value itself.[33] In this example, it's the `MSELoss` instance.
- The `optimizer.step()` line corresponds to the two lines that updated `b` and `w` in our earlier code.
- And of course the `optimizer.zero_grad()` line corresponds to the two lines that zeroed out `b.grad` and `w.grad`. Notice that we don't need to use with `torch.no_grad()` here since this is done automatically by the optimizer, inside the `step()` and `zero_grad()` functions.

[33] which is computed at each training iteration using the criterion.

Most people prefer to call `zero_grad()` before calling `loss.backward()`, rather than after: this might be a bit safer in case the gradients are non-zero when calling the function, but in general it makes no difference since gradients are automatically initialized to None.

Now let's call this function to train our model!

All good; the model is trained, and we can now use it to make predictions by simply calling it like a function:[34]

[34] preferably inside a `no_grad()` context, as we saw earlier.

These predictions are similar to the ones our previous model made, but not exactly the same. That's because the `nn.Linear` module initializes the parameters slightly differently.

The module uses a **uniform random distribution** from $\sqrt{2}/4$ to $+\sqrt{2}/4$ for both the weights and the bias term.

2.3 Implementing a Regression MLP

PyTorch provides a helpful `nn.Sequential` module which chains multiple modules:

> when we call this module with some inputs, it feeds these inputs to the first module, then feeds the output of the first module to the second module, and so on.

Most NN contain stacks of modules, and in fact many NN are just one big stack of modules:

> This makes the `nn.Sequential` module one of the most useful modules in PyTorch.

The MLP we want to build is just that: a simple stack of modulestwo hidden layers and one output layer. So let's build it using the

```python
torch.manual_seed(42)
model = nn.Sequential(
    nn.Linear(n_features, 50),
    nn.ReLU(),
    nn.Linear(50, 40),
    nn.ReLU(),
    nn.Linear(40, 1)
)
```

Let's go through each layer:

1. The first layer must have the right number of inputs for our data: `n_features` which is equal to 8 in our case. However, it can have any number of outputs: let's pick 50 which is a hyperparameter we can tune.

2. Next we have an `nn.ReLU` module, which implements the ReLU activation function for the first hidden layer. This module does not contain any model parameters, and it acts itemwise so the shape of its output is equal to the shape of its input.

3. The second hidden layer must have the same number of inputs as the output of the previous layer: in this case, 50. However, it can have any number of outputs. It's common to use the same number of output dimensions in all hidden layers, but in this example I used 40 to make it clear that the output of one layer must match the input of the next layer.

4. Then again, an `nn.ReLU` module to implement the second hidden layer's activation function.

5. Finally, the output layer must have 40 inputs, but this time its number of outputs is not free: it must match the targets' dimensionality. Since our targets have a single dimension, we must have just one output dimension in the output layer.

Now let's train the model just like we did before:

We are still using batch gradient descent, computing the gradients over the entire training set at each iteration. This works with small datasets, but if we want to be able to scale up to large datasets and large models, we need to switch to mini-batch GD.

2.4 Mini-Batch Gradient Descent Using DataLoaders

To help implement mini-batch GD, PyTorch provides a class named `DataLoader` in the `torch.utils.data` module. It can efficiently load batches of data of the desired size, and shuffle the data at each epoch if we want it to. The `DataLoader` expects the dataset to be represented as an object with at least two methods:

- `__len__(self)` to get the number of samples in the dataset, and
- `__getitem__(self, index)` to load the sample at the given index, including the target.

In our case, the training set is available in the `X_train` and `y_train` tensors, so we first need to wrap these tensors in a dataset object with the required API. To help with this, PyTorch provides a `TensorDataset` class. So let's build a `TensorDataset` to wrap our training set, and a `DataLoader` to pull batches from this dataset.

During training, we want the dataset to be shuffled, so we specify `shuffle=True`:

```python
from torch.utils.data import TensorDataset, DataLoader

train_dataset = TensorDataset(X_train, y_train)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
```

Now that we have a larger model and we have the tools to train it one batch at a time, it's a good time to start using **hardware acceleration**, we just need to move the model to the GPU, which will move all of its parameters to the GPU RAM, and then at the start of each iteration during training we must copy each batch to the GPU.

To move the model, we can just use its `to()` method, just like we did with tensors:

```python
# extra code  build the model just like earlier
torch.manual_seed(42)
model = nn.Sequential(
    nn.Linear(n_features, 50), nn.ReLU(),
    nn.Linear(50, 40), nn.ReLU(),
    nn.Linear(40, 1)
)

model = model.to(device)

# extra code  build the optimizer and loss function, as earlier
learning_rate = 0.02
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0)
mse = nn.MSELoss()
```

We can also create the loss function and optimizer, as earlier.[35]

[35] Using a lower learning rate, such as 0,02.

Some optimizers have some internal state. The optimizer will usually allocate its state on the same

device as the model parameters, so it's important to create the optimizer after you have moved the model to the GPU.

Now let's create a `train()` function to implement mini-batch GD:

```python
def train(model, optimizer, criterion, train_loader, n_epochs):
    model.train()
    for epoch in range(n_epochs):
        total_loss = 0.
        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            loss = criterion(y_pred, y_batch)
            total_loss += loss.item()
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()

        mean_loss = total_loss / len(train_loader)
        print(f"Epoch {epoch + 1}/{n_epochs}, Loss: {mean_loss:.4f}")
```

At every epoch, the function iterates through the whole training set, one batch at a time, and processes each batch just like earlier.

But what about the very first line: `model.train()`?

Well, this switches the model and all of its submodules to training mode. For now, this makes no difference at all, but it will be important later when we start using layers which behave differently during training and evaluation, such as with `nn.Dropout` or `nn.BatchNorm1d`. Whenever we want to use the model outside of training,[36] we must first switch the model to evaluation mode by running `model.eval()`. It is also worth mentioning here that `model.training` holds a boolean that indicates the current mode.

[36] Such as for evaluation, or to make predictions on new instances.

Working with Loops — 2.12 Information

PyTorch itself does not provide a training loop implementation out of the box. If we need to use a well-tested, off-the-shelf training loop with all the bells and whistles, then we can use a library such as PyTorch Lightning, FastAI, Catalyst, or Keras which are built on top of PyTorch.

Now let's call this `train()` function to train our model on the GPU:

We actually reached a much lower loss in the same number of epochs. However, each epoch was much slower. There are two (**2**) easy tweaks we can make to considerably speed up training:

- If we are using a CUDA device, we should generally set `pin_memory=True` when creating the data loader: this will allocate the data in **page-locked memory** which guarantees a fixed physical memory location in the CPU RAM, and therefore allows direct memory access (DMA) transfers to the GPU, eliminating an extra copy operation that would otherwise be needed.

While this could use more CPU RAM since the memory cannot be swapped out to disk, it typically results in significantly faster data transfers, which means faster training. When transferring a tensor to the GPU using its `to()` method, we may also set `non_blocking=True` to avoid blocking the CPU during the data transfer, which only works if `pin_memory=True`.

- The current training loop waits until a batch has been fully processed before it loads the next batch, which we can often speed up training by pre-fetching the next batches on the CPU while the GPU is still working on the current batch.

  For this, set the data loader's `num_workers` argument to the number of processes we want to use for data loading and preprocessing.

  > The optimal number depends on our platform, hardware, and workload.

  We can also tweak the number of batches which each worker pre-fetches by setting the data loader's `prefetch_factor` argument.

  > The overhead of spawning and synchronizing workers can often slow down training rather than speed it up, especially on Windows.

  In this case, you can try setting `persistent_workers=True` to reuse the same workers across epochs.

The next step is to learn how to evaluate your model.

2.5 Model Evaluation

Let's write a function to evaluate the model. It takes the model and a `DataLoader` for the dataset that we want to evaluate the model on, as well as a function to compute the metric for a given batch, and lastly a function to aggregate the batch metrics:[37]

[37] By default, it just computes the mean.

```python
def evaluate(model, data_loader, metric_fn, aggregate_fn=torch.mean):
    model.eval()
    metrics = []
    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            metric = metric_fn(y_pred, y_batch)
            metrics.append(metric)
    return aggregate_fn(torch.stack(metrics))
```

Now let's build a `TensorDataset` and a `DataLoader` for our validation set, and pass it to our `evaluate()` function to compute the validation MSE:

```python
valid_dataset = TensorDataset(X_valid, y_valid)
valid_loader = DataLoader(valid_dataset, batch_size=32)
valid_mse = evaluate(model, valid_loader, mse)
valid_mse
```

It works fine. But now suppose we want to use the RMSE instead of the MSE. PyTorch does not have a built-in function for that, but it's easy enough to write:

```python
def rmse(y_pred, y_true):
    return ((y_pred - y_true) ** 2).mean().sqrt()

evaluate(model, valid_loader, rmse)
```

The RMSE should be equal to the square root of the MSE. However, when we compute the square root of the MSE that we found earlier, we get a different result:

```python
valid_mse.sqrt()
```

The reason is that instead of calculating the RMSE over the whole validation set, we computed it over each batch and then computed the mean of all these batch RMSEs.

> That's **NOT** mathematically equivalent to computing the RMSE over the whole validation set.

To solve this, we can use the MSE as our `metric_fn`, and use the `aggregate_fn` to compute the square root of the mean MSE:[38]

[38] The mean of the batch MSEs is equal to the overall MSE since all batches have the same size. Well, except the last batch, which is often smaller, but that makes very little difference.

```python
evaluate(model, valid_loader, mse,
         aggregate_fn=lambda metrics: torch.sqrt(torch.mean(metrics)))
```

Rather than implement metrics yourself, you may prefer to use the TorchMetrics library,[39] which provides many well-tested streaming metrics.

[39] made by the same team as PyTorch Lightning.

> A streaming metric is an object that keeps track of a given metric, and can be updated one batch at a time.

```python
import torchmetrics

def evaluate_tm(model, data_loader, metric):
    model.eval()
    metric.reset()  # reset the metric at the beginning
    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            metric.update(y_pred, y_batch)  # update it at each iteration
    return metric.compute()  # compute the final result at the end
```

Then we can create an RMSE streaming metric, move it to the GPU, and use it to evaluate the validation set:

```python
rmse = torchmetrics.MeanSquaredError(squared=False).to(device)
evaluate_tm(model, valid_loader, rmse)
```

If the performance on the training set is much better than on the validation set, your model is probably overfitting the training set, or there is a bug, such as a data mismatch between the training set and the validation set.

So far we have only looked at simple sequential models, composed of a sequence of linear layers and ReLU activation functions.

> How would you build a more complex, nonsequential model?

For this, we will need to build custom modules.

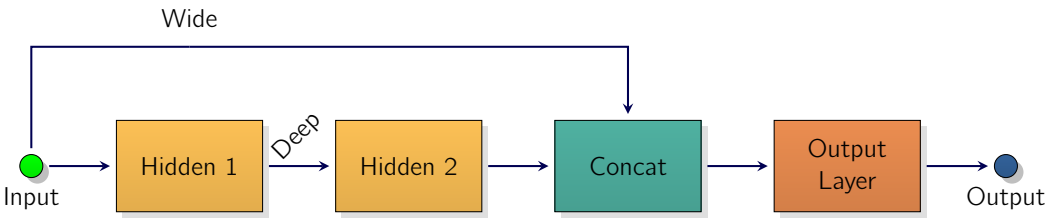


Figure 2.1
Wide & Deep neural network

2.6 Building Nonsequential Models Using Custom Modules

One example of a nonsequential NN is a Wide & Deep NN. This neural network architecture was introduced in a 2016 paper by *Heng-Tze Cheng* et al [1]. It connects all or part of the inputs directly to the output layer, as shown in **Fig.** 2.1 This architecture makes it possible for the NN to learn both deep patterns, using the deep path, and simple rules, through the short path.

The short path can also be used to provide manually engineered features to the NN. In contrast, a regular MLP forces all the data to flow through the full stack of layers.

> Simple patterns in the data may end up being **distorted by this sequence of transformations**.

Let's build such a NN to tackle the *California housing dataset*. As this wide and deep architecture is nonsequential, we have to create a custom module. It's easier than it sounds:

> We just create a class derived from `torch.nn.Module`, then create all the layers you need in the constructor,[40] and define how these layers should be used by the module in the `forward()` method:

[40] after calling the base class's `__init__()` method.

```python
class WideAndDeep(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.deep_stack = nn.Sequential(
            nn.Linear(n_features, 50), nn.ReLU(),
            nn.Linear(50, 40), nn.ReLU(),
            nn.Linear(40, 30), nn.ReLU(),
        )
        self.output_layer = nn.Linear(30 + n_features, 1)

    def forward(self, X):
        deep_output = self.deep_stack(X)
        wide_and_deep = torch.concat([X, deep_output], dim=1)
        return self.output_layer(wide_and_deep)
```

We can use any kind of module inside our custom module. In this example, we use an `nn.Sequential` module to build the **deep** part of our model.[41] It's the same MLP as earlier, except we separated the output layer because we need to feed it the concatenation of the model's inputs and the deep part's

[41] It's actually not that deep as this is just a toy example.

outputs. For this same reason, the output layer now has 40 + `n_features` inputs instead of just 40.

In the `forward()` method, we just feed the input `X` to the deep stack, concatenate the input and the deep stack's output, and feed the result to the output layer.

Child's Method — 2.13 Information

Modules have a `children()` method which returns an iterator over the module's submodules **non-recursively**. There's also a `named_children()` method. If our model has a variable number of submodules, we should store them in an `nn.ModuleList` or an `nn.ModuleDict`, which are returned by the `children()` and `named_children()` methods.[42] Similarly, if our model has a variable number of parameters, we should store them in an `nn.ParameterList` or an `nn.ParameterDict` to ensure they are returned by the `paramters()` and `named_parameters()` methods.

[42] as opposed to regular Python lists and dicts.

Now we can create an instance of our custom module, move it to the GPU, train it, evaluate it, and use it exactly like our previous models:

python
```
torch.manual_seed(42)
model = WideAndDeep(n_features).to(device)
learning_rate = 0.002  # the model changed, so did the optimal learning rate

optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0)
mse = nn.MSELoss()
rmse = torchmetrics.MeanSquaredError(squared=False).to(device)
history = train2(model, optimizer, mse, rmse, train_loader, valid_loader,
                 n_epochs)

```

But what if we want to send a subset of the features through the wide path and a different subset, possibly overlapping, through the deep path, as shown in **Fig.** 2.2?

In this case, one approach is to split the inputs inside the `forward()` method, for example:

python
```
class WideAndDeepV2(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.deep_stack = nn.Sequential(
            nn.Linear(n_features - 2, 50), nn.ReLU(),
            nn.Linear(50, 40), nn.ReLU(),
            nn.Linear(40, 30), nn.ReLU(),
        )
        self.output_layer = nn.Linear(30 + 5, 1)

    def forward(self, X):
        X_wide = X[:, :5]
        X_deep = X[:, 2:]
        deep_output = self.deep_stack(X_deep)
        wide_and_deep = torch.concat([X_wide, deep_output], dim=1)
        return self.output_layer(wide_and_deep)
```

This works fine; however, in many cases it's preferable to just let the model take two separate

tensors as input. Let's see why and how.

2.6.1 Building Models with Multiple Inputs

Some models require multiple inputs that cannot easily be combined into a single tensor. For example, the inputs may have a different number of dimensions.[43] To make our Wide & Deep model, take two separate inputs, as shown in **Fig.** 2.2, we must start by changing the model's `forward()` method:

[43] i.e., when we want to feed both images and text to the NN.

```python
class WideAndDeepV3(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.deep_stack = nn.Sequential(
            nn.Linear(n_features - 2, 50), nn.ReLU(),
            nn.Linear(50, 40), nn.ReLU(),
            nn.Linear(40, 30), nn.ReLU(),
        )
        self.output_layer = nn.Linear(30 + 5, 1)

    def forward(self, X_wide, X_deep):
        deep_output = self.deep_stack(X_deep)
        wide_and_deep = torch.concat([X_wide, deep_output], dim=1)
        return self.output_layer(wide_and_deep)
```

Next, we need to create datasets that return the wide and deep inputs separately:

```python
torch.manual_seed(42)
train_data_wd = TensorDataset(X_train[:, :5], X_train[:, 2:], y_train)
train_loader_wd = DataLoader(train_data_wd, batch_size=32, shuffle=True)
valid_data_wd = TensorDataset(X_valid[:, :5], X_valid[:, 2:], y_valid)
valid_loader_wd = DataLoader(valid_data_wd, batch_size=32)
test_data_wd = TensorDataset(X_test[:, :5], X_test[:, 2:], y_test)
test_loader_wd = DataLoader(test_data_wd, batch_size=32)
```

Since the data loaders now return three tensors instead of two at each iteration, we need to update the main loop in the evaluation and training functions:

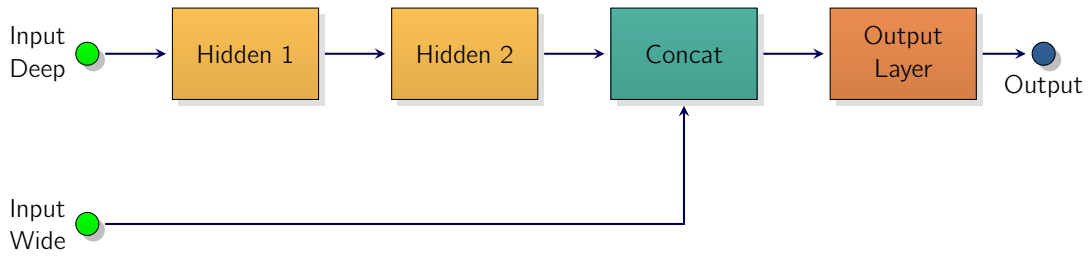


Figure 2.2
Handling multiple inputs

```python
        for X_batch_wide, X_batch_deep, y_batch in data_loader:
            X_batch_wide = X_batch_wide.to(device)
            X_batch_deep = X_batch_deep.to(device)
            y_batch = y_batch.to(device)
            y_pred = model(X_batch_wide, X_batch_deep)
```

Alternatively, since the order of the inputs matches the order of the `forward()` method's arguments, we can use Python's $*$ operator to unpack all the inputs returned by the `data_loader` and pass them to the model.

> The advantage of this implementation is that it will work with models that take any number of inputs, not just two, **as long as the order is correct**.

```python
        for *X_batch_inputs, y_batch in train_loader:
            model.train()
            X_batch_inputs = [X.to(device) for X in X_batch_inputs]
            y_batch = y_batch.to(device)
            y_pred = model(*X_batch_inputs)
```

When our model has many inputs, it's easy to make a mistake and mix up the order of the inputs, which can lead to hard-to-debug issues. To avoid this, it can be a good idea to name each input.

For this, you can define a custom dataset that returns a dictionary from input names to input values, like this:

```python
class WideAndDeepDataset(torch.utils.data.Dataset):
    def __init__(self, X_wide, X_deep, y):
        self.X_wide = X_wide
        self.X_deep = X_deep
        self.y = y

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        input_dict = {"X_wide": self.X_wide[idx], "X_deep": self.X_deep[idx]}
        return input_dict, self.y[idx]
```

Then create the datasets and data loaders:

```python
torch.manual_seed(42)
train_data_named = WideAndDeepDataset(
    X_wide=X_train[:, :5], X_deep=X_train[:, 2:], y=y_train)
train_loader_named = DataLoader(train_data_named, batch_size=32, shuffle=True)
valid_data_named = WideAndDeepDataset(
    X_wide=X_valid[:, :5], X_deep=X_valid[:, 2:], y=y_valid)
valid_loader_named = DataLoader(valid_data_named, batch_size=32)
test_data_named = WideAndDeepDataset(
    X_wide=X_test[:, :5], X_deep=X_test[:, 2:], y=y_test)
test_loader_named = DataLoader(test_data_named, batch_size=32)
```

Once again, we also need to update the main loop in the evaluation and training functions:

```python
        for inputs, y_batch in data_loader:
            inputs = {name: X.to(device) for name, X in inputs.items()}
            y_batch = y_batch.to(device)
            y_pred = model(X_wide=inputs["X_wide"], X_deep=inputs["X_deep"])
            metric.update(y_pred, y_batch)
```

Alternatively, since all the input names match the `forward()` method's argument names, we can use Python's `**` operator to unpack all the tensors in the inputs dictionary and pass them as named arguments to the model: `y_pred = model(**inputs)`.

Now that we know how to build sequential and nonsequential models with one or more inputs, let's look at models with multiple outputs.

2.6.2 Building Models with Multiple Outputs

There are many use cases where you may need a NN with multiple outputs:

- The task may demand it. For instance, we may want to locate and classify the main object in a picture: a **regression task** and a **classification** task.
- Similarly, we may have multiple independent tasks based on the same data. Sure, we could train one NN per task, but in many cases we will get better results on all tasks by training a single NN with one output per task. This is because the NN can learn features in the data that are useful across tasks.

  For example, we could perform multitask classification on pictures of faces, using one output to classify the person's facial expression[44] and another output to identify whether they are wearing glasses or not.

[44] smiling, surprised, etc.

- Another use case is regularization.[45]

  For example, you may want to add an auxiliary output in a NN architecture, shown in **Fig.** 2.3, to ensure that the underlying part of the network learns something useful on its own, without relying on the rest of the network.

[45] i.e., a training constraint whose objective is to reduce overfitting and thus improve the model's ability to generalize

Let's add an auxiliary output to our Wide & Deep model to ensure the deep part can make good predictions on its own. Since the deep stack's output dimension is 40, and the targets have a single dimension, we must add an `nn.Linear` layer for the auxiliary output to go from 40 dimensions down to 1.

We also need to make the `forward()` method compute the auxiliary output, and return both the main output and the auxiliary output:

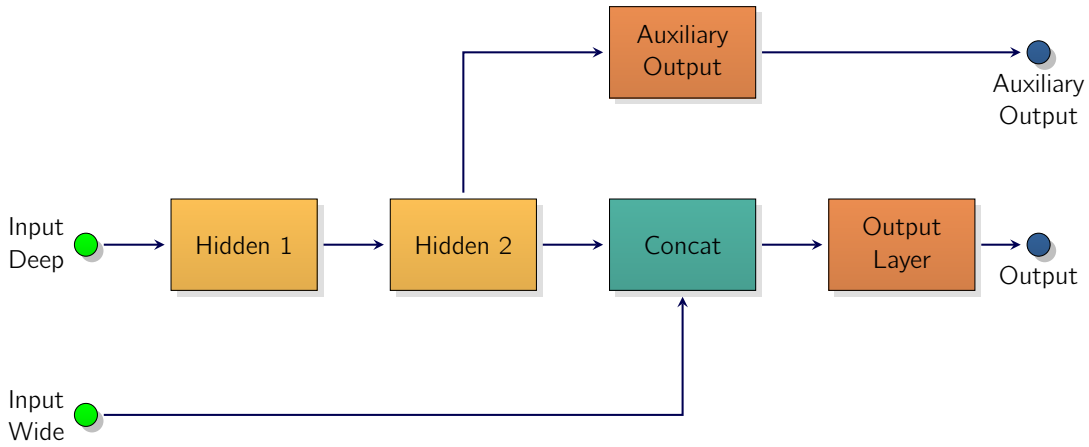


Figure 2.3 Handling multiple outputs, in this example to add an auxiliary output for regularization

```python
class WideAndDeepV4(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.deep_stack = nn.Sequential(
            nn.Linear(n_features - 2, 50), nn.ReLU(),
            nn.Linear(50, 40), nn.ReLU(),
            nn.Linear(40, 30), nn.ReLU(),
        )
        self.output_layer = nn.Linear(30 + 5, 1)
        self.aux_output_layer = nn.Linear(30, 1)

    def forward(self, X_wide, X_deep):
        deep_output = self.deep_stack(X_deep)
        wide_and_deep = torch.concat([X_wide, deep_output], dim=1)
        main_output = self.output_layer(wide_and_deep)
        aux_output = self.aux_output_layer(deep_output)
        return main_output, aux_output
```

Next, we need to update the main loop in the training function:

```python
def evaluate_named(model, data_loader, metric):
    model.eval()
    metric.reset()  # reset the metric at the beginning
    with torch.no_grad():
        for inputs, y_batch in data_loader:
            inputs = {name: X.to(device) for name, X in inputs.items()}
            y_batch = y_batch.to(device)
            y_pred = model(X_wide=inputs["X_wide"], X_deep=inputs["X_deep"])
            metric.update(y_pred, y_batch)
    return metric.compute()  # compute the final result at the end

def train_named(model, optimizer, criterion, metric, train_loader,
                valid_loader, n_epochs):
    history = {"train_losses": [], "train_metrics": [], "valid_metrics": []}
    for epoch in range(n_epochs):
        total_loss = 0.
        metric.reset()
```

Continues on next page →

← Continued from previous page

```python
        for inputs, y_batch in train_loader:
            model.train()
            inputs = {name: X.to(device) for name, X in inputs.items()}
            y_batch = y_batch.to(device)
            y_pred = model(**inputs)
            loss = criterion(y_pred, y_batch)
            total_loss += loss.item()
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
            metric.update(y_pred, y_batch)
        mean_loss = total_loss / len(train_loader)
        history["train_losses"].append(mean_loss)
        history["train_metrics"].append(metric.compute().item())
        history["valid_metrics"].append(
            evaluate_named(model, valid_loader, metric).item())
        print(f"Epoch {epoch + 1}/{n_epochs}, "
              f"train loss: {history['train_losses'][-1]:.4f}, "
              f"train metric: {history['train_metrics'][-1]:.4f}, "
              f"valid metric: {history['valid_metrics'][-1]:.4f}")
    return history

torch.manual_seed(42)
learning_rate = 0.01
model = WideAndDeepV3(n_features).to(device)
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate, momentum=0)
mse = nn.MSELoss()
rmse = torchmetrics.MeanSquaredError(squared=False).to(device)
history = train_named(model, optimizer, mse, rmse, train_loader_named,
                      valid_loader_named, n_epochs)
```

Notice that the model now returns both the main predictions `y_pred` and the auxiliary predictions `y_pred_aux`.

In this example, we can use the same targets and the same loss function to compute the main output's loss and the auxiliary output's loss. In other cases, we may have different targets and loss functions for each output, in which case you would need to create a custom dataset to return all the necessary targets.

Once we have a loss for each output, we must combine them into a single loss that will be minimized by gradient descent. In general, this final loss is just a weighted sum of all the output losses.

In this example, we use a higher weight for the main loss (0.8), because that's what we care about the most, and a lower weight for the auxiliary loss (0.2).

> This ratio is a **regularization hyperparameter** that we can tune.

We also need to update the main loop in the evaluation function. However, in this case we can just ignore the auxiliary output, since we only really care about the main outputthe auxiliary output is just there for regularization during training:

```python
        for inputs, y_batch in train_loader:
            model.train()
            inputs = {name: X.to(device) for name, X in inputs.items()}
            y_batch = y_batch.to(device)
            y_pred, y_pred_aux = model(**inputs)
            main_loss = criterion(y_pred, y_batch)
            aux_loss = criterion(y_pred_aux, y_batch)
            loss = 0.8 * main_loss + 0.2 * aux_loss
            total_loss += loss.item()
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
            metric.update(y_pred, y_batch)
```

We can now build and train all sorts of NN architectures, combining predefined modules and custom modules in any way you please, and with any number of inputs and outputs. The flexibility of NNs is one of their main qualities. But so far we have only tackled a regression task, so let's now turn to classification.

2.7 Building an Image Classifier with PyTorch

As in Chapter 1, we will tackle the *Fashion MNIST dataset*, so the first thing we need to do is to download the dataset. We could use the `fetch_openml()` function like we did in Chapter 1, but we will show another method instead, using the *TorchVision library*.

2.7.1 Using TorchVision to Load the Dataset

The *TorchVision library* is an important part of the PyTorch ecosystem:

> it provides many tools for computer vision, including utility functions to download common datasets, such as *MNIST* or *Fashion MNIST*, as well as pretrained models for various computer vision tasks, functions to transform images,[46] and more.

[46] e.g., crop, rotate, resize, etc.

Let's go ahead and use it to load *Fashion MNIST*. It is already split into a training set (60,000 images) and a test set (10,000 images), but we'll hold out the last 5,000 images from the training set for validation, using PyTorch's `random_split()` function:

```python
import torchvision
import torchvision.transforms.v2 as T

toTensor = T.Compose([T.ToImage(), T.ToDtype(torch.float32, scale=True)])

train_and_valid_data = torchvision.datasets.FashionMNIST(
    root="datasets", train=True, download=True, transform=toTensor)
test_data = torchvision.datasets.FashionMNIST(
    root="datasets", train=False, download=True, transform=toTensor)

torch.manual_seed(42)
train_data, valid_data = torch.utils.data.random_split(
    train_and_valid_data, [55_000, 5_000])
```

After the imports and before loading the datasets, we create a `toTensor` object. By default, the *FashionMNIST* class loads images as PIL (*Python Image Library*) images, with integer pixel values ranging from 0 to 255. But we need PyTorch float tensors instead, with scaled pixel values. Luckily, TorchVision datasets accept a transform argument which lets you pass a preprocessing function that will get executed on the fly whenever the data is accessed.[47] TorchVision provides many transform objects that you can use for this.[48]

[47] There's also a `target_transform` argument if you need to preprocess the targets.

[48] Most of these transforms are PyTorch modules.

In this code, we create a `Compose` transform to chain two transforms:

> a `ToImage` transform followed by a `ToDtype` transform.

`ToImage` converts various formatsincluding PIL images, NumPy arrays, and tensorsto TorchVision's Image class, which is a subclass of `Tensor`. The `ToDtype` transform converts the data type, in

this case to 32-bit floats. We also set its scale argument to `True` to ensure the values get scaled between 0.0 and 1.0.12

Next, we load the dataset. First the training and validation data, then the test data. The root argument is the path to the directory where *TorchVision* will create a subdirectory for the *Fashion MNIST dataset*. The train argument indicates whether you want to load the training set (`True` by default) or the test set. The download argument indicates whether to download the dataset if it cannot be found locally (`False` by default). And we also set `transform=toTensor` to use our custom preprocessing pipeline.

As usual, we must create data loaders:

```python
torch.manual_seed(42)
train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
valid_loader = DataLoader(valid_data, batch_size=32)
test_loader = DataLoader(test_data, batch_size=32)
```

Now let's look at the first image in the training set:

```python
X_sample, y_sample = train_data[0]
X_sample.shape
X_sample.dtype
```

In Chapter 1, each image was represented by a 1D array containing 784 pixel intensities, but now each image tensor has 3 dimensions, and its shape is: $[1, 28, 28]$.

- The first dimension is the channel dimension. For grayscale images, there is a single channel.[49]
- The other two dimensions are the height and width dimensions.

  For example, `X_sample[0, 2, 4]` represents the pixel located in channel 0, row 2, column 4. In Fashion MNIST, a larger value means a darker pixel.

[49] color images usually have three channels, as we will see later.

Be Careful of the Channel — 2.14 Information

PyTorch expects the channel dimension to be first, while many other libraries, such as *Matplotlib*, *PIL*, *TensorFlow*, *OpenCV*, or *Scikit-Image*, expect it to be last.

Always make sure to move the channel dimension to the right place, depending on the library you are using. `ToImage` already took care of moving the channel dimension to the first position, otherwise we could have used the `torch.permute()` function.

As for the targets, they are integers from 0 to 9, and we can interpret them using the same `class_names` array as in Chapter 1. In fact, many datasets, including `FashionMNIST`, have a classes attribute containing the list of class names.

For example, here's how we can tell that the sample image represents an ankle boot:

```python
train_and_valid_data.classes[y_sample]
```

2.7.2 Building the Classifier

Let's build a custom module for a classification MLP with two (2) hidden layers:

```python
class ImageClassifier(nn.Module):
    def __init__(self, n_inputs, n_hidden1, n_hidden2, n_classes):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Flatten(),
            nn.Linear(n_inputs, n_hidden1),
            nn.ReLU(),
            nn.Linear(n_hidden1, n_hidden2),
            nn.ReLU(),
            nn.Linear(n_hidden2, n_classes)
        )

    def forward(self, X):
        return self.mlp(X)

torch.manual_seed(42)
model = ImageClassifier(n_inputs=1 * 28 * 28, n_hidden1=300, n_hidden2=100,
                        n_classes=10).to(device)
xentropy = nn.CrossEntropyLoss()
```

There are a few things to note in this code:

- First, the model is composed of a single sequence of layers, which is why we used the `nn.Sequential` module. We did not have to create a custom module; we could have written `model = nn.Sequential(...)` instead, but it's generally preferable to wrap our models in custom modules, as it makes our code easier to deploy and reuse, and it's also easier to tune the **hyperparameters**.
- The model starts with an `nn.Flatten` layer. This layer does not have any parameters, it just reshapes each input sample to a single dimension, which is needed for the `nn.Linear` layers.

  For example, a batch of 32 *Fashion MNIST* images has a shape of $[32, 1, 28, 28]$, but after going through the `nn.Flatten` layer, it ends up with a shape of $[32, 784]$, given $28 \times 28 = 784$.
- The first hidden layer must have the correct number of inputs ($28 \times 28 = 784$), and the output layer must have the correct number of outputs which is 10 with one per class.
- We use a ReLU activation function after each hidden layer, and no activation function at all after the output layer.
- Since this is a multiclass classification task, we use nn.CrossEntropyLoss. It accepts either

class indices as targets (as in this example), or class probabilities.[50]

Didn't we say in Chapter 1 that we should use the softmax activation function on the output layer for multiclass classification tasks?

Well it turns out that PyTorch's `nn.CrossEntropyLoss` computes the cross-entropy loss directly from the logits, the class scores, rather than from the class probabilities. This bypasses some costly computations during training,[51] saving both compute and RAM. It's also more numerically stable.

The downside is that the model must output logits, which means that we will have to call the softmax function manually on the logits whenever we want class probabilities, as we will see shortly.

Other Classification Losses — 2.15 Information

For **multiclass classification**, another option is to add the `nn.LogSoftmax` activation function to the output layer, and then use the `nn.NLLLoss`.[52] The model then outputs log probabilities, rather than **logits**, and the loss computes the cross-entropy based on these log probabilities. Whenever we need actual estimated probabilities, we just pass the log probabilities through the exponential function.

This approach is a bit slower than using `nn.CrossEntropyLoss`, so it's not used as often, but it can sometimes be useful if we want our model to output log probabilities, or when we wish to tweak the probability distribution before computing the loss.

For binary classification tasks, we need to use a single output neuron in the output layer, and use the `nn.BCEWithLogitsLoss`.[53] The model outputs logits, so we must apply the sigmoid function to get estimated probabilities (for the positive class).

Alternatively, you can add the `nn.Sigmoid` activation function to the output layer, and use the `nn.BCELoss`: the model will then output estimated probabilities directly.[54]

For multilabel binary classification, the only difference is that you must have one neuron per label in the output layer.

Now we can train the model as usual.[55] To evaluate the model, we can use the Accuracy streaming metric from the `torchmetrics` library, and move it to the GPU:

```python
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
accuracy = torchmetrics.Accuracy(task="multiclass", num_classes=10).to(device)
_ = train2(model, optimizer, xentropy, accuracy, train_loader, valid_loader,
           n_epochs)
```

Training the model will take a few minutes with a GPU.[56] Handling images requires significantly more compute and memory than handling low-dimensional data.

The model reaches around 92.8% accuracy on the training set, and 87.2on the validation set.[57] This means there's a little bit of overfitting going on, so you may want to reduce the number of neurons

[50] such as one-hot vectors.

[51] i.e., logarithms and exponentials that cancel out.

[52] meaning negative log-likelihood loss.

[53] Here BCE stands for binary cross-entropy.

[54] but it's a bit slower and less numerically stable.

[55] e.g., using the `train()` function with an SGD optimizer

[56] or much longer without one.

[57] The results differ depending on the hardware accelerator you use.

or add some regularization.

Now that the model is trained, we can use it to make predictions on new images. As an example, let's make predictions for the first batch in the validation set, and look at the results for the first three images:

```python
model.eval()
X_new, y_new = next(iter(valid_loader))
X_new = X_new[:3].to(device)
with torch.no_grad():
    y_pred_logits = model(X_new)
y_pred = y_pred_logits.argmax(dim=1)  # index of the largest logit
y_pred
```

For each image, the predicted class is the one with the highest logit. In this example, all three predictions are correct.

But what if we want the model's estimated probabilities? For this, we need to compute the softmax of the logits manually, since the model does not include the softmax activation function on the output layer, as we discussed earlier. We could create an `nn.Softmax` module and pass it the logits, but we can also just call the `softmax()` function, which is just one of many functions you will find in the `torch.nn.functional` module.[58] It doesn't make much difference, it just avoids creating a module instance that we don't need:

[58] by convention, this module is usually imported as `F`.

```python
import torch.nn.functional as F
y_proba = F.softmax(y_pred_logits, dim=1)
if device == "mps":
    y_proba = y_proba.cpu()
y_proba.round(decimals=3)
```

Similar to Chapter 1, the model is very confident about the first two predictions: 91.1% and 99.6%, respectively. If we wish to apply label smoothing during training, just set the `label_smoothing` hyperparameter of the `nn.CrossEntropyLoss` to the amount of smoothing you wish, between 0 and 1, such as 0.05.

It can often be useful to get the model's top k predictions. For this, we can use the `torch.topk()` function, which returns a tuple containing both the top k values and their indices:

```python
y_top4_values, y_top4_indices = torch.topk(y_pred_logits, k=4, dim=1)
y_top4_probas = F.softmax(y_top4_values, dim=1)
if device == "mps":
    y_top4_probas = y_top4_probas.cpu()
y_top4_probas.round(decimals=3)
y_top4_indices
```

For the first image, the model's best guess is class 7 (Sneaker) with 91.1% confidence, its second best guess is class 9 (Ankle boot) with 8.8% confidence, and so on.

Perfectly Balanced, as All Things Should Be 2.16 Information

The Fashion MNIST dataset is **balanced**, meaning it has the same number of instances of each class. When dealing with an unbalanced dataset, we should generally give more weight to the rare classes and less weight to the frequent ones, or else your model will be biased toward the more frequent classes. We can do this by setting the weight argument of the `nn.CrossEntropyLoss`.

For example, if there are three classes with 900, 700, and 400 instances, respectively,[59] then the respective weights should be 2000/900, 2000/700, and 2000/400. It's preferable to normalize these weights to ensure they add up to 1, so in this example you would set `weight=torch.tensor([0.2205, 0.2835, 0.4961])`.

[59] i.e., 2000 instances in total.

We can now build, train, and evaluate both regression and classification NN. The next step is to learn how to fine-tune the model hyperparameters.

2.8 Fine-Tuning Hyperparameters with Optuna

We've discussed how to manually pick reasonable values for our model's hyperparameters in Chapter 1, but what if we want to go further and **automatically search** for good hyperparameter values?

One option is to convert your *PyTorch* model to a *Scikit-Learn* estimator, either by writing our own custom estimator class or by using a wrapper library such as *Skorch*, and then use `GridSearchCV` or `RandomizedSearchCV` to fine-tune the hyperparameters, as we did in **B.Sc Data Science I**. However, we will usually get better results by using a dedicated finetuning library such as *Optuna*, *Ray Tune*, or *Hyperopt*.

> These libraries offer several powerful tuning strategies, and they're highly customizable.

Let's tune the learning rate and the number of neurons in the hidden layers[60] using *Optuna*. First, we need to define a function that Optuna will call many times to perform hyperparameter tuning: this function must take a Trial object and use it to ask *Optuna* for hyperparameter values, and then use these hyperparameter values to build and train a model. Finally, the function must evaluate the model, typically on the validation set, and return the metric:

[60] for simplicity, we will use the same number of neurons in both hidden layers.

```python
import optuna

def objective(trial):
    learning_rate = trial.suggest_float("learning_rate", 1e-5, 1e-1, log=True)
    n_hidden = trial.suggest_int("n_hidden", 20, 300)
    model = ImageClassifier(n_inputs=1 * 28 * 28, n_hidden1=n_hidden,
                            n_hidden2=n_hidden, n_classes=10).to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    xentropy = nn.CrossEntropyLoss()
    accuracy = torchmetrics.Accuracy(task="multiclass", num_classes=10)
    accuracy = accuracy.to(device)
    history = train2(model, optimizer, xentropy, accuracy, train_loader,
                     valid_loader, n_epochs=10)
    validation_accuracy = max(history["valid_metrics"])
    return validation_accuracy
```

The `suggest_float()` and `suggest_int()` methods let us ask *Optuna* for a good hyperparameter value in a given range.[61] For the `learning_rate` hyperparameter, we ask for a value between 10^5 and 10^1, and since we don't know what the optimal scale is, we add `log=True`: this will make *Optuna* sample values from a log distribution, which makes it explore all possible scales.

[61] Optuna also provides a `suggest_categorical()` method.

> If we used the default **uniform distribution** instead, *Optuna* would be very unlikely to explore tiny values.

To start hyperparameter tuning, we create a `Study` object and call its `optimize()` method, passing it the objective function we just defined, as well as the number of trials to run.[62] Since our objective function returns a score, higher is better, we set `direction="maximize"` when creating the study.[63]

[62] i.e., the number of times *Optuna* should call the objective function.

[63] By default, *Optuna* tries to minimize the objective.

To ensure reproducibility, we also set *PyTorch*'s random seed, as well as the random seed used by *Optuna*'s sampler:

```python
torch.manual_seed(42)
sampler = optuna.samplers.TPESampler(seed=42)
study = optuna.create_study(direction="maximize", sampler=sampler)
study.optimize(objective, n_trials=5)
```

By default, Optuna uses the *Tree-structured Parzen Estimator* (TPE) algorithm to optimize the hyperparameters: this is a sequential model-based optimization algorithm, meaning it learns from past results to better select promising hyperparameters.

In other words, *Optuna* starts with random hyperparameter values, but it progressively focuses its search on the most promising regions of the hyperparameter space.

This allows *Optuna* to find much better hyperparameters than random search.

Additional Configuration 2.17 Information

We can add more hyperparameters to the search space, such as the **batch size**, the **type of optimizer**, the **number of hidden layers**, or the type of activation function, but remember that the search space will grow exponentially as we add more hyperparameters, so make sure it's worth the extra search time and compute.

Once *Optuna* is done, we can look at the best hyperparameters it found, as well as the corresponding validation accuracy:

```python
study.best_params
study.best_value
```

This is slightly better than the performance we got earlier. If we increase `n_trials` up to 50 or more, we will get much better results, but of course it will take hours to run. We can also just run `optimize()` repeatedly and stop once we are happy with the performance.

Optuna can also **run trials in parallel across multiple machines**, which can offer a near linear speed boost. For this, we will need to set up a SQL database,[64] and set the storage parameter of the `create_study()` function to point to that database. We also need to set the study's name via the `study_name` parameter, and set `load_if_exists=True`. After that, we can copy your hyperparameter tuning script to multiple machines, and run it on each one.[65] The scripts will work in parallel, reading and writing the trial results to the database.

[64] e.g., SQLite or PostgreSQL.

[65] If we are using random seeds, make sure they are different on each machine.

This has the additional benefit of keeping a full log of all your experiment results.

We may have noticed that we assumed that the `objective()` function had direct access to the training set and validation, presumably via global variables. In general, it's much cleaner to pass them as extra arguments to the `objective()` function, for example, like this:

```python
import optuna

def objective(trial):
    learning_rate = trial.suggest_float("learning_rate", 1e-5, 1e-1, log=True)
    n_hidden = trial.suggest_int("n_hidden", 20, 300)
    model = ImageClassifier(n_inputs=1 * 28 * 28, n_hidden1=n_hidden,
                            n_hidden2=n_hidden, n_classes=10).to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    xentropy = nn.CrossEntropyLoss()
    accuracy = torchmetrics.Accuracy(task="multiclass", num_classes=10)
    accuracy = accuracy.to(device)
    history = train2(model, optimizer, xentropy, accuracy, train_loader,
                     valid_loader, n_epochs=10)
    validation_accuracy = max(history["valid_metrics"])
    return validation_accuracy
```

To set the extra arguments (the dataset loaders in this case), we just create a lambda function when needed and pass it to the optimize() method. Alternatively, you can use the functools.partial() function which creates a thin wrapper function around the given callable to provide default values for any number of arguments:

```python
from functools import partial

objective_with_data = partial(objective, train_loader=train_loader,
                              valid_loader=valid_loader)
```

It's often possible to quickly tell that a trial is absolutely terrible.

> For example, when the loss shoots up during the first epoch, or when the model barely improves during the first few epochs.

In such a case, it's a good idea to interrupt training early to avoid wasting time and compute. You can simply return the model's current validation accuracy and hope that *Optuna* will learn to avoid this region of hyperparameter space. Alternatively, we can interrupt training by raising the `optuna.TrialPruned` exception:

this tells *Optuna* to ignore this trial altogether.

In many cases, this leads to a more efficient search because it avoids polluting *Optuna*'s search algorithm with many noisy model evaluations.

Optuna comes with several `Pruner` classes which can detect and prune bad trials.

> For example, the `MedianPruner` will prune trials whose performance is below the median performance, at regular intervals during training.

It starts pruning after a given number of trials have completed, controlled by `n_startup_trials`, which is `5` by default. For each trial after that, it lets training start for a few epochs, controlled by `n_warmup_steps`, which is `0` by default. Then every few epochs, controlled by `interval_steps`, it

ensures that the model's performance is better than the median performance at the same epoch in past trials. To use this pruner, create an instance and pass it to the `create_study()` method:

```python
torch.manual_seed(42)
sampler = optuna.samplers.TPESampler(seed=42)
study = optuna.create_study(direction="maximize", sampler=sampler)
study.optimize(objective, n_trials=5)
```

Then in the `objective()` function, add the following code so it runs after each epoch:

```python
def objective(trial, train_loader, valid_loader):
    learning_rate = trial.suggest_float("learning_rate", 1e-5, 1e-1, log=True)
    n_hidden = trial.suggest_int("n_hidden", 20, 300)
    model = ImageClassifier(n_inputs=1 * 28 * 28, n_hidden1=n_hidden,
                            n_hidden2=n_hidden, n_classes=10).to(device)
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
    xentropy = nn.CrossEntropyLoss()
    accuracy = torchmetrics.Accuracy(task="multiclass", num_classes=10)
    accuracy = accuracy.to(device)
    best_validation_accuracy = 0.0
    for epoch in range(n_epochs):
        history = train2(model, optimizer, xentropy, accuracy, train_loader,
                         valid_loader, n_epochs=1)
        validation_accuracy = max(history["valid_metrics"])
        if validation_accuracy > best_validation_accuracy:
            best_validation_accuracy = validation_accuracy
        trial.report(validation_accuracy, epoch)
        if trial.should_prune():
            raise optuna.TrialPruned()
    return best_validation_accuracy
```

The `report()` method informs *Optuna* of the current validation accuracy and epoch, so it can determine whether the trial should be pruned. If trial.`should_prune()` returns True, we raise a `TrialPruned` exception.

2.18 Information

Usefulness of Optuna

Optuna has many other features well worth exploring, such as visualization tools, persistence tools for trial results and other artifacts, a dashboard for human-in-the-loop optimization, and many other algorithms for hyperparameter search and trial pruning.

Once we are happy with the hyperparameters, we can train the model on the full training set,[66] then evaluate it on the test set. Hopefully, it will perform great. If it does, we will want to save the model, then load it and use it in production which is the final topic of this chapter.

[66] i.e., the training set plus the validation set.

2.9 Saving and Loading PyTorch Models

The easiest method to save a PyTorch model is to use the `torch.save()` method, passing it the `model` and the `filepath`. The model object is **serialized** using Python's pickle module,[67] then the result is compressed (`.zip`) and saved to disk.

[67] which can convert objects into a sequence of bytes

> The convention is to use the `.pt` or `.pth` extension for PyTorch files.

```python
torch.save(model, "my_fashion_mnist.pt")
```

With this we have save the command. When we need to, we can load the model[68] just as easily:

[68] For example in our production code.

```python
loaded_model = torch.load("my_fashion_mnist.pt", weights_only=False)
```

If our model uses any custom functions or classes, for example `ImageClassifier`, then `torch.save()` only saves **references** to them, not the code itself. Therefore we need to ensure any custom code is loaded in the Python environment before calling `torch.load()`.

> Also we have to make sure to use the same version of the code to avoid any mismatch issues.

Setting `weights_only=False` ensures the whole model object is loaded rather than just the model parameters. Then we can use the loaded model for inference. However, don't forget to switch to evaluation mode first using the `eval()` method:

```python
loaded_model.eval()
y_pred_logits = loaded_model(X_new)
```

This was relatively simple, but unfortunately this approach has some very serious drawbacks:



[69] It is very possible someone could inject code to your ML model during loading.

- Pickle's serialization format is notoriously insecure. While `torch.save()` doesn't save custom code, the pickle format supports it, so it is possible that a hacker[69] could inject malicious code in a saved PyTorch model:

  this code would be run automatically by the pickle module when the model is loaded.

  So always make sure you fully trust the model's source before you load it this way.

- Pickle is somewhat brittle. It can vary depending on the Python version,[70] and it saves specific filepaths to locate code, which can break if the loading environment has a different folder structure.

[70] e.g., there were big changes between Python 3.7 and 3.8.

To avoid these issues, it is recommended to save and load the **model weights only**, rather than the full model object:

```python
torch.save(model.state_dict(), "my_fashion_mnist_weights.pt")
```

The state dictionary returned by the `state_dict()` method is just a Python Ordered Dict containing an entry for each parameter returned by the `named_parameters()` method. It also contains buffers, if the model has any:

> A buffer is just a regular tensor that was registered with the model, or any of its submodules, using the `register_buffer()` method.

Buffers hold extra data which needs to be stored along with the model, but that is not a model parameter.[71]

[71] We will see an example in Chapter 11 with the batch-norm layer.

To load these weights, we must first create a model with the exact same structure, then load the weights using `torch.load()` with `weights_only=True`, and finally call the model's `load_state_dict()` method with the loaded weights:

```python
new_model = ImageClassifier(n_inputs=1 * 28 * 28, n_hidden1=300, n_hidden2=100,
                            n_classes=10)
loaded_weights = torch.load("my_fashion_mnist_weights.pt", weights_only=True)
new_model.load_state_dict(loaded_weights)
new_model.eval()
```

The saved model contains only data, and the `load()` function makes sure of that, so this is safe, and also much less likely to break between Python versions or to cause any deployment issue. However, it only works if you are able to create the exact same model architecture before loading the state dictionary. For this, you need to know the number of layers, the number of neurons per layer, and so on. It's a good idea to save this information along with the state dictionary:

```python
model_data = {
    "model_state_dict": model.state_dict(),
    "model_hyperparameters": {
        "n_inputs": 1 * 28 * 28,
        "n_hidden1": 300,
        "n_hidden2": 100,
        "n_classes": 10,
    }
}
torch.save(model_data, "my_fashion_mnist_model.pt")
```

You can then load this dictionary, construct the model based on the saved hyperparameters, and load the state dictionary into this model:

```python
loaded_data = torch.load("my_fashion_mnist_model.pt", weights_only=True)
new_model = ImageClassifier(**loaded_data["model_hyperparameters"])
new_model.load_state_dict(loaded_data["model_state_dict"])
new_model.eval()
```

If you want to be able to continue training where it left off, you will also need to save the optimizer's state dictionary, its hyperparameters, and any other training information you may need, such as the current epoch and the loss history. The safetensors library by Hugging Face is another popular way to save model weights safely.

There is yet another way to save and load your model: by first converting it to TorchScript. This also makes it possible to speed up your model's inference.

2.10 Compiling and Optimizing a PyTorch Model

PyTorch comes with a very nice feature.

It can automatically convert our model's code to TorchScript,[72] a statically typed subset of Python.

[72] TorchScript is an **intermediate representation** of a *PyTorch* model that can be run in high-performance, Python-free environments (like C++). It converts standard dynamic Python code into a statically typed, serializable format that allows for compiler optimizations, multi-threading without the Global Interpreter Lock (GIL), and deployment on mobile and edge devices.

There are two (**2**) primary benefits:

1. TorchScript code can be compiled and optimized to produce significantly faster models.

   For example, multiple operations can often be fused into a single, more efficient operation. Such as Operations on constants (e.g., `2*3`) can be replaced with their result (e.g., `6`)

   This is called **constant folding** and during this process unused code can be pruned.

2. TorchScript can be serialized, saved to disk, and then loaded and executed in Python or in a C++ environment using the `LibTorch` library.

   For example, this makes it possible to run PyTorch models on a wide range of devices, including embedded devices.

There are two ways to convert a PyTorch model to TorchScript. The first way is called tracing. PyTorch just runs your model with some sample data, logs every operation that takes place, and then converts this log to TorchScript. This is done using the `torch.jit.trace()` function:

```python
torchscript_model = torch.jit.trace(model, X_new)
```

This generally works well with static models whose `forward()` method doesn't use conditionals or loops. However, if we try to trace a model that includes an if or match statement, then only the branch that is actually executed will be captured by TorchScript, which is generally not what we would want. Similarly, if we use tracing with a model that contains a loop, then the TorchScript code will contain one copy of the operations within that loop for each iteration that was actually executed. Again, not what we generally want.

For such dynamic models, we will probably want to try another approach named **scripting**. In this case, PyTorch actually parses our Python code directly and converts it to TorchScript.

> This method supports if statements and while loops properly, **as long as the conditions are tensors**.

It also supports for loops when iterating over tensors. However, it only works on a subset of Python.

For example, we cannot use global variables, Python generators (`yield`), complex list comprehensions, variable length function arguments (`*args` or `**kwargs`), or match statements.

Moreover, types must be fixed,[73] and we can only call other functions if they also respect these rules, so no standard library, no third-party libraries, etc. This sounds daunting, but for most real-world models, these rules are actually not too hard to respect, and we can save our model like this:

[73] A function cannot return an integer in some cases and a float in others.

```python
torchscript_model = torch.jit.script(model)
```

Regardless of whether we use tracing or scripting to produce your TorchScript model, we can then further optimize it:

```python
optimized_model = torch.jit.optimize_for_inference(torchscript_model)
```

TorchScript models can only be used for inference, not for training, since the TorchScript environment doesn't support gradient tracking or parameter updates. Finally, you can save a TorchScript model using its `save()` method:

And then load it using the `torch.jit.load()` function:

```python
loaded_torchscript_model = torch.jit.load("my_fashion_mnist_torchscript.pt")
```

One important caveat:

> TorchScript is **no longer under active development**. Bugs are fixed but no new features are added.

It still works fine and it remains one of the best ways to run your PyTorch models in a C++ environment,[74] but since the release of *PyTorch* 2.0 in March 2023, the PyTorch team has been focusing its efforts on a new set of compilation tools centered around the `torch.compile()` function, which we can use very easily:

[74] Another popular option is exporting your PyTorch model to the open *ONNX* standard using `torch.onnx.export()`. The *ONNX* model can then be used for inference in a wide variety of environments.

```python
compiled_model = torch.compile(model)
```

The resulting model can now be used normally, and it will automatically be compiled and optimized when you use it. This is called Just-In-Time (JIT) compilation, as opposed to Ahead-Of-Time (AOT) compilation. Under the hood, `torch.compile()` relies on TorchDynamo (or Dynamo for short) which hooks directly into Python bytecode to capture the model's computation graph at inference time.

Having access to the bytecode allows Dynamo to efficiently and reliably capture the computation graph, properly handling conditionals and loops, while also benefiting from dynamic information that can be used to better optimize the model. The actual compilation and optimization is performed by default by a backend component named TorchInductor, which in turn relies on the Triton language to generate highly efficient GPU code (Nvidia only), or on the OpenMP API for CPU optimization. PyTorch 2.x offers a few other optimization backends, including the XLA backend for Google's TPU devices: just set `device="xla"` when calling `torch.compile()`.

References

1. Cheng, H.-T. *et al. Wide and deep learning for recommender systems* in *Proceedings of the 1st workshop on deep learning for recommender systems* (2016), 7–10.

Chapter 3

Training Deep Neural Networks

Table of Contents

3.1 The Vanishing/Exploding Gradients Problems 84
3.2 Learning Rate Scheduling 116
3.3 Avoiding Overfitting Through Regularization 125

In Chapter 2 we've built, trained, and fine-tuned several ANNs using **PyTorch**. But they were **shallow nets** with just a few hidden layers.

> What if we need to tackle a complex problem, such as detecting hundreds of types of objects in high-resolution images?

We may need to train a much deeper ANN, perhaps with dozens or even *hundreds* of layers, each containing hundreds of neurons, linked by hundreds of thousands of connections. Training a DNN isn't a walk in the park.

Here are some of the problems we could run into:

- Gradients growing smaller or larger when flowing backward through the DNN during training.[1]
- Not having enough training data for such a large network, or the data being too costly to label.
- Training taking impractical amount of time.
- A model with millions of parameters risks severely overfitting the training set, especially if there are not enough training instances or if they are **too noisy**.

[1] Both of these problems make lower layers very hard to train.

Chapter Structure In this chapter we will go through each of these problems and present different techniques to solve them. We will start by exploring the **vanishing and exploding gradients**

problems and some of their most popular solutions, including:

> smart weight initialization, better activation functions, batch-norm, Layer Normalization (LN), and gradient clipping.

Next, we will look at **transfer learning** and **unsupervised pretraining**, which can help us tackling complex tasks even when we have little labeled data. Then we will discuss a variety of optimizers which can speed up training large models tremendously. We will also discuss how we can tweak the learning rate during training to speed up training and produce better models.

Finally, we will cover a few popular regularization techniques for large ANNs:

> ℓ_1 and ℓ_2 regularization, dropout, Monte Carlo dropout, and max-norm regularization.

With these tools, we will be able to train all sorts of DNNs.

3.1 The Vanishing/Exploding Gradients Problems

As we have seen in Chapter 1, the **backpropagation algorithm**'s 2nd-phase works by going from the output layer to the input layer, propagating the error gradient along the way.

> Once the algorithm has computed the gradient of the cost function with regard to each parameter in the network, it uses these gradients to update each parameter with a gradient descent step.

Unfortunately, gradients often get smaller and smaller as the algorithm progresses down to the lower layers. As a result, the gradient descent update leaves the lower layers' connection weights virtually unchanged, and training never converges to a good solution. [1]

> This is called the vanishing gradients problem [2].

Interestingly in some cases, the opposite can happen:

> the gradients can grow bigger and bigger until layers get insanely large weight updates and the algorithm diverges.

> This is the exploding gradients problem.[2] [3]

[2] which surfaces most often in Recurrent Neural Networks (RNN)s.

More generally, DNNs suffer from unstable gradients where different layers may learn at widely different speeds.

This unfortunate behavior was empirically observed long ago, and it was one of the reasons DNNs were mostly abandoned in the early 2000s. It wasn't clear what caused the gradients to be so unstable when training a DNN, but some light was shed in [4], where the authors found a few

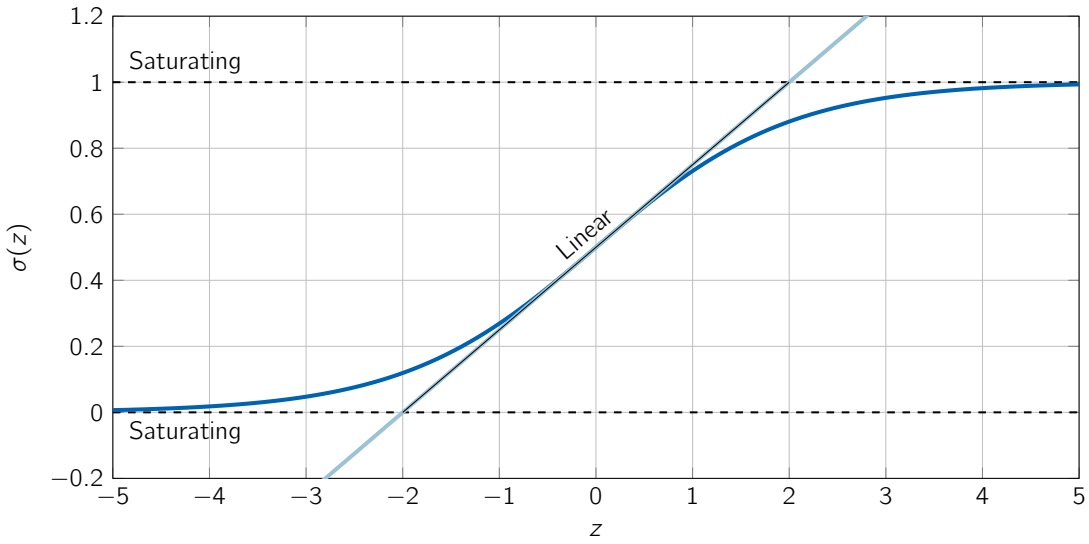


Figure 3.1
Sigmoid activation function saturation.

suspects, including the combination of the popular sigmoid (logistic) activation function and the weight initialization technique that was most popular at the time.[3]

In short, the authors showed that with this activation function and this initialization scheme, the variance of the outputs of each layer is much greater than the variance of its inputs. Going forward in the network, the variance keeps increasing after each layer until the activation function saturates at the top layers. This saturation is actually made worse by the fact that the sigmoid function has a mean of 0.5, not 0.[4]

Looking at the *sigmoid* activation function, we see that when inputs become large,[5] the function saturates at 0 or 1, with a derivative extremely close to 0.[6] Therefore, when backpropagation kicks in it has virtually no gradient to propagate back through the network, and what little gradient exists keeps getting diluted as backpropagation progresses down through the top layers, so there is really nothing left for the lower layers.

[3] i.e., a normal distribution with a mean of 0 and a standard deviation of 1

[4] the hyperbolic tangent function has a mean of 0 and behaves slightly better than the sigmoid function in DNNs.

[5] either in the negative or positive direction.

[6] i.e., the curve is flat at both extremes, meaning the rate of change becomes vanishingly small.

3.1.1 Glorot Initialization and He Initialization

In [4], the authors proposed a way to significantly alleviate the unstable gradients problem. They pointed out that we need the signal to flow properly in **both directions**:

> in the forward direction when making predictions, and in the reverse direction when backpropagating gradients.

We don't want the signal to die out, nor do we want it to explode and saturate. For the signal to flow properly, the authors argued that we need the variance of the outputs of each layer to be equal to the variance of its inputs,[7] and we need the gradients to have equal variance before and after flowing through a layer in the reverse direction.

[7] Here's an analogy: if we set a microphone amplifier's volume knob too close to zero, people won't hear our voice, but if we set it too close to the max, our voice will be saturated and people won't understand what we are saying. Now imagine a chain of such amplifiers: they all need to be set properly in order for our voice to come out loud and clear at the end of the chain. Our voice has to come out of each amplifier at the same amplitude as it came in.

It is actually **NOT** possible to guarantee both unless the layer has an equal number of inputs and outputs,[8] but the authors proposed a good compromise which has proven to work very well in practice, where the connection weights of each layer must be initialized randomly, as described in Eq. (3.1), where

$$\text{fan}_{avg} = (\text{fan}_{in} + \text{fan}_{out})/2.$$

This initialization strategy in literature is called Xavier initialization or Glorot initialization, after the authors of [4].

$$\begin{aligned} &\text{Normal distribution with mean 0 and variance} \sigma^2 = \frac{1}{\text{fan}_{avg}} \\ &\text{Or a uniform distribution between } r \text{ and } +r \text{ with} r = \sqrt{\frac{3}{\text{fan}_{avg}}} \end{aligned} \tag{3.1}$$

If we replace fan_{avg} with fan_{in} in Eq. (3.1), we get an initialization strategy *Yann LeCun* proposed in the 1990s. He called it LeCun initialization.[9]

LeCun initialization is equivalent to Glorot initialization when $\text{fan}_{in} = \text{fan}_{out}$.

It took over a decade for researchers to realize how important this trick is.

Using **Glorot initialization** can speed up training considerably, and it is one of the tricks that led to the success of DL.

Some papers have provided similar strategies for different activation functions, most notably in [5]. These strategies differ only by the scale of the variance and whether they use fan_{avg} or fan_{in}, as shown in **Tbl.** 3.1.[10] The initialization strategy proposed for the Rectified Linear Unit (RELU) activation function and its variants is called He initialization or Kaiming initialization, after the paper's first author.

For Scaled Exponential Linear Unit (SELU), use *Yann LeCun*'s initialization method, preferably with a normal distribution.

Table 3.1 **Initialization parameters for each type of activation function.**

Initialisation	Activation Functions	Variance
Xavier Glorot	None, tanh, sigmoid, softmax	$\frac{1}{fan_{avg}}$
Kaiming He	ReLU, Leaky ReLU, ELU, GELU, Swish, Mish, SwiGLU, ReLU2	$\frac{2}{fan_{in}}$
Yann LeCun	SELU	$\frac{1}{fan_{in}}$

For historical reasons, PyTorch's `nn.Linear` module initializes its weights using **Kaiming uniform initialization**, except the weights are scaled down by a factor of $\sqrt{6}$.[11] Unfortunately, this is not the optimal scale for any common activation function.

[8] these numbers are called the *fan-in* and *fan-out* of the layer.

[9] *Genevieve Orr* and *Klaus-Robert Müller* even recommended it in their 1998 book *Neural Networks: Tricks of the Trade*.

[10] for the uniform distribution, just use $r = \sqrt{3\sigma^2}$

[11] and the bias terms are also initialized randomly.

One solution is to simply multiply the weights by $\sqrt{6}$ just after creating the `nn.Linear` layer to get proper *Kaiming initialization*. To do this, we can update the parameter's `data` attribute. We will also zero out the biases, as there's no benefit in randomly initializing them:

```python
import torch
import torch.nn as nn

layer = nn.Linear(40, 10)
layer.weight.data *= 6 ** 0.5  # Kaiming init (or 3 ** 0.5 for LeCun init)
torch.zero_(layer.bias.data)
```

This works, but it's clearer and less error-prone to use one of the initialization functions available in the `torch.nn.init` module:

```python
layer = nn.Linear(40, 10)
nn.init.kaiming_uniform_(layer.weight)
nn.init.zeros_(layer.bias)
```

If we want to apply the same initialization method to the weights of every `nn.Linear` layer in a model, we can do so in the model's constructor, after creating each `nn.Linear` layer. Alternatively, we can write a subclass of the `nn.Linear` class and tweak its constructor to initialize the weights as we wish. But arguably the simplest option is to write a little function that takes a module, checks whether it's an instance of the `nn.Linear` class, and if so, applies the desired initialization function to its weights.

We can then apply this function to the model and all of its submodules by passing it to the model's `apply()` method. For example:

```python
def use_he_init(module):
    if isinstance(module, nn.Linear):
        nn.init.kaiming_uniform_(module.weight)
        nn.init.zeros_(module.bias)

model = nn.Sequential(nn.Linear(50, 40), nn.ReLU(), nn.Linear(40, 1), nn.ReLU())
model.apply(use_he_init)
```

The `torch.nn.init` module also contains an `orthogonal_()` function which initializes the weights using a random orthogonal matrix, as proposed in [6].

Usefulness of Orthogonality Orthogonal matrices have a number of useful mathematical properties, including the fact that they **preserve norms**:

> given an orthogonal matrix **W** and an input vector **x**, the norm of **Wx** is equal to the norm of **x**, and therefore the magnitude of the inputs is preserved in the outputs. When the inputs are standardized, this results in a stable variance through the layer, which prevents the activations and gradients from vanishing or exploding in a DNN.

This initialization technique is much less common than the initialization techniques discussed earlier,

but it can work well in RNNs or General Adveserial Networks (GAN).

Scaling the weights properly will give a DNN a much better starting point for training.

Scale Down the Weights 3.1 Information

In a classifier, it's generally a good idea to scale down the weights of the output layer during initialization.[12] Indeed, this will result in smaller logits at the beginning of training, which means they will be closer together, and hence the estimated probabilities will also be closer together.

In other words, it encourages the model to be less confident about its predictions when training starts. This will avoid extreme losses and huge gradients that can often make the model's weights bounce around randomly at the start of training, losing time and potentially preventing the model from learning anything.

[12] e.g., by a factor of 10.

3.1.2 Better Activation Functions

One of the insights in [4] was that the problems with unstable gradients were in part due to a **poor choice of activation function**.

> Until then most people had assumed that if nature had chosen to use something pretty close to *sigmoid activation functions* in biological neurons, they must be an excellent choice.

But it turns out that other activation functions behave much better in DNNs, particularly, the RELU activation function, mostly because it does not saturate for positive values, and also because it is very fast to compute.

Unfortunately, the RELU activation function is **NOT** perfect, as it suffers from a problem known as the dying ReLUs:

> during training, some neurons effectively "die", meaning they stop outputting anything other than 0.

In some cases, we may find that half of our network's neurons are **dead**, especially if we've used a large learning rate.

A neuron dies when its weights get tweaked in such a way that the input of the RELU function[13] is negative for all instances in the training set.

[13] i.e., the weighted sum of the neuron's inputs plus its bias term.

When this happens, the neuron just keeps outputting zeros, and gradient descent does not affect it anymore because the gradient of the RELU function is zero when its input is negative.[14]

[14] A dead neuron may come back to life if its inputs evolve over time and eventually return within a range where the RELU activation function gets a positive input again. For example, this may happen if gradient descent tweaks the neurons in the layers below the dead neuron.

To solve this problem, you may want to use a variant of the RELU function, such as the **leaky ReLU**.

Leaky ReLU

The leaky RELU activation function is defined as [7]:

$$\text{LeakyReLU}_\alpha(z) = \max(\alpha z, z))$$

The hyperparameter, α defines how much the function "leaks":

> it is the slope of the function for $z < 0$.

Having a slope for $z < 0$ ensures that leaky RELUs never actually die.[15] A 2015 paper [8] compared several variants of the RELU activation function, and one of its conclusions was that the leaky variants always outperformed the strict RELU activation function.

[15] They can go into a *dormant state*, but have a chance to eventually wake up.

> In fact, setting $\alpha = 0.2$, which is a huge leak, seemed to result in better performance than $\alpha = 0.01$, which is a small leak.

The paper also evaluated the Randomized ReLU (RRELU), where α is picked randomly in a given range during training and is fixed to an average value during testing. RRELU also performed fairly well and seemed to act as a regularizer, reducing the risk of overfitting. Finally, the paper evaluated the Parametric ReLU (PRELU), where α is authorized to be learned during training:

> instead of being a hyperparameter, it becomes a parameter that can be modified by backpropagation like any other parameter.

> PRELU was reported to strongly outperform RELU on **large image datasets**, but on smaller datasets it runs the **risk of overfitting the training set**.

As we might expect, PyTorch includes modules for each of these activation functions:

> `nn.LeakyReLU`, `nn.RReLU`, and `nn.PReLU`.

Just like for other RELU variants, we should use these along with *Kaiming initialization*, but the variance should be slightly smaller due to the negative slope: it should be scaled down by a factor of $1 + \alpha^2$. PyTorch supports this:

> we can pass the α hyperparameter to the `kaiming_uni form_()` and `kaiming_nor mal_()` functions, along with `nonlinearity="leaky_relu"` to get the appropriately adjusted *Kaiming initialization*:

```python
torch.manual_seed(42)
alpha = 0.2
model = nn.Sequential(nn.Linear(50, 40),
                      nn.LeakyReLU(negative_slope=alpha))
```

Continues on next page →

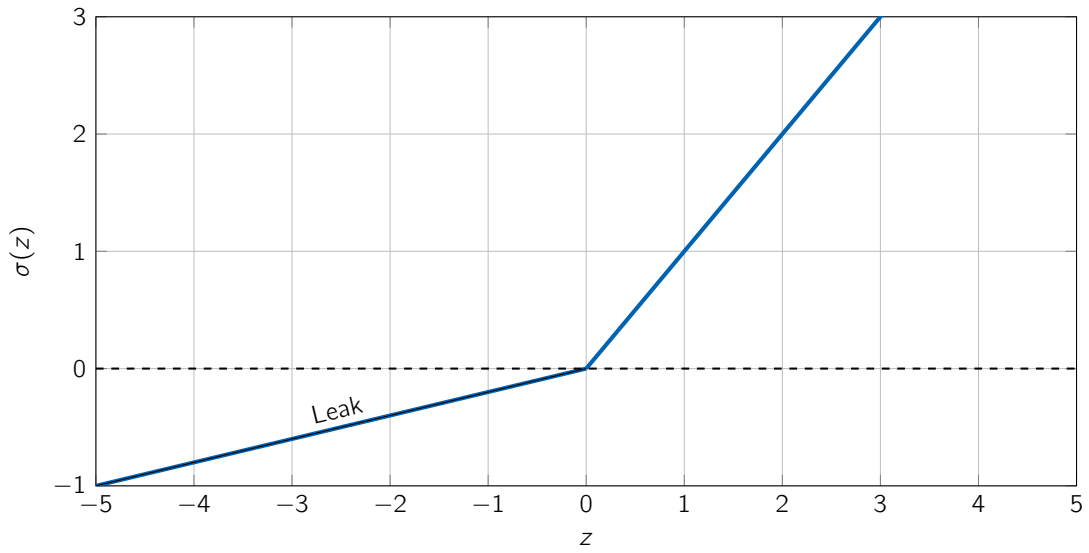


Figure 3.2
Leaky ReLU: like ReLU, but with a small slope for negative values.

← Continued from previous page

```python
nn.init.kaiming_uniform_(model[0].weight, alpha, nonlinearity="leaky_relu")
model(torch.rand(2, 50)).shape
```

RELU, leaky RELU, and PRELU all suffer from the fact that they are not smooth functions as their slopes abruptly change at $z = 0$. As we saw in **B.Sc Data Science I** when we discussed Least Absolute Shrinkage and Selection Operator (LASSO), this sort of discontinuity in the derivatives can make gradient descent bounce around the optimum and slow down convergence.

So now we will look at some smooth variants of the ReLU activation function, starting with Exponential Linear Unit (ELU) and SELU.

ELU and SELU

In 2015, a paper by *Djork-Arné Clevert* et al. [9] proposed a new activation function, called ELU, which outperformed all the RELU variants in the authors' experiments: training time was reduced, and the NN performed better on the test set. Eq. (3.2) shows this activation function's definition.

$$\text{ELU}_\alpha(x) = \begin{cases} \alpha(\exp(z) - 1) & \text{if} \quad z < 0, \\ z & \text{if} \quad z \geqslant 0. \end{cases} \tag{3.2}$$

The ELU activation function looks a lot like the RELU function, seen in **Fig.** 3.3, with a few major differences:

- It takes on negative values when $z < 0$, which allows the unit to have an average output closer to 0 and helps alleviate the vanishing gradients problem.

  The hyperparameter α defines the opposite of the value that the ELU function approaches when z is a large negative number. It is usually set to 1, but we can tweak it like any other hyperparameter.

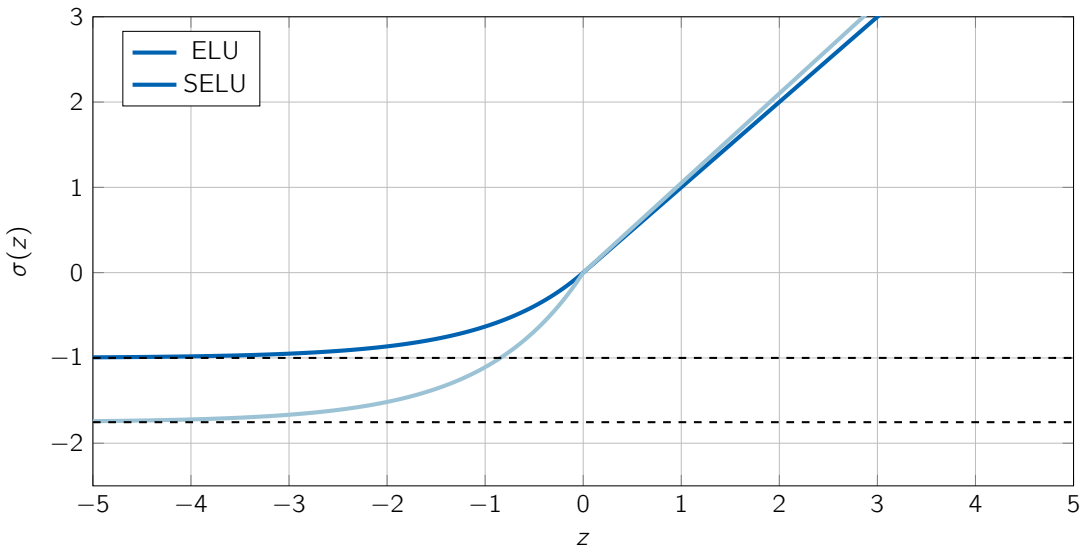


Figure 3.3
ELU and SELU activation functions.

- It has a nonzero gradient for $z < 0$, which **avoids the dead neurons problem**.
- If α is equal to 1, then the function is smooth everywhere, including around $z = 0$, which helps speed up gradient descent since it does not bounce as much to the left and right of $z = 0$.

Using ELU with PyTorch is as easy as using the `nn.ELU` module, along with *Kaiming initialization*. The main drawback of the ELU activation function is that it is slower to compute than the ReLU function and its variants.[16]

[16] due to the use of the exponential function.

> Its faster convergence rate during training may compensate for that slow computation, but still, at test time an ELU network will be a bit slower than a RELU network.

Not long after, a 2017 paper by *Günter Klambauer* et al. [10] introduced the scaled ELU (SELU) activation function:

> as its name suggests, it is a scaled variant of the ELU activation function, which is about 1.05 times ELU, using $\alpha \approx 1.67$.

The authors showed that if we build a NN composed exclusively of a stack of dense layers,[17] and if all hidden layers use the SELU activation function, then the network will **self-normalize**:

[17] i.e., MLP.

> the output of each layer will tend to preserve a mean of 0 and a standard deviation of 1 during training, which solves the vanishing/exploding gradients problem.

As a result, the SELU activation function may outperform other activation functions for MLPs, especially deep ones. To use it with PyTorch, just use `nn.SELU`. There are, however, a few conditions for self-normalization to happen:

- The input features must be standardized, meaning a mean 0 and standard deviation 1.

- Every hidden layer's weights must be initialized using *LeCun normal initialization*.
- The self-normalizing property is only guaranteed with plain MLPs. If we try to use SELU in other architectures, like RNN or networks with skip connections,[18] it will probably not outperform ELU.
- We cannot use regularization techniques like ℓ_1 or ℓ_2 regularization, batch-norm, layer-norm, max-norm, or regular dropout.

[18] i.e., connections that skip layers, such as in Wide & Deep neural networks.

These are significant constraints, so despite its promises, SELU did not gain a lot of traction. Moreover, other activation functions seem to outperform it quite consistently on most tasks. Let's look at some of the most popular ones.

GELU, Swish, SwiGLU, Mish, and RELU2

The *Gaussian Error Linear Unit (GELU)* was introduced in a 2016 paper by *Dan Hendrycks* and *Kevin Gimpel* [11]. Once again, we can think of it as a smooth variant of the RELU activation function. Its definition is given in Eq. (**3.3**), where Φ is the standard Gaussian Cumulative Density Function (CDF):

> $\Phi(z)$ corresponds to the probability that a value sampled randomly from a normal distribution of mean 0 and variance 1 is lower than z.

$$\text{GELU}(x) = z\Phi(z) \tag{3.3}$$

As we can see in **Fig.** 3.4, GELU resembles RELU:

> it approaches 0 when its input z is very negative, and it approaches z when z is very positive.

However, whereas all the activation functions we've discussed so far were both convex and monotonic,[19] the GELU activation function is neither: from left to right, it starts by going straight, then it wiggles down, reaches a low point around 0.17, near $z0.75$), and finally bounces up and ends up going straight toward the top right. This fairly complex shape and the fact that it has a curvature at every point may explain why it works so well, especially for complex tasks:

> gradient descent may find it easier to fit complex patterns.

[19] A function is convex if the line segment between any two points on the curve never lies below the curve. A monotonic function only increases, or only decreases.

In practice, it often outperforms every other activation function discussed so far. However, it is a bit more computationally intensive, and the performance boost it provides is not always sufficient to justify the extra cost. That said, it is possible to show that it is approximately equal to $z\sigma(1.702z)$, where σ is the sigmoid function:

> using this approximation also works very well, and it has the advantage of being much faster to compute.

The GELU paper also introduced the *sigmoid linear unit (SiLU)* activation function, which is equal to $z\sigma(z)$, but it was outperformed by GELU in the authors' tests.

Interestingly, a 2017 paper [12] rediscovered the SiLU function by automatically searching for good activation functions. The authors named it Swish, and the name caught on. In their paper, Swish outperformed every other function, including GELU. *Ramachandran et al.* later generalized Swish by adding an extra scalar hyperparameter (β) to scale the sigmoid function's input. The generalized Swish function is $\text{Swish}_\beta(z) = z\sigma(\beta z)$, so GELU is approximately equal to the generalized Swish function using $\beta = 1.702$. We can tune β like any other hyperparameter. Alternatively, it's also possible to make β trainable and let gradient descent optimize it, which is a bit like PRELU:

> there is typically a single trainable β parameter for the whole model, or just one per layer, to keep the model efficient and **avoid overfitting**.

A popular Swish variant is SwiGLU [13]: the inputs go through the Swish activation function, and in parallel through a linear layer, then both outputs are multiplied item-wise. That's:

$$\text{SwiGLU}(\mathbf{z}) = \text{Swish}_\beta(\mathbf{z}) \otimes \text{Linear}(\mathbf{z})$$

This is often implemented by doubling the output dimensions of the previous linear layer, then splitting the outputs in two (**2**) along the feature dimension to get $\mathbf{z}_1$ and $\mathbf{z}_2$, and finally applying:

$$\text{SwiGLU}(\mathbf{z}) = \text{Swish}_\beta(\mathbf{z}_1) \otimes \mathbf{z}_2$$

This is a variant of the gated linear unit (GLU) [14] introduced by Facebook researchers in 2016. The itemwise multiplication gives the model more expressive power, allowing it to learn when to turn off (i.e., multiply by 0) or amplify specific features: this is called a **gating mechanism**. SwiGLU is very common in modern transformers, which will be the focus in Chapter **??**.

Another GELU-like activation function is Mish, which was introduced in a 2019 paper [15]. It is

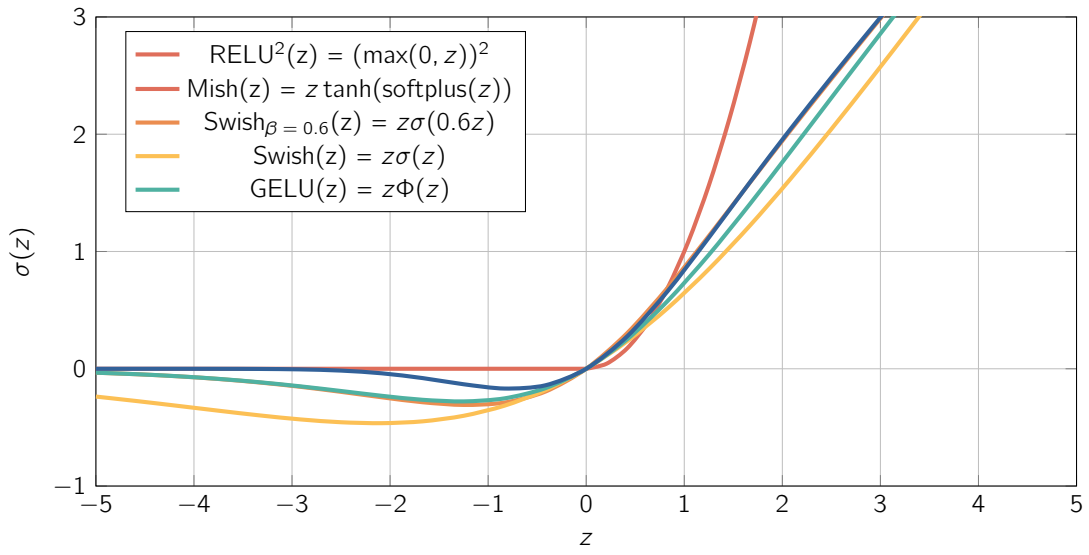


Figure 3.4 GELU, Swish, parametrized Swish, Mish, and ReLU² activation functions .

defined as:

$$\text{mish}(z) = z\tanh(\text{softplus}(z)) \qquad \text{where} \qquad \text{softplus}(z) = \log(1+\exp(z))$$

Just like GELU and Swish, it is a smooth, nonconvex, and nonmonotonic variant of ReLU, and once again the author ran many experiments and found that Mish generally outperformed other activation functionseven Swish and GELU, by a tiny margin. Figure 11-4 shows GELU, Swish (both with the default = 1 and with = 0.6), and lastly Mish. As you can see, Mish overlaps almost perfectly with Swish when z is negative, and almost perfectly with GELU when z is positive.

Lastly, in 2021, Google researchers ran an automated architecture search to improve large transformers, and the search found a very simple yet effective activation function: RELU^2 [16]. As its name suggests, it's simply RELU squared:

$$\text{ReLU}^2 = (\max(0, z))^2$$

It has all the qualities of RELU[20] but it also has smooth gradients at $z = 0$, and it often outperforms other activation functions, especially for sparse models. However, training can be less stable, due to its increased sensitivity to outliers and dying RELUs.

[20] simplicity, computational efficiency, sparse output, no saturation on the positive side.

Choosing the Right Activation Function — 3.2 Information

So, which activation function should we use for the hidden layers of our DNNs?

ReLU remains a **good default for most tasks**. It is often just as good as the more sophisticated activation functions, plus it's very fast to compute, and many libraries and hardware accelerators provide RELU-specific optimizations.

Swish is probably a better default for complex tasks, and we can even try **parametrized Swish** with a learnable β parameter for the most complex tasks.

Mish and **SwiGLU** may give you slightly better results, but they require a bit more computation effort. Therefore, if we care a lot about runtime latency, then you may prefer leaky ReLU, or parametrized leaky ReLU for complex tasks, or even ReLU, especially for sparse models.

PyTorch supports GELU, Mish, and Swish out of the box, using `nn.GELU`, `nn.Mish`, `nn.SiLU` respectively. To implement SwiGLU, double the previous linear layer's output dimension, then use `z1, z2 = z.chunk(2, dim=-1)` to split its output in two, and compute `F.silu(beta * z1) * z2`, where `F` is `torch.nn.functional`.

For RELU^2, simply compute `F.relu(z).square()`. PyTorch also includes simplified and approximated versions of several activation functions, which are much faster to compute and often more stable during training. These simplified versions have names starting with Hard, such as `nn.HardSigmoid`, `nn.Hardtanh`, and `nn.Hardswish`, and they are often used on mobile devices. That's all for activation functions! Now, let's look at a completely different way to solve the unstable gradients problem: batch normalization.

3.1.3 Batch Normalization

Although using *Kaiming initialization* along with ReLU[21] can significantly reduce the danger of the vanishing/exploding gradients problems at the beginning of training, it doesn't guarantee it won't come back during training.

[21] or any one of its variants.

In a 2015 paper [17], *Sergey Ioffe* and *Christian Szegedy* proposed a technique called Batch Normalization (BN) which addresses these problems. The technique consists of adding an operation in the model just before or after the activation function of each hidden layer. This operation simply zero-centers and normalizes each input, then scales and shifts the result using two new parameter vectors per layer:

> one for scaling, the other for shifting, or in other words, the operation lets the model learn the optimal scale and mean of each of the layer's inputs.

In many cases, if we were to add a BN layer as the very first layer of our NN, we do not need to standardize our training set without needing `StandardScaler` as the BN layer will do it for us.[22]

[22] well, approximately, as it only looks at one batch at a time, and it can also rescale and shift each input feature.

To zero-center and normalize the inputs, the algorithm needs to estimate each input's *mean* and *standard deviation*. It does this by evaluating the mean and standard deviation of the input over the current mini-batch.[23] The whole operation is summarized as:

[23] hence the name "batch normalization".

$$\begin{aligned} &1.\quad \mu_B = \frac{1}{m_B}\sum_{i=1}^{m_B}\mathbf{x}^{(i)} \qquad 2.\quad \sigma_B^2 = \frac{1}{m_B}\sum_{i=1}^{m_B}\left(\mathbf{x}^{(i)} - \mu_B\right)^2 \\ &3.\quad \hat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \qquad 4.\quad \mathbf{z}^{(i)} = \gamma \otimes \hat{\mathbf{x}}^{(i)} + \beta \end{aligned} \tag{3.4}$$

In this algorithm:

- μ_B is the vector of input means, evaluated over the whole mini-batch B.[24]
- m_B is the number of instances in the mini-batch.
- $\mathbf{x}^{(i)}$ is the input vector of the batch-norm layer for instance i.
- σ_B is the vector of input standard deviations, also evaluated over the whole mini-batch.[25]
- $\hat{x}_{(i)}$ is the vector of zero-centered and normalized inputs for instance i.
- ϵ is a tiny number that avoids division by zero and ensures the gradients don't grow too large, which is typically 1×10^{-5}. This is called a smoothing term.
- γ is the output scale parameter vector for the layer and it contains one scale parameter per input.
- $\otimes$ represents element-wise multiplication where each input is multiplied by its corresponding output scale parameter.

[24] it contains one mean per input.

[25] it contains one standard deviation per input.

- β is the output shift, or offset, parameter vector for the layer as it contains one shift parameter per input. Each input is offset by its corresponding shift parameter.
- $\mathbf{z}^{(i)}$ is the output of the BN operation. It is a rescaled and shifted version of the inputs.

So during training, BN standardizes its inputs, then re-scales and offsets them.

What about at test time?

Well, it's not that simple. Indeed, we may need to make predictions for individual instances rather than for batches of instances:

in this case, we will have no way to compute each input's standard deviation. Moreover, even if we do have a batch of instances, it may be too small, or the instances may not be independent and identically distributed, so computing statistics over the batch instances would be unreliable.

One solution is to wait until the end of training, then run the whole training set through the NN and compute the mean and standard deviation of each input of the BN layer. These "final" input means and standard deviations can then be used instead of the batch input means and standard deviations when making predictions.

However, most implementations of batch norm estimate these final statistics during training by using a moving average of the layer's batch input means and variances.

This is what PyTorch does automatically when you use its batch-norm layers, such as `nn.BatchNorm1d`.

To sum up, four (**4**) parameter vectors are learned in each batch-norm layer:

γ, the output scale vector, and β, the output offset vector, are learned through regular backpropagation, and μ, the final input mean vector, and σ^2, the final input variance vector, are estimated using an exponential moving average.

Note that μ and σ^2 are estimated during training, but they are used only after training, once you switch the model to evaluation mode using `model.eval()`: μ and σ^2 then replace μ and σB^2 in Eq. (**3.4**).

In their paper, *Ioffe* and *Szegedy* demonstrated that batch norm considerably improved all the DNN they experimented with, leading to a huge improvement in the ImageNet classification task.[26] The vanishing gradients problem was strongly reduced, to the point that they could use saturating activation functions such as tanh and even sigmoid. The networks were also much less sensitive to the weight initialization. The authors were able to use much larger learning rates, significantly speeding up the learning process.

[26] ImageNet is a large database of images classified into many classes, commonly used to evaluate computer vision systems.

Specifically, they note that:

> Applied to a state-of-the-art image classification model, BN achieves the same accuracy with 14 times fewer training steps, and beats the original model by a significant margin. [···] Using an ensemble of batch-normalized networks, we improve upon the best published result on ImageNet classification: reaching 4.9% top-5 validation error, and 4.8% test error, exceeding the accuracy of human raters.

Finally, like a gift that keeps on giving, batch norm acts like a regularizer, reducing the need for other regularization techniques.[27]

[27] such as dropout, described later in this chapter.

BN does, however, add some complexity to the model.[28] Moreover, there is a runtime penalty: the NN makes slower predictions due to the extra computations required at each layer. Fortunately, it's often possible to fuse the BN layer with the previous layer after training, thereby avoiding the runtime penalty.

[28] although it can remove the need for normalizing the input data.

This is done by updating the previous layer's weights and biases so that it directly produces outputs of the appropriate scale and offset. For example, if the previous layer computes $\mathbf{XW} + \mathbf{b}$, then the BN layer will compute:

$$\gamma \otimes \frac{(\mathbf{XW} + \mathbf{b})}{\sigma} + \beta \qquad \text{ignoring the smoothing term } (\epsilon) \text{ in the denominator}$$

If we define:

$$\mathbf{W}' = \gamma \otimes \frac{\mathbf{W}}{\sigma} \qquad \text{and} \qquad \mathbf{b}' = \gamma \otimes \frac{\mathbf{b} - \mu}{\sigma} + \beta$$

In which case, the equation simplifies to $\mathbf{XW}' + \mathbf{b}'$. So, if we replace the previous layer's weights and biases ($\mathbf{W}$ and $\mathbf{b}$) with the updated weights and biases ($\mathbf{W}'$ and $\mathbf{b}'$), we can get rid of the BN layer. This is one of the optimizations performed by `optimize_for_inference()`.

3.3 Information

This Method Feels Slower

It may seem like that training is rather slow, because each epoch takes much more time when you use batch norm. This is usually counterbalanced by the fact that convergence is much faster with BN, so it will take fewer epochs to reach the same performance.

Implementing Batch-Norm with PyTorch

As with most things in PyTorch, implementing batch norm is straightforward and intuitive. Just add an `nn.BatchNorm1d` layer before or after each hidden layer's activation function, and specify the number of inputs of each BN layer. We may also add a BN layer as the first layer in our model, which removes the need to standardize the inputs manually.

[29] similar to the one we built in Chapter 1.

For example, let's create a Fashion MNIST image classifier[29] using BN as the first layer in the model,[30] then again after each hidden layer:

[30] after flattening the input images.

python

```
torch.manual_seed(42)

model = nn.Sequential(
    nn.Flatten(),
```

Continues on next page →

← Continued from previous page

```python
    nn.BatchNorm1d(1 * 28 * 28),
    nn.Linear(1 * 28 * 28, 300),
    nn.ReLU(),
    nn.BatchNorm1d(300),
    nn.Linear(300, 100),
    nn.ReLU(),
    nn.BatchNorm1d(100),
    nn.Linear(100, 10)
)
```

We can now train the model normally, as we did in Chapter 1.

In this tiny example with just two (2) hidden layers, batch norm is unlikely to have a large impact, but for DNN it can make a tremendous difference.

Different Modes for Different Norms 3.4 Information

Since batch norm behaves differently during training and during evaluation, it's critical to switch to training mode during training using `model.train()`, and switch to evaluation mode during evaluation using `model.eval()`.

Forgetting to do so is one of the most common mistakes.

If you look at the parameters of the first BN layer, we will find two, which are `weight` and `bias`, which correspond to γ and β in Eq. (3.4):

```python
dict(model[1].named_parameters()).keys()
```

And if we look at the buffers of this same BN layer, we will find three:

`running_mean`, `running_var`, and `num_batches_tracked`.

The first two correspond to the running means μ and σ^2 discussed earlier, and `num_batches_tracked` simply counts the number of batches seen during training:

```python
dict(model[1].named_buffers()).keys()
```

The authors of the BN paper argued in favor of adding the BN layers before the activation functions, rather than after, which is similar to we just did. There is some debate about this, and it seems to depend on the task, so experiment with this to see which option works best on your dataset.

If we move the BN layers before the activation functions, we can also remove the bias term from the previous `nn.Linear` layers by setting their bias hyperparameter to `False`. Indeed, a batch-norm layer already includes one bias term per input. We can also drop the first BN layer to avoid sandwiching the first hidden layer between two BN layers, but this means you should normalize the training set before training. The updated code looks like this:

```python
torch.manual_seed(42)

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(1 * 28 * 28, 300, bias=False),
    nn.BatchNorm1d(300),
    nn.ReLU(),
    nn.Linear(300, 100, bias=False),
    nn.BatchNorm1d(100),
    nn.ReLU(),
    nn.Linear(100, 10)
)
```

The `nn.BatchNorm1d` class has a few hyperparameters we can tweak. The defaults will usually be fine, but we may occasionally need to tweak the `momentum`. This hyper-parameter is used by the `BatchNorm1d` layer when it updates the exponential moving averages.

Given a new value $\mathbf{v}$,[31] the layer updates the running average $\hat{\mathbf{v}}$ using the following equation:

$$\hat{\mathbf{v}} \leftarrow \mathbf{v} \times \text{momentum} + \hat{\mathbf{v}} \times (1 - \text{momentum})$$

[31] i.e., a new vector of input means or variances computed over the current batch

A good momentum value is typically close to 0:

For example, 0.01 or 0.001.

We want more 0s for smaller mini-batches, and fewer for larger mini-batches. The default is 0.1, which is good for large batch sizes, but not great for small batch sizes such as 32 or 64.

What does Momentum even Mean? 3.5 Information

When people talk about "momentum" in the context of a running mean, they usually refer to the weight of the current running mean in the update equation. Sadly, for historical reasons, PyTorch uses the opposite meaning in the BN layers.

However, other parts of PyTorch use the conventional meaning (e.g., in optimizers), so don't get confused.

Batch norm 1D, 2D, and 3D

In the previous examples, we flattened the input images before sending them through the first `nn.BatchNorm1d` layer. This is because an `nn.BatchNorm1d` layer works on batches of shape `[batch_size, num_features]`, similar to what `nn.Linear layer` does, so we would get an error if you moved it before the `nn.Flatten` layer.

However, we could use an `nn.BatchNorm2d` layer before the `nn.Flatten` layer: indeed, it expects its inputs to be image batches of shape `[batch_size, channels, height, width]`, and it computes the batch mean and variance across both the batch dimension (dimension 0) and the spatial

dimensions (dimensions 2 and 3). This means that all pixels in the same batch and channel get normalized using the same mean and variance: the `nn.BatchNorm2d` layer only has one weight per channel and one bias per channel.[32]

[32] e.g., three weights and three bias terms for color images with three channels for red, green, and blue.

> This generally works better when dealing with image datasets.

There's also an `nn.BatchNorm3d` layer which expects batches of shape `[batch_size, channels, depth, height, width]`

> this is useful for datasets of 3D images, such as CT scans [18].

The `nn.BatchNorm1d` layer can also work on batches of sequences.

The convention in PyTorch is to represent batches of sequences as 3D tensors of shape `[batch_size, sequence_length, num_features]`.

> For example, suppose we work on particle physics and you have a dataset of particle trajectories, where each trajectory is composed of a sequence of 100 points in 3D space, then a batch of 32 trajectories will have a shape of `[32, 100, 3]`. However, the `nn.BatchNorm1d` layer expects the shape to be `[batch_size, num_features, sequence_length]`, and it computes the batch mean and variance across the first and last dimensions to get one mean and variance per feature. So we must permute the last two dimensions of the data using `X.permute(0, 2, 1)` before letting it go through the `nn.BatchNorm1d` layer.

Batch normalization has become one of the most-used layers in DNNs, especially deep Convolutional Neural Networks (CNN)s discussed in Chapter **??**, to the point that it is often omitted in the architecture diagrams:

> With the assumption that BN is added after every layer

However, it is **NOT** perfect. In particular, the computed statistics for an instance are biased by the other samples in a batch, which may reduce performance.[33] Moreover, BN struggles with some architectures, such as RNNs. For these reasons, batch-norm is more and more often replaced by layer-norm.

[33] especially for small batch sizes.

3.1.4 Layer Normalization

LN is very similar to batch norm, but instead of normalizing across the batch dimension, LN normalizes across the feature dimensions. This simple idea was introduced by *Jimmy Lei Ba et al.* in a 2016 paper [19], and initially applied mostly to RNNs.

> However, in recent years it has been successfully applied to many other architectures, such as CNNs, transformers, diffusion nets, and more.

One advantage is that LN can compute the required statistics on the fly, at each time step, independently for each instance. This also means that it behaves the same way during training and testing (as opposed to BN), and it does not need to use exponential moving averages to estimate the feature statistics across all instances in the training set, like BN does. Lastly, LN learns a scale and an offset parameter for each input feature, just like BN does.

PyTorch includes an `nn.LayerNorm` module. To create an instance, we must simply indicate the size of the dimensions that you want to normalize over. These must be the last dimension(s) of the inputs.

> For example, if the inputs are batches of 100×200 RGB images of shape `[3, 100, 200]`, and we want to normalize each image over each of the three color channels separately, we would use the following `nn.LayerNorm` module:

```python
torch.manual_seed(42)

inputs = torch.randn(32, 3, 100, 200)  # a batch of random RGB images
layer_norm = nn.LayerNorm([100, 200])
result = layer_norm(inputs)  # normalizes over the last two dimensions
```

The following code produces the same result:

```python
means = inputs.mean(dim=[2, 3], keepdim=True)
vars_ = inputs.var(dim=[2, 3], keepdim=True, unbiased=False)
stds = torch.sqrt(vars_ + layer_norm.eps)
result2 = layer_norm.weight * (inputs - means) / stds + layer_norm.bias
assert torch.allclose(result, result2)
```

However, most computer vision architectures that use LN normalize over all channels at once. For this, we must include the size of the channel dimension when creating the `nn.LayerNorm` module:

```python
layer_norm = nn.LayerNorm([3, 100, 200])
result = layer_norm(inputs)  # normalizes over the last three dimensions
```

And that's all there is to it. Now let's look at one last technique to stabilize gradients during training: **gradient clipping**.

3.1.5 Gradient Clipping

Another technique to mitigate the exploding gradients problem is to clip the gradients during backpropagation so that they never exceed some threshold. This is called **gradient clipping** [20].

> This technique is generally used in RNNs, where using **batch norm is tricky**.

In PyTorch, gradient clipping is generally implemented by calling either `torch.nn.utils.clip_grad_norm_()` or `torch.nn.utils.clip_grad_value_()` at each iteration during training, right after the gradients are computed, such as after `loss.back ward()`). Both functions take as a first argument the list of model parameters whose gradients must be clippedtypically all of them using `model.parameters()`. The `clip_grad_norm_()` function clips each gradient vector's norm if it exceeds the given `max_norm` argument. This is a hyperparameter you can tune.[34] The `clip_grad_value_()` function independently clips the individual components of the gradient vector between `-clip_value` and `+clip_value`, where `clip_value` is a hyperparameter you can tune.

[34] a typical default value is 1.0.

For example, this training loop clips the norm of each gradient vector to 1.0:

```python
    for epoch in range(n_epochs):
        total_loss = 0.0
        metric.reset()
        model.train()
        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            total_loss += loss.item()
            loss.backward()
            # Uncomment to activate gradient clipping:
            #nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
            optimizer.step()
            optimizer.zero_grad()
            metric.update(y_pred, y_batch)
```

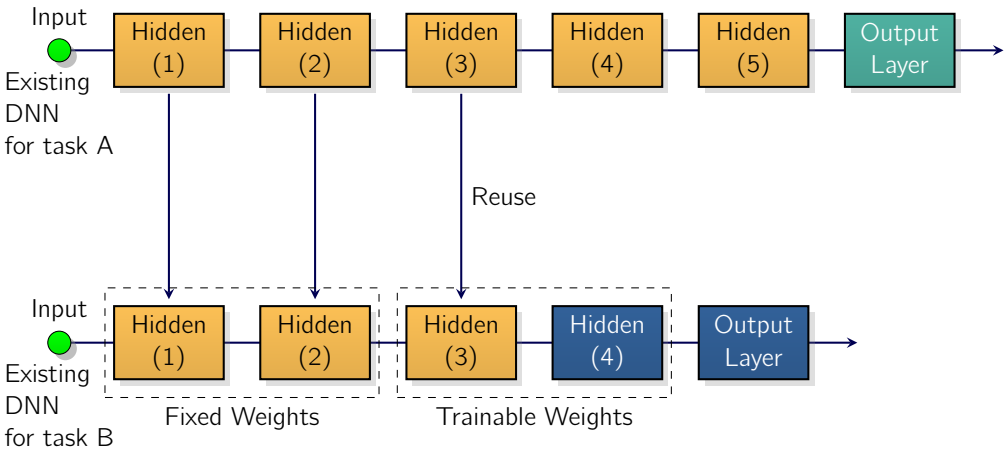
Note that `clip_grad_value_()` will change the orientation of the gradient vector when its components are clipped. For example, if the original gradient vector is `[0.9, 100.0]`, it points mostly in the direction of the second dimension; but once we clip it by value, we get `[0.9, 1.0]`, which points roughly at the diagonal between the two axes. Despite this reorientation, this approach actually works quite well in practice. If we clipped the same vector by norm, the result would be `[0.00899964, 0.9999595]`: this would preserve the vector's orientation, but almost eliminate the first component. The best clipping function to use depends on the dataset.

3.1.6 Reusing Pretrained Layers

It is generally **NOT** a good idea to train a very large DNN from scratch without first trying to find an existing NN which accomplishes a similar task to the one we are trying to tackle. If we find such a NN, then we can generally reuse most of its layers, except for the top ones. This technique is called transfer learning.

> It will **NOT** only speed up training considerably, but also require significantly less training data.

Figure 3.5
Reusing pretrained layers



Suppose we have access to a DNN which was trained to classify pictures into one hundred different categories;

including animals, plants, vehicles, and everyday objects,

and we now want to train a DNN to classify specific types of vehicles. These tasks are very similar, even partly overlapping, so we should try to reuse parts of the first network, which can be seen in **Fig.** 3.5.

3.6 Information

Keep Images The Same Size

If the input pictures for our new task don't have the same size as the ones used in the original task, we will usually have to add a **preprocessing step** to resize them to the size expected by the original model. More generally, **transfer learning** will work best when the inputs have similar low-level features.

For example, a NN trained on regular pictures taken from mobile phones will help with many other tasks on mobile phone pictures, but it will likely not help at all on satellite images or medical images.

The output layer of the original model should usually be replaced because it is most likely not useful at all for the new task, and it may not even have the right number of outputs.

Similarly, the upper hidden layers of the original model are less likely to be as useful as the lower layers, as the high-level features which are most useful for the new task may differ significantly from the ones that were most useful for the original task.

We want to find the right number of layers to reuse.

What is Familiar is also Easy 3.7 Information

The more similar the tasks are, the more layers we will want to reuse.[35] For very similar tasks, try to keep all the hidden layers and just replace the output layer.

[35] starting with the lower layers.

Try freezing all the reused layers first,[36] then train our model and see how it performs. Then try unfreezing one or two of the top hidden layers to let backpropagation tweak them and see if performance improves. The more training data we have, the more layers we can unfreeze. It is also useful to reduce the learning rate when you unfreeze reused layers:

[36] i.e., make their parameters nontrainable by setting `requires_grad` to `False` so gradient descent won't modify them and they will remain fixed.

this will avoid wrecking their fine-tuned weights.

If we still cannot get good performance, and we have little training data, try dropping the top hidden layer(s) and freezing all the remaining hidden layers again. We can iterate until we find the right number of layers to reuse. If we have plenty of training data, we may try replacing the top hidden layers instead of dropping them, and even adding more hidden layers.

3.1.7 Transfer Learning with PyTorch

Let's look at an example. Suppose the *Fashion MNIST* dataset only contained eight classes:

For example, all the classes except for Pullover and T-shirt/top

Someone built and trained a PyTorch model on that set and got reasonably good performance (92% accuracy). Let's call this model A. You now want to tackle a different task:

We have images of T-shirts and pullovers, and you want to train a binary classifier: positive for T-shirt/top, negative for Pullover.

Our dataset is tiny; you only have 20 labeled images! When you train a new model for this task (let's call it model B) with the same architecture as model A, you get 71.6% test accuracy. While drinking your morning coffee, you realize that your task is quite similar to task A, so perhaps transfer learning can help? Let's find out!

First, let's look at model A:

```python
torch.manual_seed(42)

model_A = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 100),
    nn.ReLU(),
    nn.Linear(100, 100),
    nn.ReLU(),
    nn.Linear(100, 100),
    nn.ReLU(),
```

Continues on next page →

← Continued from previous page

```python
    nn.Linear(100, 8)
)
model_A = model_A.to(device)
```

We can now reuse the layers we want, for example, all layers except for the output layer:

```python
import copy

torch.manual_seed(43)
reused_layers = copy.deepcopy(model_A[:-1])
model_B_on_A = nn.Sequential(
    ,*reused_layers,
    nn.Linear(100, 1)  # new output layer for task B
).to(device)
```

In this code, we use Python's `copy.deepcopy()` function to copy all the modules in the nn.Sequential module,[37] except for the last layer. Since we're making a deep copy, all the submodules are copied as well. Then we create `model_B_on_A`, which is an `nn.Sequential` model based on the reused layers of model A, plus a new output layer for task B:

[37] along with all their data and submodules.

it has a single output since task B is binary classification.

You could start training `model_B_on_A` for task B now, but since the new output layer was initialized randomly, it will make large errors,[38] so there will be large error gradients that may wreck the reused weights. To avoid this, one approach is to freeze the reused layers during the first few epochs, giving the new layer some time to learn reasonable weights:

[38] at least during the first few epochs.

```python
for layer in model_B_on_A[:-1]:
    for param in layer.parameters():
        param.requires_grad = False
```

Now we can train `model_B_on_A`. But don't forget that task B is **binary classification**, so we must switch the loss to `nn.BCEWithLogitsLoss`,[39] as we discussed in Chapter 1. Also, if we are using `torchmetrics`, make sure to set `task="binary"` when creating the Accuracy metric:

[39] or to `nn.BCELoss` if you prefer to add an `nn.Sigmoid` activation function on the output layer.

```python
n_epochs = 10
optimizer = torch.optim.SGD(model_B_on_A.parameters(), lr=0.005)
xentropy = nn.BCEWithLogitsLoss()
accuracy = torchmetrics.Accuracy(task="binary").to(device)
history_B = train(model_B_on_A, optimizer, xentropy, accuracy,
                  train_loader_B, valid_loader_B, n_epochs)
```

After we have trained the model for a few epochs, we can unfreeze the reused layers,[40] reduce the learning rate, and continue training to fine-tune the reused layers for task B.

[40] setting `param.requires_grad = True` for all parameters.

So, what's the final verdict? Well, this model's test accuracy is 92.5%, which is much better than the 71.6% accuracy we reached without pretraining.

Here, we might have cheated a bit to get to this result as many configurations were tried beforehand

until one was found that demonstrated a strong improvement. If you try to change the classes or the random seed, you will see that the improvement generally drops, or even vanishes or reverses.

> What was done here is called **torturing the data until it confesses**.

When a paper looks too positive, we should be suspicious. Perhaps the flashy new technique does not actually help much,[41] but the authors tried many variants and reported only the best resultswhich may be due to sheer luckwithout mentioning how many failures they encountered along the way. That's called p-hacking. Most of the time, this is not malicious, but it is part of the reason why so many results in science can never be reproduced.

[41] in fact, it may even degrade performance.

But why did I cheat? It turns out that transfer learning does not work very well with small dense networks, presumably because small networks learn few patterns, and dense networks learn very specific patterns, which are unlikely to be useful for other tasks.

> Transfer learning works best with deep CNNs and with Transformer architectures.

3.1.8 Unsupervised Pretraining

Suppose we want to tackle a complex task for which we don't have much labeled training data, but unfortunately we cannot find a model trained on a similar task.

First, we should try to gather more labeled training data, but if we can't, we may still be able to perform unsupervised pretraining, shown in **Fig.** 3.6.

> It is often cheap to gather unlabeled training examples, but expensive to label them.

If we can gather plenty of unlabeled training data, we can try to use it to train an unsupervised model, such as an autoencoder (see Chapter 18). Then we can reuse the lower layers of the autoencoder, add the output layer for our task on top, and fine-tune the final network using supervised learning.[42]

[42] i.e., with the labeled training examples.

It is this technique that *Geoffrey Hinton* and his team used in 2006, and which led to the revival of ANN and the success of DL. Until 2010, unsupervised pretraining, typically with restricted Boltzmann machines,[43] was the norm for DNNs, and only after the *vanishing gradients problem* was alleviated did it become much more common to train DNNs purely using supervised learning. Unsupervised pretraining[44] is still a good option when you have a complex task to solve, no similar model you can reuse, and little labeled training data, but plenty of unlabeled training data.

[43] A two-layer, stochastic generative ANN which learns a probability distribution over a set of inputs without requiring labels.

[44] Today typically using autoencoders or diffusion models rather than RBMs.

Note that in the early days of deep learning it was difficult to train deep models, so people would use a technique called greedy layer-wise pretraining, as shown in **Fig.** 3.6. They would first train an unsupervised model with a single layer, typically an RBM, then they would freeze that layer and add another one on top of it, then train the model again,[45] then freeze the new layer and add another layer on top of it, train the model again, and so on. Nowadays, things are much simpler: people

[45] effectively just training the new layer.

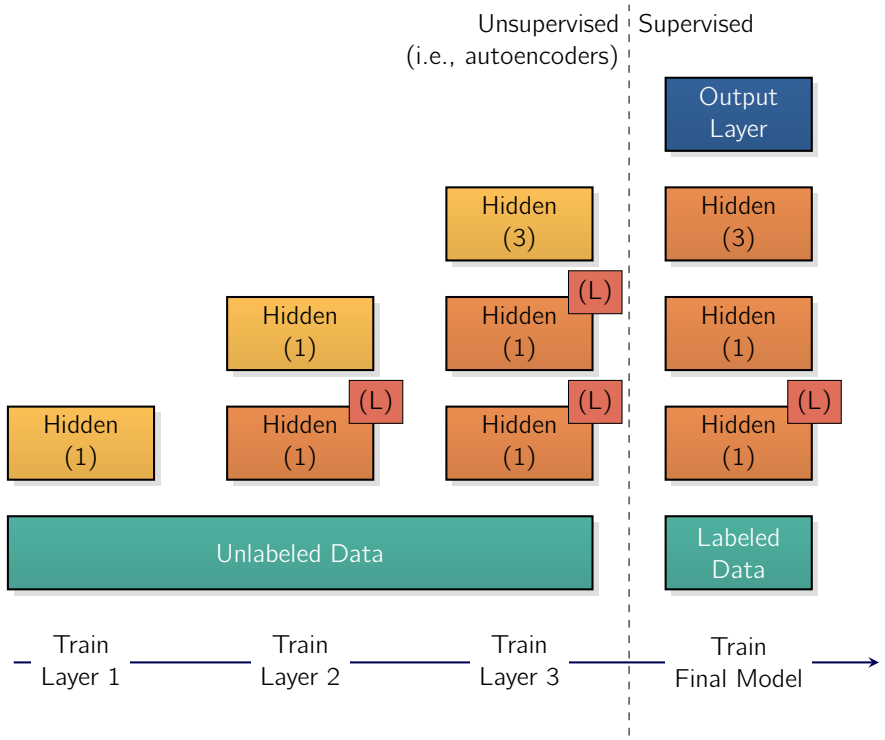


Figure 3.6 Greedy layer-wise pretraining used in the early days of deep learning; nowadays the unsupervised part is typically done in one shot on all the data rather than one layer at a time. Here (L) stands for locked layers.

generally train the full unsupervised model in one shot and use models such as autoencoders or diffusion models rather than RBMs.

3.1.9 Pretraining on an Auxiliary Task

If we do not have much labeled training data, one last option is to train a first ANN on an auxiliary task for which we can easily obtain or generate labeled training data, then reuse the lower layers of that network for our actual task.

> The first ANN's lower layers will learn feature detectors that will likely be reusable by the second ANN.

For example, if we want to build a system to recognize faces, we may only have a few pictures of each individual which is clearly not enough to train a good classifier. Gathering hundreds of pictures of each person would not be practical either.

We could, however, use a public dataset containing millions of pictures of people[46] and train a first ANN to detect whether two different pictures feature the same person. Such a network would learn good feature detectors for faces, so reusing its lower layers would allow us to train a good face classifier which uses little training data.

[46] such as VGGFace2.

Web-scraping 3.8 Information

We could also just scrape pictures of random people from the web, but this would probably be **illegal**.

- Firstly, photos are usually copyrighted by their creators, and websites like Instagram or Facebook enforce these copyright protections through their terms of service, which prohibit scraping and unauthorized use.
- Secondly, over 40 countries require explicit consent for collecting and processing personal data, including facial images.

For NLP applications, we can download a corpus of millions of text documents and automatically generate labeled data from it.

> For example, you could randomly mask out some words and train a model to predict what the missing words are.[47]

[47] e.g., it should predict that the missing word in the sentence "What ___ you saying?" is probably "are" or "were".

If we can train a model to reach good performance on this task, then it will already know quite a lot about language, and we can certainly reuse it for your actual task and fine-tune it on our labeled data.[48]

[48] this is basically how large language models are trained and fine-tuned, as we will see in Chapter **??**.

Self-Supervised Learning 3.9 Information

Self-supervised learning is when you **automatically generate the labels from the data** itself, as in the text-masking example, then you train a model on the resulting "labeled" dataset using supervised learning techniques.

3.1.10 Faster Optimizers

Training a very large DNN can be painfully slow. So far we have seen four ways to speed up training:[49] applying a good initialization strategy for the connection weights, using a good activation function, using batch-norm or LN, and reusing parts of a pretrained network.[50] Another huge speed boost comes from using a faster optimizer than the regular gradient descent optimizer.

[49] and reach a better solution.

[50] possibly built for an auxiliary task or using unsupervised learning.

In this section we will present the most popular optimization algorithms:

> momentum, Nesterov accelerated gradient, AdaGrad, RMSProp, and finally, Adam and its variants.

Momentum

Imagine a bowling ball rolling down a gentle slope on a smooth surface: it will start out slowly, but it will quickly pick up momentum until it eventually reaches terminal velocity.[51] This is the core idea behind momentum optimization, proposed by *Boris Polyak* in 1964 [21].

[51] if there is some friction or air resistance.

In contrast, regular gradient descent will take small steps when the slope is gentle and big steps when the slope is steep, but it will never pick up speed.

> Regular gradient descent is generally much slower to reach the minimum than momentum optimization.

As we saw in Chapter 4, gradient descent updates the weights θ by directly subtracting the gradient of the cost function $J(\theta)$ with regard to the weights ($\nabla_\theta J(\theta)$) multiplied by the learning rate θ. The equation is:

$$\theta \leftarrow \theta - \eta \nabla_\theta J(\theta)$$

It does not care about what the earlier gradients were. If the local gradient is tiny, it goes very slowly.

Momentum optimization cares a great deal about what previous gradients were:

> at each iteration, it subtracts the local gradient from the *momentum vector* $\mathbf{m}$, multiplied by the learning rate η, and it updates the weights by adding this momentum vector, shown in Eq. (**3.5**).

In other words, the gradient is used as a force learning to an acceleration, not as a speed. To simulate some sort of friction mechanism and prevent the momentum from growing too large, the algorithm introduces a new hyperparameter β, called the *momentum coefficient*, which must be set between 0 (high friction) and 1 (no friction).

> A typical momentum value is 0.9.

$$\mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_\theta J(\theta) \qquad \text{and} \qquad \theta \leftarrow \theta + \mathbf{m} \tag{3.5}$$

We can verify that if the gradient remains constant, the terminal velocity[52] is equal to that gradient multiplied by the learning rate η multiplied by $1/(1\beta)$, ignoring the sign.

[52] i.e., the maximum size of the weight updates.

> For example, if $\beta = 0.9$, then the terminal velocity is equal to 10 times the gradient times the learning rate, so momentum optimization ends up going 10 times faster than gradient descent!

In practice, the gradients are not constant, so the speedup is not always as dramatic, but momentum optimization can escape from plateaus much faster than regular gradient descent. We saw in **B.Sc Data Science I** that when the inputs have very different scales, the cost function will look like an elongated bowl. Gradient descent goes down the steep slope quite fast, but then it takes a very long time to go down the valley. In contrast, momentum optimization will roll down the valley faster and faster until it reaches the bottom, i.e., the optimum. In DNNs that don't use batch-norm or layer-norm, the upper layers will often end up having inputs with very different scales, so using momentum optimization helps a lot. It can also help roll past local optima.

Damping is Always Good 3.10 Information

Due to the momentum, the optimizer may **overshoot** a bit, then come back, overshoot again, and oscillate like this many times before stabilizing at the minimum. This is one of the reasons why it's good to have a bit of friction in the system: it reduces these oscillations and thus speeds up convergence.

Implementing momentum optimization in PyTorch is simple. just use the SGD optimizer and set its momentum hyperparameter:

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), momentum=0.9, lr=0.05)
history_momentum, acc_momentum = test_optimizer(model, optimizer)  # extra code
```

The one drawback of momentum optimization is that it adds yet another hyperparameter to tune. However, the momentum value of 0.9 usually works well in practice and almost always goes faster than regular gradient descent.

Nesterov Accelerated Gradient

One small variant to momentum optimization, proposed by Yurii Nesterov in 1983 [22], is almost always faster than regular momentum optimization. The *Nesterov accelerated gradient* (NAG) method, also known as Nesterov momentum optimization, measures the gradient of the cost function not at the local position but slightly ahead in the direction of the momentum, at $\theta + \beta\mathbf{m}$:

$$\mathbf{m} \leftarrow \beta\mathbf{m} - \eta\nabla_\theta J(\theta + \beta\mathbf{m}) \quad \text{and} \quad \theta \leftarrow \theta + \mathbf{m}$$

This small tweak works because in general the momentum vector will be pointing in the right direction,[53] so it will be slightly more accurate to use the gradient measured a bit farther in that direction rather than the gradient at the original position, as you can see in **Fig.** 3.7.[54]

As we can see, the Nesterov update ends up closer to the optimum. After a while, these small improvements add up and NAG ends up being significantly faster than regular momentum optimization. Moreover, note that when the momentum pushes the weights across a valley, ∇_1 continues to push farther across the valley, while ∇_2 pushes back toward the bottom of the valley. This helps reduce oscillations and thus NAG converges faster.

To use NAG, simply set `nesterov=True` when creating the SGD optimizer:

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), momentum=0.9, lr=0.05)
history_momentum, acc_momentum = test_optimizer(model, optimizer)  # extra code
```

[53] i.e., toward the optimum.

[54] where ∇_1 represents the gradient of the cost function measured at the starting point θ, and ∇_2 represents the gradient at the point located at $\theta + \beta_m$.

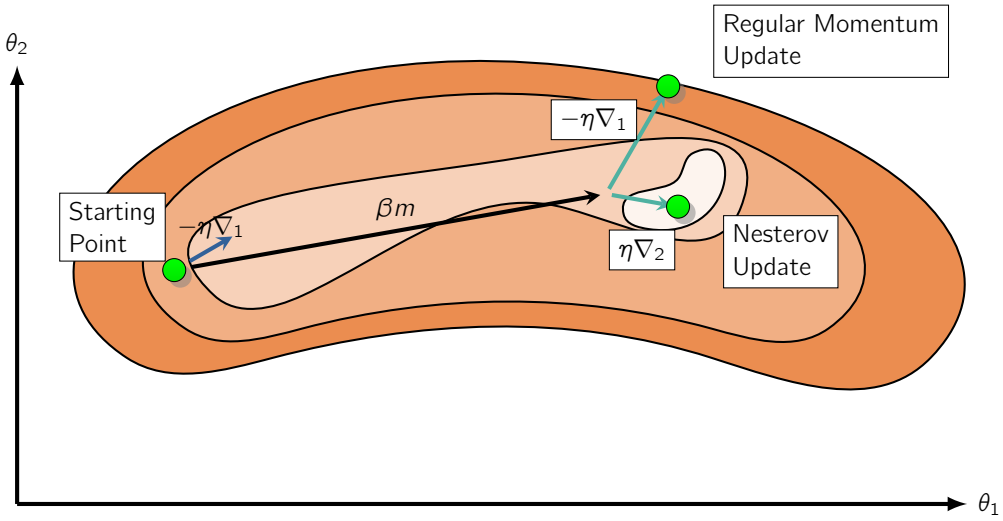


Figure 3.7
Regular versus Nesterov momentum optimization: the former applies the gradients computed before the momentum step, while the latter applies the gradients computed after.

3.1.11 AdaGrad

Consider the elongated bowl problem again: gradient descent starts by quickly going down the steepest slope, which does not point straight toward the global optimum, then it very slowly goes down to the bottom of the valley. It would be nice if the algorithm could correct its direction earlier to point a bit more toward the global optimum. The AdaGrad algorithm [23] achieves this correction by scaling down the gradient vector along the steepest dimensions (see Equation 11-7).

$$\mathbf{s} \leftarrow \mathbf{s} + \nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta) \qquad \text{and} \qquad \theta \leftarrow \theta - \eta \nabla_\theta J(\theta) \oslash \sqrt{s + \epsilon}$$

The first step accumulates the square of the gradients into the vector s.[55] This vectorized form is equivalent to computing

$$s_i \leftarrow s_i \left(\frac{\partial J(\theta)}{\partial \theta_i} \right)^2$$

for each element s_i of the vector $\mathbf{s}$; in other words, each s_i accumulates the squares of the partial derivative of the cost function with regard to parameter θ_i. If the cost function is steep along the ith dimension, then s_i will get larger and larger at each iteration.

[55] recall that the $\otimes$ symbol represents the element-wise multiplication.

The second step is almost identical to gradient descent, but with one big difference:

the gradient vector is scaled down by a factor of $\sqrt{\mathbf{s} + \varepsilon}$.[56]

This vectorized form is equivalent to simultaneously computing:

$$\theta_i \leftarrow \left(\frac{\partial J(\theta)}{\partial \theta_i} \right) \times \left(\frac{1}{\sqrt{s_i + \epsilon}} \right)$$

[56] the $\oslash$ symbol represents the element-wise division, the square root is also computed element-wise, and ϵ is a smoothing term to avoid division by zero, typically set to 1×10^{-10}.

In short, this algorithm decays the learning rate, but it does so faster for steep dimensions than for dimensions with gentler slopes. This is called an **adaptive learning rate**. It helps point the resulting updates more directly toward the global optimum (see Figure 11-8). One additional benefit is that it requires much less tuning of the learning rate hyperparameter η.

AdaGrad frequently performs well for simple quadratic problems, but it often stops too early when training neural networks:

> the learning rate gets scaled down so much that the algorithm ends up stopping entirely before reaching the global optimum.

So even though PyTorch has an `Adagrad` optimizer, we should not use it to train DNNs.[57] Still, understanding AdaGrad is helpful to comprehend the other adaptive learning rate optimizers.

[57] it may be efficient for simpler tasks such as linear regression, though.

3.1.12 RMSProp

As we've seen, AdaGrad runs the risk of slowing down a bit too fast and never converging to the global optimum. The RMSProp algorithm[58] fixes this by accumulating only the gradients from the most recent iterations, as opposed to all the gradients since the beginning of training.

[58] This algorithm was created by *Geoffrey Hinton* and *Tijmen Tieleman* in 2012 and presented by Geoffrey Hinton in his Coursera class on neural networks. Amusingly, since the authors did not write a paper to describe the algorithm, researchers often cite *slide 29 in lecture 6e* in their papers.

It does so by using exponential decay in the first step:

$$\mathbf{s} \leftarrow \alpha\mathbf{s}\,(1-\alpha)\,\nabla_\theta J(\theta) \otimes \nabla_{\theta J(\theta)} \qquad \text{and} \qquad \theta \leftarrow \theta - \eta\nabla_\theta \oslash \sqrt{\mathbf{s}+\epsilon}$$

The decay rate α is typically set to 0.9. Yes, it is once again a new hyperparameter, but this default value often works well, so you may not need to tune it at all.

As it is quite standard by now, PyTorch has an `RMSprop` optimizer:

```python
model = build_model()  # extra code
optimizer = torch.optim.RMSprop(model.parameters(), alpha=0.9, lr=0.05)
history_rmsprop, acc_rmsprop = test_optimizer(model, optimizer)  # extra code
```

Except on very simple problems, this optimizer almost always performs much better than *AdaGrad*.

> It was the preferred optimization algorithm of many researchers until Adam optimization came around.

3.1.13 Adam

Adam [24], which stands for adaptive moment estimation, combines the ideas of momentum optimization and RMSProp: just like momentum optimization, it keeps track of an exponentially decaying average of past gradients; and just like RMSProp, it keeps track of an exponentially decaying average of past squared gradients (see Equation 11-9). These are estimations of the mean and (uncentered) variance of the gradients. The mean is often called the first moment, while the variance is often called the second moment, hence the name of the algorithm.

$$
\begin{aligned}
\mathbf{m} &\leftarrow \beta_1 \mathbf{m} - (1 - \beta_1)\nabla_\theta J(\theta) \\
\mathbf{s} &\leftarrow \beta_2 \mathbf{s} + (1 + \beta 2)\nabla_\theta J(\theta) \otimes \nabla_\theta J(\theta) \\
\hat{\mathbf{m}} &\leftarrow \frac{\mathbf{m}}{1 - \beta_1^t} \\
\hat{\mathbf{s}} &\leftarrow \frac{\mathbf{s}}{1 - \beta_2^t} \\
\theta &\leftarrow \theta + \eta \hat{\mathbf{m}} \oslash \sqrt{\hat{\mathbf{s}} + \epsilon}
\end{aligned}
\tag{3.6}
$$

In this equation, t represents the iteration number (starting at 1). If you just look at steps 1, 2, and 5, you will notice Adam's close similarity to both momentum optimization and RMSProp: β_1 corresponds to β in momentum optimization, and 2 corresponds to in RMSProp. The only difference is that step 1 computes an exponentially decaying average rather than an exponentially decaying sum, but these are actually equivalent except for a constant factor (the decaying average is just 1 1 times the decaying sum). Steps 3 and 4 are somewhat of a technical detail: since m and s are initialized at 0, they will be biased toward 0 at the beginning of training, so these two steps will help boost m and s at the beginning of training.

The momentum decay hyperparameter β_1 is typically initialized to 0.9, while the scaling decay hyperparameter β_2 is often initialized to 0.999. As earlier, the smoothing term ϵ is usually initialized to a tiny number such as 1×10^{-8}. These are the default values for the `Adam` class. Here is how to create an Adam optimizer using PyTorch:

```python
model = build_model()  # extra code
optimizer = torch.optim.Adam(model.parameters(), betas=(0.9, 0.999), lr=0.05)
history_adam, acc_adam = test_optimizer(model, optimizer)  # extra code
```

Since Adam is an adaptive learning rate algorithm, like AdaGrad and RMSProp, it requires less tuning of the learning rate hyperparameter η. We can often use the default value $\eta = 0.001$, making Adam even easier to use than gradient descent.

Finally, three variants of Adam are worth mentioning: AdaMax, NAdam, and AdamW.

AdaMax

The Adam paper also introduced AdaMax. Notice that in step 2 of Eq. (3.6), Adam accumulates the squares of the gradients in $\mathbf{s}$:[59] In step 5, if we ignore ϵ and steps 3 and 4, Adam scales down the parameter updates by the square root of $\mathbf{s}$.

[59] with a greater weight for more recent gradients.

In short, Adam scales down the parameter updates by the ℓ_2 norm of the time-decayed gradients.

> recall that the ℓ_2 norm is the square root of the sum of squares.

AdaMax replaces the ℓ_2 norm with the ℓ_∞ norm. Specifically, it replaces step 2 in Eq. (3.6) with:

$$\mathbf{s} \leftarrow \max\left(\beta_2 \mathbf{s},\ \text{abs}\left(\nabla_\theta J(\theta)\right)\right)$$

it drops step 4, and in step 5 it scales down the gradient updates by a factor of **s**, which is the max of the absolute value of the time-decayed gradients.

In practice, this can make AdaMax more stable than Adam, but it really depends on the dataset, and in general Adam performs better. So, this is just one more optimizer you can try if you experience problems with Adam on some task.

NAdam

NAdam optimization is *Adam* optimization with the inclusion of the *Nesterov* trick, so it will often converge **slightly faster** than *Adam*. In [25], where this technique was introduced, the Timothy Dozat compares many different optimizers on various tasks and finds that *NAdam* generally outperforms Adam but is sometimes outperformed by *RMSProp*.

AdamW

AdamW [26] is a variant of Adam, integrating a regularization technique called **weight decay**. Weight decay reduces the size of the model's weights at each training iteration by multiplying them by a decay factor such as 0.99. This may remind you of ℓ_2 regularization, which also aims to keep the weights small, and indeed it can be shown mathematically that ℓ_2 regularization is equivalent to weight decay when using SGD. However, when using Adam or its variants, ℓ_2 regularization and weight decay are not equivalent:

> in practice, combining Adam with ℓ_2 regularization results in models that often don't generalize as well as those produced by SGD.

AdamW fixes this issue by properly combining Adam with weight decay.

Knowing the Tools of the Trade — 3.11 Information

Adaptive optimization methods[60] are often great, converging fast to a good solution. However, in [27], the authors showed that they can lead to solutions which generalize poorly on some datasets.

So when you are disappointed by your model's performance, try using NAG instead because your dataset may just be allergic to adaptive gradients.

[60] which includes RMSProp, Adam, AdaMax, NAdam, and AdamW optimization.

To use NAdam, AdaMax, or AdamW in PyTorch, replace `torch.optim.Adam` with `torch.optim.NAdam`, `torch.optim.Adamax`, or `torch.optim.AdamW`. For AdamW, you probably want to tune the `weight_decay` hyperparameter.

All the optimization techniques discussed so far only rely on the first-order partial derivatives.[61] The optimization literature also contains amazing algorithms based on the second-order partial derivatives.[62]

[61] Jacobians, which measure the slope of the loss function along each axis.

[62] The Hessians, which are the partial derivatives of the Jacobians, measuring how each Jacobian changes along each axis; in other words, measuring the loss function's curvature.

Unfortunately, these Hessian-based algorithms are hard to apply directly to deep neural networks because there are n^2 second-order derivatives per output, where n is the number of parameters, as opposed to just n number of first-order derivatives per output.

> Since DNNs typically have hundreds of thousands of parameters or more, the 2nd optimization algorithms often don't even fit in memory, and even when they do, computing the Hessian matrix is just too slow.

Luckily, it is possible to use stochastic methods which can efficiently approximate second-order information. One such algorithm is Shampoo [28], which uses accumulated gradient information to approximate the second-order terms, similar to how Adam accumulates first-order statistics. It is not included in the PyTorch library, but we can get it in the PyTorch-Optimizer library (`pip install torch_optimizer`).

Training Sparse Models 3.12 Information

All the optimization algorithms we just discussed produce dense models, meaning that most parameters will be nonzero. If you need a blazingly fast model at runtime, or if you need it to take up less memory, you may prefer to end up with a sparse model instead.

One way to achieve this is to train the model as usual, then get rid of the tiny weights by setting them to zero, using `torch.nn.utils.prune.l1_unstructured()`. Or you can get rid of entire neurons, channels, or layers, not just individual weights, using `torch.nn.utils.prune.ln_structured()`, or other functions in the `torch.nn.utils.prune` package.

You should generally also apply fairly strong sparsity inducing regularization during training, such as ℓ_1 regularization, since it pushes the optimizer to zero out as many weights as it can. You can also try scaling down random weights during initialization to encourage sparsity.

3.2 Learning Rate Scheduling

Finding a good learning rate is very important. If you set it too high, training will diverge. If you set it too low, then training will be painfully slow, and it may also get stuck in a local optimum and produce a suboptimal model. If you set the learning rate fairly high, then training will often make rapid progress at first, but it will end up dancing around the optimum toward the end of training and thereby produce a suboptimal model. If you find a really good learning rate, you can end up with an excellent model, but training will generally be a bit too slow. Luckily, you can do better than a constant learning rate. In particular, it's a good idea to start with a fairly high learning rate and then reduce it toward the end of training (or whenever progress stops):

> this ensures that training starts fast, while also allowing backprop to settle down toward the end to really fine-tune the model parameters (see Figure 11-9).

There are various other strategies to tweak the learning rate during training. These are called **learning schedules**. The `torch.optim.lr_scheduler` module provides several implementations of common learning schedules. Let's look at the most important ones, starting with exponential scheduling.

3.2.1 Exponential Scheduling

The `ExponentialLR` class implements exponential scheduling, whereby the learning rate is multiplied by a constant factor `gamma` at some regular interval, typically at every epoch. As a result, after the nth epoch, the learning rate will be equal to the initial learning rate times `gamma` to the power of n. This factor gamma is yet another hyperparameter we can tune. In general, you will want to set gamma to a value lower than 1, but fairly close to 1 to avoid decreasing the learning rate too fast.

> For example, if gamma is set to 0.9, then after 10 epochs the learning rate will be about 35% of the initial learning rate, and after 20 epochs it will be about 12%.

The `ExponentialLR` constructor expects at least two argumentsthe optimizer whose learning rate will be tweaked during training, and the factor gamma:

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  # or any other optim.
exp_scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, gamma=0.9)
```

Next, you must update the training loop to call `scheduler.step()` at the end of each epoch to tweak the optimizer's learning rate:

```python
def train_with_scheduler(model, optimizer, loss_fn, metric, train_loader,
                         valid_loader, n_epochs, scheduler):
    history = {"train_losses": [], "train_metrics": [], "valid_metrics": []}
```

Continues on next page →

← Continued from previous page

```
    for epoch in range(n_epochs):
        losses = []
        metric.reset()
        for X_batch, y_batch in train_loader:
            model.train()
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            losses.append(loss.item())
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
            metric.update(y_pred, y_batch)
        history["train_losses"].append(np.mean(losses))
        history["train_metrics"].append(metric.compute().item())
        val_metric = evaluate_tm(model, valid_loader, metric).item()
        history["valid_metrics"].append(val_metric)
        print(f"Epoch {epoch + 1}/{n_epochs}, "
              f"train loss: {history['train_losses'][-1]:.4f}, "
              f"train metric: {history['train_metrics'][-1]:.4f}, "
              f"valid metric: {history['valid_metrics'][-1]:.4f}")
        if isinstance(scheduler, torch.optim.lr_scheduler.ReduceLROnPlateau):
            scheduler.step(val_metric)
        else:
            scheduler.step()
        print(f"Learning rate: {scheduler.get_last_lr()[0]:.5f}")
    return history

def test_scheduler(model, optimizer, scheduler, n_epochs=10, batch_size=32):
    train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
    valid_loader = DataLoader(valid_set, batch_size=32)
    test_loader = DataLoader(test_set, batch_size=32)
    xentropy = nn.CrossEntropyLoss()
    metric = torchmetrics.Accuracy(task="multiclass", num_classes=10).to(device)
    history = train_with_scheduler(
        model, optimizer, xentropy, metric, train_loader, valid_loader,
        n_epochs, scheduler)
    return history, evaluate_tm(model, test_loader, metric)
```

If we interrupt training and you later want to resume it where we left off, you should set the `last_epoch` argument of the scheduler's constructor to the last epoch you ran (zero-indexed). The default is `1`, which makes the scheduler start training from scratch.

3.2.2 Cosine Annealing

Instead of decreasing the learning rate exponentially, we can use the cosine function to go from the maximum learning rate max at the start of training, down to the minimum learning rate min at the end. This is called **cosine annealing**. Compared to exponential scheduling, cosine annealing ensures that the learning rate remains fairly high during most of training, while getting closer to the minimum near the end (see Figure 11-10). All in all, cosine annealing generally performs better. The

learning rate at epoch t (zero-indexed) is given by Equation 11-10, where Tmax is the maximum number of epochs.

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{t}{T_{\max}}\pi\right)\right)$$

PyTorch includes the `CosineAnnealingLR` scheduler, which you can create as follows (`T_max` is Tmax and `eta_min` is min). You can then use it just like the ExponentialLR scheduler:

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  # or any other optim.
cosine_scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=20, eta_min=0.001)
```

One problem with cosine annealing is that we have to set two new hyperparameters, $T_{\max}$ and $\eta_{\min}$, and it's not easy to know in advance how many epochs to train and when to stop decreasing the learning rate. This is why it is preferred to use the performance scheduling technique.

3.2.3 Performance Scheduling

Performance scheduling, also called **adaptive scheduling**, is implemented by PyTorch's `ReduceLROnPlateau` scheduler:

> it keeps track of a given metric during training-typically the validation loss-and if this metric stops improving for some time, it multiplies the learning rate by some factor.

This scheduler has quite a few hyperparameters, but the default values work well for most of them. We may occasionally need to tweak the following

`mode`
: If the tracked metric must be maximized, such as the validation accuracy,, then you must set the mode to `'max'`. The default is `'min'`, which is fine if the tracked metric must be minimized,such as the validation loss.

`patience`
: The number of consecutive steps (typically epochs) to wait for improvement in the monitored metric before reducing the learning rate. It defaults to 10, which is generally fine. If each epoch is very long, then you may want to reduce this value. factor

`factor`
: The factor by which the learning rate will be multiplied whenever the monitored metric fails to improve for too long. It defaults to 0.1, again a reasonable default, but perhaps a bit small in some cases.

For example, let's implement performance scheduling based on the validation accuracy:[63]

[63] i.e., which we want to maximize.

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  # or any other optim.
perf_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="max", patience=2, factor=0.1)
```

The training loop needs to be tweaked again because we must evaluate the desired metric at each epoch,[64] and we must then pass the result to the scheduler's `step()` method:

[64] in this example, we are using the `evaluate_tm()` function that we defined in Chapter 1

```python
# extra code
def train_with_warmup(model, optimizer, loss_fn, metric, train_loader,
                      valid_loader, n_epochs, warmup_scheduler, scheduler):
    history = {"train_losses": [], "train_metrics": [], "valid_metrics": []}
    for epoch in range(n_epochs):

        warmup_scheduler.step()

        # extra code
        losses = []
        metric.reset()

        for X_batch, y_batch in train_loader:

            # extra code
            model.train()
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            losses.append(loss.item())
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
            metric.update(y_pred, y_batch)
        history["train_losses"].append(np.mean(losses))
        history["train_metrics"].append(metric.compute().item())
        val_metric = evaluate_tm(model, valid_loader, metric).item()
        history["valid_metrics"].append(val_metric)
        print(f"Epoch {epoch + 1}/{n_epochs}, "
              f"train loss: {history['train_losses'][-1]:.4f}, "
              f"train metric: {history['train_metrics'][-1]:.4f}, "
              f"valid metric: {history['valid_metrics'][-1]:.4f}")

        if epoch >= 3:  # deactivate other scheduler(s) during warmup
            scheduler.step(val_metric)

        # extra code
        print(f"Learning rate: {scheduler.get_last_lr()[0]:.5f}")
    return history

def test_warmup_scheduler(model, optimizer, warmup_scheduler, scheduler,
                          n_epochs=10, batch_size=32):
    train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
    valid_loader = DataLoader(valid_set, batch_size=32)
    test_loader = DataLoader(test_set, batch_size=32)
```

Continues on next page →

← Continued from previous page

```python
    xentropy = nn.CrossEntropyLoss()
    metric = torchmetrics.Accuracy(task="multiclass", num_classes=10).to(device)
    history = train_with_warmup(
        model, optimizer, xentropy, metric, train_loader, valid_loader,
        n_epochs, warmup_scheduler, scheduler)
    return history, evaluate_tm(model, test_loader, metric)
```

3.2.4 Warming Up the Learning Rate

So far, we have always started training with the maximum learning rate. However, this can sometimes cause gradient descent to bounce around randomly at the beginning of training, neither exploding nor making any significant progress. This typically happens with sensitive models, such as recurrent neural networks (Chapter **??**), or when using a very large batch size. In such cases, one solution is to **warm up** the learning rate, starting close to zero and gradually increasing the learning rate over a few epochs, up to the maximum learning rate.

> During this warm-up phase, gradient descent has time to stabilize into a better region of the loss landscape, where it can then make quick progress using a high learning rate.

Why does this work? Well, the loss landscape sometimes resembles the Himalayas:

> it's very high up and full of gigantic spikes. If you start with a high learning rate, you might jump from one mountain peak to the next for a very long time. If instead you start with a small learning rate, you will just walk down the mountain and valleys and escape the spiky mountain range altogether until you reach flatter lands. From then on, you can use a large learning rate for the rest of your journey, slowing down only toward the end.

A common way to implement learning rate warm up using PyTorch is to use a LinearLR scheduler to increase the learning rate linearly over a few epochs. For example, the following scheduler will increase the learning rate from 10% to 100% of the optimizer's original learning rate over 3 epochs (i.e., 10% during the first epoch, 40% during the second epoch, 70% during the third epoch, and 100% after that):

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  # or any other optim.
warmup_scheduler = torch.optim.lr_scheduler.LinearLR(
    optimizer, start_factor=0.1, end_factor=1.0, total_iters=3)
```

If you would like more flexibility, you can write your own custom function and wrap it in a `LambdaLR` scheduler. For example, the following scheduler is equivalent to the `LinearLR` scheduler we just defined:

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  # or any other optim.
warmup_scheduler = torch.optim.lr_scheduler.LambdaLR(
    optimizer,
    lambda epoch: (min(epoch, 3) / 3) * (1.0 - 0.1) + 0.1)
```

You must then insert `warmup_scheduler.step()` at the beginning of each epoch, and make sure you deactivate the scheduler(s) you are using for the rest of training during the warm-up phase. And that's all!

```python
# extra code
def train_with_warmup(model, optimizer, loss_fn, metric, train_loader,
                      valid_loader, n_epochs, warmup_scheduler, scheduler):
    history = {"train_losses": [], "train_metrics": [], "valid_metrics": []}
    for epoch in range(n_epochs):

        warmup_scheduler.step()

        # extra code
        losses = []
        metric.reset()

        for X_batch, y_batch in train_loader:

            # extra code
            model.train()
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            losses.append(loss.item())
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()
            metric.update(y_pred, y_batch)
        history["train_losses"].append(np.mean(losses))
        history["train_metrics"].append(metric.compute().item())
        val_metric = evaluate_tm(model, valid_loader, metric).item()
        history["valid_metrics"].append(val_metric)
        print(f"Epoch {epoch + 1}/{n_epochs}, "
              f"train loss: {history['train_losses'][-1]:.4f}, "
              f"train metric: {history['train_metrics'][-1]:.4f}, "
              f"valid metric: {history['valid_metrics'][-1]:.4f}")

        if epoch >= 3:  # deactivate other scheduler(s) during warmup
            scheduler.step(val_metric)

        # extra code
        print(f"Learning rate: {scheduler.get_last_lr()[0]:.5f}")
    return history

def test_warmup_scheduler(model, optimizer, warmup_scheduler, scheduler,
                          n_epochs=10, batch_size=32):
    train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
```

Continues on next page →

← Continued from previous page

```python
    valid_loader = DataLoader(valid_set, batch_size=32)
    test_loader = DataLoader(test_set, batch_size=32)
    xentropy = nn.CrossEntropyLoss()
    metric = torchmetrics.Accuracy(task="multiclass", num_classes=10).to(device)
    history = train_with_warmup(
        model, optimizer, xentropy, metric, train_loader, valid_loader,
        n_epochs, warmup_scheduler, scheduler)
    return history, evaluate_tm(model, test_loader, metric)
```

In short, you pretty much always want to cool down the learning rate at the end of training, and you may also want to warm it up at the beginning if gradient descent needs a bit of help getting started. But are there any cases where you may want to tweak the learning rate in the middle of training? Well yes, there are; for example, if gradient descent gets stuck in a local optimum or a high plateau. Gradient descent could remain stuck here for a long time, or even forever. Luckily, there's a way to escape this trap: just increase the learning rate for a little while.

You could spend your time staring at the learning curves during training, and manually interrupting it to tweak the learning rate when needed, but you probably have better things to do. Alternatively, you could implement a custom scheduler that monitors the validation metricmuch like the `ReduceL| ROnPlateau` schedulerand increases the learning rate for a while if the validation metric is stuck in a bad plateau. For this, you could subclass the `LRScheduler` base class. But a much simpler option is to use the cosine annealing with warm restarts learning schedule. Let's look at it now.

3.2.5 Cosine Annealing with Warm Restarts

Cosine annealing with warm restarts was introduced in a 2016 paper by Ilya Loshchilov and Frank Hutter [29]. This schedule just repeats the cosine annealing schedule over and over again. Since the learning rate regularly shoots back up, this schedule allows gradient descent to escape local optima and plateaus automatically. The authors recommend starting with a fairly short round of cosine annealing, but then doubling Tmax after each round (see Figure 11-11).

> This allows gradient descent to do a lot of quick explorations at the start of training, while also taking the time to properly optimize the model later during training, possibly escaping a plateau or two along the way.

Conveniently, PyTorch includes a `CosineAnnealingWarmRestarts` scheduler. You must set `T_0`, which is the value of T_{max} for the first round of cosine annealing. You may also set `T_mult` to 2 if you want to double T_{max} at each round.[65] Finally, you can set `eta_min` which it defaults to 0:

[65] the default is 1, meaning Tmax stays constant and all rounds have the same length.

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)  # or any other optim.
cosine_repeat_scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
    optimizer, T_0=2, T_mult=2, eta_min=0.001)

lrs = []
```

Continues on next page →

← Continued from previous page

```python
for epoch in range(62):
    lrs.append(cosine_repeat_scheduler.get_last_lr()[0])
    cosine_repeat_scheduler.step()

plt.figure(figsize=(8, 3))
plt.hlines([eta_min, eta_max], 0, 64, color="k", linestyle="dotted")
plt.plot(lrs, label="Cosine")
plt.text(30.5, eta_max - 0.005, r"$\eta_\text{max}$")
plt.text(58, eta_min + 0.003, r"$\eta_\text{min}$")
plt.axis([0, 62, 0, eta_max * 1.05])
plt.grid(True)
plt.xlabel("Epoch")
plt.ylabel("Learning rate")
plt.show()
```

3.2.6 1cycle Scheduling

Yet another popular learning schedule is *1cycle*, introduced in a 2018 paper by Leslie Smith [30]. It starts by warming up the learning rate, starting at η_0 and growing linearly up to η_1 halfway through training. Then it decreases the learning rate linearly down to η_0 again during the second half of training, finishing the last few epochs by dropping the rate down by several orders of magnitude (still linearly).

The maximum learning rate η_1 is chosen using the same approach we used to find the optimal learning rate, and the initial learning rate η_0 is usually 10 times lower. When using a momentum, we start with a high momentum first (e.g., 0.95), then drop it down to a lower momentum during the first half of training (e.g., down to 0.85, linearly), and then bring it back up to the maximum value (e.g., 0.95) during the second half of training, finishing the last few epochs with that maximum value. Smith did many experiments showing that this approach was often able to speed up training considerably and reach better performance.

For example, on the popular CIFAR10 image dataset, this approach reached 91.9% validation accuracy in just 100 epochs, compared to 90.3% accuracy in 800 epochs through a standard approach, using the same NN architecture. This feat was dubbed **super-convergence**.

PyTorch implements this schedule in the `OneCycleLR` scheduler.

Which Scheduler to Pick — 3.13 Information

If you are not sure which learning schedule to use, **1cycle** can be a good default, but I tend to have more luck with performance scheduling. If you run into instabilities at the start of training, try adding learning rate warm-up. And if training gets stuck on plateaus, try **cosine annealing** with warm restarts.

We have now covered the most popular learning schedules, but PyTorch offers a few extra schedulers:

such as, a polynomial scheduler, a cyclic scheduler, a scheduler that makes it easy to chain other schedulers, and a few more

so make sure to check out the documentation.

Now let's move on to one final topic before we complete this chapter on DL training techniques: **regularization**. DL is highly prone to overfitting, so regularization is key.

3.3 Avoiding Overfitting Through Regularization

DNNs typically have tens of thousands of parameters, sometimes even millions or billions. This gives them an incredible amount of freedom and means they can fit a huge variety of complex datasets. But this great flexibility also makes the network prone to overfitting the training set. Regularization is often needed to prevent this.

We already implemented a common regularization technique in **B.Sc Data Science I**: **early stopping**. Moreover, even though batch-norm and layer-norm were designed to solve the unstable gradients problems, they also act like pretty good regularizers. In this section we will examine other popular regularization techniques for neural networks: ℓ_1 and ℓ_2 regularization, dropout, MC dropout, and max-norm regularization.

3.3.1 ℓ_1 and ℓ_2 Regularization

Just like we did in **B.Sc Data Science I** for simple linear models, we can use ℓ_2 regularization to constrain a NN's connection weights, and/or ℓ_1 regularization if you want a sparse model (with many weights equal to 0). As we saw earlier (when discussing the AdamW optimizer), ℓ_2 regularization is mathematically equivalent to weight decay when using an SGD optimizer, with or without momentum, so if that's the case you can implement ℓ_2 regularization by simply setting the optimizer's `weight_decay` argument. For example, here is how to apply ℓ_2 regularization to the connection weights of a PyTorch model trained using SGD, with a regularization factor of 1×10^{-4}:

```python
model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), lr=0.05, weight_decay=1e-4)
history = test_optimizer(model, optimizer)
```

If instead you are using an Adam optimizer, you should switch to AdamW and set the `weight_decay` argument. This is not exactly equivalent to ℓ_2 regularization, but as we saw earlier it's pretty close and it works better.

> Weight decay is applied to every model parameter, including bias terms, and even parameters of batch-norm and layer-norm layers.

Generally that's not a big deal, but penalizing these parameters does not contribute much to regularization and it may sometimes negatively impact training performance. So how can we apply weight decay to some model parameters and not others? One approach is to implement ℓ_2 regularization manually, without relying on the optimizer's weight decay feature. For this, you must tweak the training loop to manually compute the ℓ_2 loss based only on the parameters you want, and add this ℓ_2 loss to the main loss:

```python
n_epochs = 3
train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
loss_fn = nn.CrossEntropyLoss()

model = build_model()  # extra code
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)
params_to_regularize = [
    param for name, param in model.named_parameters()
    if not "bias" in name and not "bn" in name]
for epoch in range(n_epochs):
    total_loss = 0.0
    total_l2_loss = 0.0
    for X_batch, y_batch in train_loader:
        model.train()
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        y_pred = model(X_batch)
        main_loss = loss_fn(y_pred, y_batch)
        l2_loss = sum(param.pow(2.0).sum() for param in params_to_regularize)
        loss = main_loss + 1e-4 * l2_loss
        total_loss += loss.item()
        total_l2_loss += l2_loss.item()
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    print(f"Epoch {epoch + 1}/{n_epochs}"
          f"  l2 loss: {total_l2_loss / len(train_loader):.3f}"
          f"  loss: {total_loss / len(train_loader):.3f}")
```

Another approach is to use PyTorch's *parameter groups* feature, which lets the optimizer apply different hyperparameters to different groups of model parameters. So far, we have always created optimizers by passing them the full list of model parameters:

> PyTorch automatically put them all in a single parameter group, sharing the same hyperparameters.

Instead, we can pass a list of dictionaries to the optimizer, each with a `"params"` entry containing a list of parameters, and (optionally) some hyperparameter key/value pairs specific to this group of parameters. The group-specific hyperparameters take precedence over the optimizer's global hyperparameters. For example, let's create an optimizer with two parameter groups:

> the first group will contain all the parameters we want to regularize and it will use weight decay, while the second group will contain all the bias terms and BN parameters, and it will not use weight decay at all.

```python
n_epochs = 3
train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
loss_fn = nn.CrossEntropyLoss()

model = build_model()  # extra code
params_to_regularize = [
    param for name, param in model.named_parameters()
```

Continues on next page →

← Continued from previous page

```python
    if not "bias" in name and not "bn" in name]
params_bias_and_bn = [
    param for name, param in model.named_parameters()
    if "bias" in name or "bn" in name]
optimizer = torch.optim.SGD([
    {"params": params_to_regularize, "weight_decay": 1e-4},
    {"params": params_bias_and_bn, "weight_decay": 0},
    ], lr=0.05)
for epoch in range(n_epochs):
    total_loss = 0.0
    for X_batch, y_batch in train_loader:
        model.train()
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        total_loss += loss.item()
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    print(f"Epoch {epoch + 1}/{n_epochs}"
          f"  loss: {total_loss / len(train_loader):.3f}")
```

Benefits of Parameters Groups 3.14 Information

Parameter groups also allow you to apply different learning rates to different parts of your model. This is most common for **transfer learning**, when you want new layers to be updated faster than reused ones.

Now how about ℓ_1 regularization? Well unfortunately PyTorch does not provide any helper for this, so you need to implement it manually, much like we did for ℓ_2 regularization. This means tweaking the training loop to compute the ℓ_1 loss and adding it to the main loss:

```python
n_epochs = 3
train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
loss_fn = nn.CrossEntropyLoss()

model = build_model()  # extra code
params_to_regularize = [
    param for name, param in model.named_parameters()
    if not "bias" in name and not "bn" in name]
params_bias_and_bn = [
    param for name, param in model.named_parameters()
    if "bias" in name or "bn" in name]
optimizer = torch.optim.SGD([
    {"params": params_to_regularize, "weight_decay": 1e-4},
    {"params": params_bias_and_bn, "weight_decay": 0},
    ], lr=0.05)
for epoch in range(n_epochs):
    total_loss = 0.0
    for X_batch, y_batch in train_loader:
        model.train()
```

Continues on next page →

← Continued from previous page

```python
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        y_pred = model(X_batch)
        loss = loss_fn(y_pred, y_batch)
        total_loss += loss.item()
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    print(f"Epoch {epoch + 1}/{n_epochs}"
          f"  loss: {total_loss / len(train_loader):.3f}")
```

That's all there is to it! Now let's move on to Dropout, which is one of the most popular regularization techniques for deep neural networks.

3.3.2 Dropout

Dropout was proposed in a paper [31] by *Geoffrey Hinton et al.* in 2012 and further detailed in a 2014 paper [32] by *Nitish Srivastava et al.*, and it has proven to be highly successful: many state-of-the-art NNs use dropout, as it gives them a 1% to 2% accuracy boost. This may not sound like a lot, but when a model already has 95% accuracy, getting a 2% accuracy boost means dropping the error rate by almost 40% (going from 5% error to roughly 3%).

It is a fairly simple algorithm: at every training step, every neuron (including the input neurons, but always excluding the output neurons) has a probability p of being temporarily dropped out, meaning it will be entirely ignored during this training step, but it may be active during the next step (see Figure 11-12). The hyperparameter p is called the dropout rate, and it is typically set between 10% and 50%: closer to 20% - 30% in recurrent neural nets (see Chapter **??**), and closer to 40%50% in CNNs (see Chapter **??**). After training, neurons don't get dropped anymore.

It's surprising at first that this destructive technique works at all.

> Would a company perform better if its employees were told to toss a coin every morning to decide whether to go to work?

Well, who knows; perhaps it would! The company would be forced to adapt its organization; it could not rely on any single person to work the coffee machine or perform any other critical tasks, so this expertise would have to be spread across several people. Employees would have to learn to cooperate with many of their coworkers, not just a handful of them. The company would become much more resilient. If one person quit, it wouldn't make much of a difference. It's unclear whether this idea would actually work for companies, but it certainly does for neural networks. Neurons trained with dropout cannot co-adapt with their neighboring neurons; they have to be as useful as possible on their own. They also cannot rely excessively on just a few input neurons; they must pay attention to all of their input neurons. They end up being less sensitive to slight changes in the inputs.

> In the end, you get a more robust network that generalizes better.

Another way to understand the power of dropout is to realize that a unique neural network is generated at each training step. Since each neuron can be either present or absent, there are a total of 2N possible networks (where N is the total number of droppable neurons). This is such a huge number that it is virtually impossible for the same neural network to be sampled twice. Once you have run 10,000 training steps, you have essentially trained 10,000 different neural networks, each with just one training instance. These neural networks are obviously not independent because they share many of their weights, but they are nevertheless all different. The resulting neural network can be seen as an averaging ensemble of all these smaller neural networks.

Different Layers, Different Behaviors 3.15 Information

Higher layers, which learn more complex feature combinations, benefit more from dropout because they are more prone to overfitting. So you can usually apply dropout only to the neurons of the top hidden layers (e.g., one to three hidden layers).

However, you should avoid dropping the output neurons, as this would be like changing the task during training: it wouldn't help.

There is one small but important technical detail. Suppose p = 75%: on average only 25% of all neurons are active at each step during training. This means that after training, each neuron receives four times more inputs than during training, on average. This discrepancy is so large that the model is unlikely to work well. To avoid this issue, a simple solution is to multiply the inputs by 4 during training, which is the same as dividing them by 25. More generally, we need to divide the inputs by the keep probability (1 p) during training.

To implement dropout using PyTorch, you can use the `nn.Dropout` layer. It's important to switch to training mode during training, and to evaluation mode during evaluation (just like for batch norm). In training mode, the layer randomly drops some inputs (setting them to 0) and divides the remaining inputs by the keep probability. In evaluation mode, it does nothing at all; it just passes the inputs to the next layer. The following code applies dropout regularization before every nn.Linear layer, using a dropout rate of 0.2:

```python
torch.manual_seed(42)
model = nn.Sequential(
    nn.Flatten(),
    nn.Dropout(p=0.2), nn.Linear(28 * 28, 100), nn.ReLU(),
    nn.Dropout(p=0.2), nn.Linear(100, 100), nn.ReLU(),
    nn.Dropout(p=0.2), nn.Linear(100, 10)
).to(device)
model.apply(use_he_init)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
history_dropout, acc_dropout = test_optimizer(model, optimizer, batch_size=32)
```

Since dropout is only active during training, comparing the training loss and the validation loss can be misleading. In particular, a model may be overfitting the training set and yet have similar

training and validation losses. So make sure to evaluate the training loss without dropout (e.g., after training).

If you observe that the model is overfitting, you can increase the dropout rate. Conversely, you should try decreasing the dropout rate if the model underfits the training set. It can also help to increase the dropout rate for large layers, and reduce it for small ones. Moreover, many state-of-the-art architectures only apply dropout to the last few hidden layers, so you may want to try this if full dropout is too strong. Dropout does tend to significantly slow down convergence, but it often results in a better model when tuned properly. So it is generally well worth the extra time and effort, especially for large models.

If you want to regularize a self-normalizing network based on the SELU activation function, you should use alpha dropout: this is a variant of dropout that preserves the mean and standard deviation of its inputs. It was introduced in the same paper as SELU, as regular dropout would break self-normalization.

PyTorch implements it in the `nn.AlphaDropout` layer.

3.3.3 Max-Norm Regularization

Another fairly popular regularization technique for neural networks is called *max-norm* regularization:

> for each neuron, it constrains the weights $\mathbf{w}$ of the incoming connections such that $||\mathbf{w}||_2 \leqslant r$, where r is the max-norm hyperparameter and $||\cdot||_2$ is the ℓ_2 norm.

Reducing r increases the amount of regularization and helps reduce overfitting. Max-norm regularization can also help alleviate the unstable gradients problems.[66] Rather than adding a regularization loss term to the overall loss function, max-norm regularization is typically implemented by computing $||\mathbf{w}||_2$ after each training step and rescaling $\mathbf{w}$ if needed ($\mathbf{w} \leftarrow \mathbf{w}r/||\mathbf{w}||_2$).

[66] if you are not using batch-norm or layer-norm.

Here's a common way to implement this in PyTorch:

```python
def apply_max_norm(model, max_norm=2, epsilon=1e-8, dim=1):
    with torch.no_grad():
        for name, param in model.named_parameters():
            if 'bias' not in name:
                actual_norm = param.norm(p=2, dim=dim, keepdim=True)
                target_norm = torch.clamp(actual_norm, 0, max_norm)
                param *= target_norm / (epsilon + actual_norm)
```

This function iterates through all of the model's weight matrices,[67] and for each one of them it uses the `norm()` method to compute the ℓ_2 norm of each row (`dim=1`). A `nn.Linear` layer has weights of shape `[number of neurons, number of inputs]`, so using `dim=1` means that we will get one norm per neuron, as desired. Then the function uses `torch.clamp()` to compute the target norm for each neuron's weights: this creates a copy of the `actual_norm` tensor, except that all values greater than `max_norm` are replaced by `max_norm`.[68] Lastly, we rescale the weight matrix so that

[67] i.e., all parameters except for the bias terms.

[68] this corresponds to r in the previous equation.

each column ends up with the target norm. Note that the smoothing term epsilon is used to avoid division by zero in case some columns have a norm equal to zero.

Next, all you need to do is call `apply_max_norm(model)` in the training loop, right after calling the optimizer's `step()` method. And of course you probably want to fine-tune the `max_norm` hyperparameter.

When using max-norm with layers other than `nn.Linear`, you may need to tweak the dim argument. For example, when using convolutional layers (see Chapter **??**), you generally want to set `dim=[1, 2, 3]` to limit the norm of each convolutional kernel.

3.3.4 Practical Guidelines

In this chapter we've covered a wide range of techniques, and you may be wondering which ones you should use. This depends on the task, and there is no clear consensus yet, but I have found the configuration in **Tbl.** 3.2 to work fine in most cases, without requiring much hyperparameter tuning. That said, please do not consider these defaults as hard rules!

Table 3.2 **Starting points for training DNNs.**

Hyperparameter	Default
Kernel initializer	He initialization
Activation function	ReLU if shallow; Swish if deep
Normalization	None if shallow; batch-norm or layer-norm if deep
Regularization	Early stopping; weight decay if needed
Optimizer	Nesterov accelerated gradients or AdamW
Learning rate	schedule Performance scheduling or 1cycle

You should also try to reuse parts of a pretrained neural network if you can find one that solves a similar problem, or use unsupervised pretraining if you have a lot of unlabeled data, or use pretraining on an auxiliary task if you have a lot of labeled data for a similar task.

While the previous guidelines should cover most cases, there are some exceptions:

- If you need a sparse model, use ℓ_1 regularization. You can also try zeroing out the smallest weights after training:

  For example, using the `torch.nn.utils.prune.l1_unstructured()` function

  This will break self-normalization, so you should use the default configuration in this case.

- If you need a low-latency model (one that performs lightning-fast predictions), you may need to use fewer layers; use a fast activation function such as `nn.ReLU`, `nn.LeakyReLU`, or

`nn.Hardswish`; and fold the batch-norm and layer-norm layers into the previous layers after training. Having a sparse model will also help.

Finally, you may want to reduce the float precision from 32 bits to 16 or even 8 bits. Appendix B covers several techniques to make models smaller and faster, including reduced precision models, mixed precision models, and quantization.

- If you are building a risk-sensitive application, or inference latency is not very important in your application, you can use MC dropout to boost performance and get more reliable probability estimates, along with uncertainty estimates.

Over the last three chapters, we have learned what artificial neural nets are, how to build and train them using Scikit-Learn and PyTorch, and a variety of techniques that make it possible to train deep and complex nets.

References

1. Basodi, S., Ji, C., Zhang, H.-P. & Pan, Y. Gradient Amplification: An efficient way to train deep neural networks. *Big Data Min. Anal.* **3,** 196–207 (2020).
2. Hochreiter, S. The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. *Int. J. Uncertain. Fuzziness Knowl. Based Syst.* **6,** 107–116 (1998).
3. Balduzzi, D. *et al.* The Shattered Gradients Problem: If resnets are the answer, then what is the question? *ArXiv* **abs/1702.08591** (2017).
4. Glorot, X. & Bengio, Y. *Understanding the difficulty of training deep feedforward neural networks* in *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (2010), 249–256.
5. He, K., Zhang, X., Ren, S. & Sun, J. *Delving deep into rectifiers: Surpassing human-level performance on imagenet classification* in *Proceedings of the IEEE international conference on computer vision* (2015), 1026–1034.
6. Saxe, A. M., McClelland, J. L. & Ganguli, S. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv preprint arXiv:1312.6120* (2013).
7. Maas, A. L., Hannun, A. Y., Ng, A. Y., *et al. Rectifier nonlinearities improve neural network acoustic models* in *Proc. icml* **30** (2013), 3.
8. Xu, B., Wang, N., Chen, T. & Li, M. Empirical Evaluation of Rectified Activations in Convolutional Network. *ArXiv* **abs/1505.00853** (2015).
9. Clevert, D.-A., Unterthiner, T. & Hochreiter, S. Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). *arXiv: Learning* (2015).
10. Klambauer, G., Unterthiner, T., Mayr, A. & Hochreiter, S. *Self-Normalizing Neural Networks* in *Neural Information Processing Systems* (2017).
11. Hendrycks, D. & Gimpel, K. Gaussian Error Linear Units (GELUs). *arXiv: Learning* (2016).
12. Ramachandran, P., Zoph, B. & Le, Q. V. Searching for Activation Functions. *ArXiv* **abs/1710.05941** (2018).

13. Shazeer, N. GLU Variants Improve Transformer. *ArXiv* **abs/2002.05202** (2020).

14. Dauphin, Y., Fan, A., Auli, M. & Grangier, D. *Language Modeling with Gated Convolutional Networks* in *International Conference on Machine Learning* (2016).

15. Misra, D. Mish: A Self Regularized Non-Monotonic Activation Function. *Proceedings of the British Machine Vision Conference 2020* (2020).

16. So, D. R. *et al.* Primer: Searching for Efficient Transformers for Language Modeling. *ArXiv* **abs/2109.08668** (2021).

17. Ioffe, S. & Szegedy, C. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift* in *International Conference on Machine Learning* (2015).

18. Gupta, D. & Dawn, S. Detection and Staging of Lung Cancer from CT scan Images by Deep Learning. *2023 International Conference on Disruptive Technologies (ICDT),* 274–278 (2023).

19. Ba, J., Kiros, J. R. & Hinton, G. E. Layer Normalization. *ArXiv* **abs/1607.06450** (2016).

20. Pascanu, R., Mikolov, T. & Bengio, Y. *On the difficulty of training recurrent neural networks* in *International Conference on Machine Learning* (2012).

21. Polyak, B. Some methods of speeding up the convergence of iteration methods. *Ussr Computational Mathematics and Mathematical Physics* **4,** 1–17 (1964).

22. Nesterov, Y. *A method for unconstrained convex minimization problem with the rate of convergence* $o(1/k^2)$ in (1983).

23. Duchi, J. C., Hazan, E. & Singer, Y. Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. *J. Mach. Learn. Res.* **12,** 2121–2159 (2011).

24. Kingma, D. P. & Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

25. Dozat, T. Incorporating nesterov momentum into adam (2016).

26. Loshchilov, I. & Hutter, F. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).

27. Wilson, A. C., Roelofs, R., Stern, M., Srebro, N. & Recht, B. The marginal value of adaptive gradient methods in machine learning. *Advances in neural information processing systems* **30** (2017).

28. Gupta, V., Koren, T. & Singer, Y. *Shampoo: Preconditioned stochastic tensor optimization* in *International Conference on Machine Learning* (2018), 1842–1850.

29. Loshchilov, I. & Hutter, F. SGDR: Stochastic Gradient Descent with Warm Restarts. *arXiv: Learning* (2016).

30. Smith, L. N. A disciplined approach to neural network hyper-parameters: Part 1 - learning rate, batch size, momentum, and weight decay. *ArXiv* **abs/1803.09820** (2018).

31. Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. Improving neural networks by preventing co-adaptation of feature detectors. *ArXiv* **abs/1207.0580** (2012).

32. Srivastava, N., Hinton, G. E., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. Dropout: a simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.* **15,** 1929–1958 (2014).

Acronyms

A

AI Artificial Intelligence

Artificial Intelligence

ANN Artificial Neural Networks

Artificial Neural Networks

API Application-Programming Interface

Application-Programming Interface

B

BGD Batch Gradient Descent

An iterative optimization algorithm that calculates the gradient of the cost function using the entire training dataset for each iteration.

BN Batch Normalization

A technique used in ANNs to make training faster and more stable by adjusting and scaling layer inputs.

C

CDF Cumulative Density Function

Gives the probability that a real-valued random variable X will take a value less than or equal to a given number x

CNN Convolutional Neural Networks

Convolutional Neural Networks

CPU Central Processing Unit

Central Processing Unit

D

DL Deep Learning

A subset of machine learning that uses multi-layered artificial neural networks to mimic human brain function. It automatically extracts complex patterns and abstract features from vast amounts of unstructured data, powering modern applications like computer vision, speech recognition, and large language models.

DNN Deep Neural Networks

Deep Neural Networks

E

ELU Exponential Linear Unit

A neural network activation function designed to improve learning speed and fix limitations in standard functions like ReLU

F

FNN Feedforward Neural Networks

Feedforward Neural Networks

G

GAN General Adveserial Networks

General Adveserial Networks

GD Gradient Descent

Gradient descent is a method for unconstrained mathematical optimization. It is a first-order iterative algorithm for minimizing a differentiable multivariate function.The idea is to take repeated steps in the opposite direction of the gradient of the function at the current point, because this is the direction of steepest descent. Conversely, stepping in the direction of the gradient will lead to a trajectory that maximizes that function; the procedure is then known as gradient ascent. Gradient descent should not be confused with local search algorithms, although both are iterative methods for optimization.

GPU Graphical Processing Unit

A specialized electronic chip designed to rapidly render images, video, and 3D animations. Unlike a central processing unit (CPU) that processes tasks sequentially, a GPU uses parallel processing with thousands of smaller cores to handle many calculations at the same time.

I

IC Integrated Circuit

A small electronic device combining numerous interconnected components, like transistors, resistors, and capacitors, onto a single semiconductor chip, usually silicon

L

LASSO Least Absolute Shrinkage and Selection Operator

Least Absolute Shrinkage and Selection Operator

LN Layer Normalization

A deep learning technique which stabilizes neural network training by calculating the mean and variance across the feature dimensions of each individual sample rather than across a mini-batch

LTU Linear Threshold Unit

Linear Threshold Unit

M

MAE Mean Absolute Error

Mean Absolute Error

ML Machine Learning

Machine Learning

MLP Multi-layer Perceptrons

Multi-layer Perceptrons

MNIST Modified National Institute of Standards and Technology

Modified National Institute of Standards and Technology

MSE Mean Square Error

Mean Square Error

N

NLP Natural Language Processing

Natural Language Processing

NN Neural Networks

Neural Networks

P

PRELU Parametric ReLU

An activation function in deep learning that extends the standard ReLU Activation Function by making the negative slope a learnable parameter during training

R

RAM Random Access Memory

A form of electronic computer memory that can be read and changed in any order, typically used to store working data and machine code.

RELU Rectified Linear Unit

A popular activation function in ANNs that outputs the input directly if it is positive, and returns zero if it is negative.

RMSE Root-Mean Square Error

Root-Mean Square Error

RNN Recurrent Neural Networks

A type of ANN designed to process sequential data, such as text, speech, or time series, by passing information through loops from one step to the next

RRELU Randomized ReLU

A stochastic activation function for neural networks where the negative slope is randomly sampled from a uniform distribution during training and then fixed to its average value during testing

S

SELU Scaled Exponential Linear Unit

A self-normalizing activation function that maintains a zero mean and unit variance across neural network layers

SGD Stochastic Gradient Descent

Stochastic Gradient Descent

SVM Support Vector Machines

Support Vector Machines

T

TLU Threshold Logic Unit

Threshold Logic Unit



The document is designed with no intention of publication and has only been designed for education purposes.

The current maintainer of this work along with the primary lecturer is D. T. McGuiness, Ph.D. (Daniel.McGuiness@mci.edu).

